

Universidad Nacional del Este.
Facultad Politécnica.



Sistema de Asistencia Basado en Reconocimiento Facial para la FPUNE.

Aldo Carrizo Coronel.
Jorge Daniel Sánchez Ramirez.

Año 2025.

**Universidad Nacional del Este.
Facultad Politécnica.**

**Licenciatura en Análisis de Sistemas.
TRABAJO FINAL DE GRADO II.**



Sistema de Asistencia Basado en Reconocimiento Facial para la FPUNE.

Por:

**Aldo Carrizo Coronel y
Jorge Daniel Sanchez Ramirez**

Profesor Orientador: **Ing. Jorge Luis Arrúa Ginés.**

Trabajo final de grado presentado a la Facultad Politécnica de la Universidad Nacional del Este como parte de los requisitos para optar al título Licenciatura en Análisis de Sistemas.

Ciudad del Este, Alto Paraná. Paraguay.

Noviembre 2025

FICHA CATALOGRÁFICA
BIBLIOTECA DE LA FACULTAD POLITÉCNICA
DE LA UNIVERSIDAD NACIONAL DEL ESTE

Carrizo Coronel, Aldo, 2002.

Sanchez Ramirez, Jorge Daniel, 2001.

Sistema de Asistencia Basado en Reconocimiento Facial para la FPUNE.

Ciudad del Este, Alto Paraná. Año: 2025.

Páginas: 149. Orientador: Ing. Jorge Luis Arrúa Ginés.

Área de estudio: Automatización y Control.

Carrera: Licenciatura en Análisis de Sistemas.

Titulación: Licenciado en Análisis de Sistemas.

Trabajo Final de Grado. Universidad Nacional del Este,

Facultad Politécnica.

Descriptores: 1. Reconocimiento facial, 2. Biometría,

3. Aprendizaje profundo, 4. Asistencia automatizada,

5. Visión por computadora, 6. Gestión académica.

Facial recognition-based assistance system for FPUNE.

Key words: 1. Facial recognition, 2. Biometrics,

3. Deep learning, 4. Automated attendance,

5. Computer vision, 6. Academic management.

Yo, Ing. Jorge Luis Arrúa Ginés, documento de identidad No. 2.281.931, Profesor Orientador del TFG titulado “Sistema de Asistencia Basado en Reconocimiento Facial para la FPUNE”, de los alumnos Aldo Carrizo Coronel y Jorge Daniel Sánchez Ramírez, documentos de identidad No. 7.374.786 y No. 5.144.692, respectivamente, de la carrera Licenciatura en Análisis de Sistemas de la Facultad Politécnica de la Universidad Nacional del Este; certifico que el mencionado Trabajo Final de Grado ha sido realizado por dichos alumnos, de lo cual doy fe y en mi opinión reúne las condiciones para su presentación y defensa ante la Mesa Examinadora designada por la institución.

17 de Noviembre de 2025

Ing. Jorge Luis Arrúa Ginés

Nosotros, los miembros de la Mesa Examinadora del Trabajo Final de Grado titulado “Sistema de Asistencia Basado en Reconocimiento Facial para la FPUNE”, de la carrera Licenciatura en Análisis de Sistemas de la Facultad Politécnica de la Universidad Nacional del Este, hacemos constar que el citado trabajo ha sido evaluado en fondo y forma por esta Mesa, la que por _____ ha resuelto asignar la calificación _____

Ciudad del Este, ____ de _____ de 2025

Profesor _____
Presidente de la Mesa Examinadora

Profesor _____
Miembro de la Mesa Examinadora

Profesor _____
Miembro de la Mesa Examinadora

Quiero comenzar expresando mi más profundo agradecimiento a mí mismo, por el esfuerzo, la perseverancia y la determinación que me acompañaron a lo largo de todos estos años de formación. Por cada día en que, a pesar del cansancio o la desmotivación, elegí continuar; por cada examen que enfrenté y por cada obstáculo que superé, incluso cuando las circunstancias o algunas experiencias académicas pusieron a prueba mis ganas de seguir.

Agradezco profundamente a mis padres, por su amor, apoyo incondicional y confianza en cada paso de este camino.

A mi pareja, por su comprensión, paciencia y por ser mi mayor motivación, así como por el acompañamiento constante y el apoyo incondicional que me brindaron la fortaleza necesaria para concluir este trabajo.

A todos ellos, y a quienes de alguna manera formaron parte de esta travesía, les extiendo mi más sincero agradecimiento por acompañarme en la concreción de este importante logro.

Jorge Daniel Sanchez Ramírez

Agradezco profundamente a las cuatro mujeres que marcaron mi destino: a mi madre Lourdes, quien me dio la vida, a mis madres Arminda y Carmen, K quienes me acompañaron en mis momentos más difíciles, me criaron con amor, dedicación y fortaleza. Gracias por su apoyo incondicional y por ser la base de todo lo que he logrado.

A mi familia, que me acompañó con aliento y comprensión, y a quienes ya no están pero fueron parte esencial de mi camino.

Agradezco sinceramente al Ing. Jorge Luis Arrúa Ginés por su orientación, dedicación y apoyo constante durante el desarrollo de este trabajo.

Porque, al final, somos una mezcla de todas las personas que han sido parte de nuestro camino. Me alegra saber que algunas de ellas son parte esencial de quien soy hoy.

Aldo Carrizo Coronel

“El éxito es la suma de pequeños esfuerzos repetidos día tras día.”

— Robert Collier

Resumen

Se presenta el desarrollo y la evaluación experimental de un prototipo de sistema automatizado de registro de asistencia mediante reconocimiento facial biométrico en la Facultad Politécnica de la Universidad Nacional del Este (FPUNE), Paraguay. El método manual actual consume entre 5 y 7 minutos por sesión y presenta tasas de error del 15–20 % debido a registros ilegibles y omisiones. Mediante un diseño experimental con $N=13$ estudiantes durante 8 semanas, se implementó un sistema de dos etapas: (1) detección facial con *YOLOv8n-Face* y (2) identificación mediante embeddings *ArcFace*. Se recopilaron 65 imágenes por estudiante bajo condiciones variables de iluminación (300–500 lux) y distancia (1–5 m), evaluadas en 24 sesiones lectivas reales ($M=1\,080$ registros). Los resultados mostraron una precisión promedio del 84 % destacando la iluminación como factor determinante en el rendimiento, con tiempos de procesamiento de 288 ± 23 ms por rostro. La implementación redujo el tiempo de registro en un 73 % (de 5 min a 1.35 min) y eliminó los errores de transcripción manual, aunque el rendimiento depende críticamente de la iluminación. Los resultados validan la viabilidad técnica y económica del sistema propuesto y demuestran su potencial para mejorar la gestión académica y administrativa en instituciones educativas.

Descriptores: 1. Reconocimiento facial, 2. Biometría, 3. Aprendizaje profundo, 4. Asistencia automatizada, 5. Visión por computadora, 6. Gestión académica.

Abstract

This work presents the development and experimental evaluation of a prototype automated attendance registration system based on biometric facial recognition at the Polytechnic Faculty of the National University of the East (FPUNE), Paraguay. The current manual method requires between 5 and 7 minutes per session and exhibits error rates of 15–20 % due to illegible entries and omissions. Through an experimental design involving $N=13$ students over eight weeks, a two-stage system was implemented: (1) facial detection using *YOLOv8n-Face* and (2) identification through *ArcFace* embeddings. A total of 65 images per student were collected under varying lighting conditions (300–500 lux) and distances (1–5 m), evaluated across 24 real classroom sessions ($M=1,080$ records). The results showed an average recognition accuracy of 84 %, highlighting lighting as a key factor influencing performance, with processing times of 288 ± 23 ms per face. The implementation reduced attendance registration time by 73 % (from 5 min to 1.35 min) and eliminated manual transcription errors, although performance remains highly dependent on lighting conditions. The findings validate the technical and economic feasibility of the proposed system and demonstrate its potential to enhance academic and administrative management in educational institutions.

Key words: 1. Facial recognition, 2. Biometrics, 3. Deep learning, 4. Automated attendance, 5. Computer vision, 6. Academic management.

Índice general

Resumen	IX
Abstract	X
Índice de figuras	XX
Índice de tablas	XXII
1. Introducción	1
1.1. Motivación	2
1.2. Definición del problema	2
1.3. Objetivos	4
1.3.1. Objetivo General	4
1.3.2. Objetivos Específicos	4
1.4. Hipótesis	4
1.5. Fundamentación.	5
1.6. Delimitación del Trabajo.	5
2. Conceptos fundamentales, teorías y antecedentes	7
2.1. Biometría	8
2.1.1. Historia de la biometría	8
2.1.2. Concepto de Seguridad Biométrica	8
2.2. Reconocimiento Facial	10
2.2.1. Historia de Reconocimiento Facial	10
2.2.2. Concepto del Reconocimiento Facial	12

2.2.3.	Funcionamiento del Reconocimiento Facial	13
2.2.4.	Aplicaciones de los sistemas de reconoci- miento facial.	14
2.2.5.	Precisión del Reconocimiento Facial	15
2.3.	Inteligencia Artificial	16
2.3.1.	Concepto de la Inteligencia Artificial . . .	16
2.3.2.	Aprendizaje Automático (Machine Lear- ning)	16
2.3.3.	Aprendizaje Profundo (Deep Learning) . .	17
2.3.4.	Redes Neuronales Artificiales	20
2.3.5.	Tipos Comunes de redes neuronales arti- ficiales	21
2.3.6.	Visión por Computadora	24
2.4.	Registros Académicos.	26
2.4.1.	Concepto de Registros Académicos.	26
2.4.2.	Contenidos de los registros académicos. . .	26
2.4.3.	Importancia de los Registros Académicos en la Gestión Educativa	27
2.4.4.	Importancia de los registros académicos. .	28
2.5.	Antecedentes	28
3.	Herramientas Tecnológicas	36
3.1.	Selección de Herramientas	36
3.1.1.	Python	40
3.2.	Entorno de desarrollo del modelo	42
3.2.1.	Visual Studio Code	42
3.2.2.	GitHub Desktop	43
3.2.3.	Postman	43
3.3.	Librerías utilizadas	43
3.3.1.	React	43
3.3.2.	Flask	44
3.3.3.	YOLO	44
3.3.4.	DeepFace	45

4. Método	46
4.1. Enfoque	46
4.2. Método utilizado para el desarrollo	47
4.3. Fases del proceso investigativo	51
4.4. Análisis y selección de herramientas	53
4.5. Recolección de datos	58
4.6. Entrenamiento del modelo	59
4.7. Diseño de la aplicación	63
4.7.1. Arquitectura del sistema	63
4.7.2. Integración de IA y Reconocimiento Facial	64
4.7.3. Relevamiento de requisitos	65
4.7.4. Caso de Uso Principal del Sistema	66
4.7.5. Definición de la Base de Datos y Tablas .	68
4.7.6. Diagrama de flujo del funcionamiento del sistema	73
4.7.7. Desarrollo del backend	74
4.7.8. Desarrollo del frontend	77
4.7.9. Líneas de código	79
4.8. Métricas de Evaluación	80
4.8.1. Usabilidad	81
5. Resultados	82
5.1. Interfaz y funcionalidades del sistema	82
5.2. Entorno de Evaluación y Resultados Iniciales . . .	91
5.2.1. Errores detectados y ajustes aplicados . .	92
5.2.2. Configuraciones y resultados obtenidos . .	94
5.3. Resultados finales del sistema en condiciones reales de un aula	102
6. Discusión	119
6.1. Logros alcanzados con relación a los objetivos es- pecíficos	120
6.2. Logros alcanzados con relación al Objetivo General	122

6.3. Análisis de la hipótesis	123
6.4. Solución al problema de investigación	124
6.5. Sugerencias para futuras investigaciones	125
Anexo A.	127
Anexo B.	133
Referencias bibliográficas	142

Índice de figuras

2.1.	Ejemplo de Seguridad Biométrica	9
2.2.	Una vista de la tableta digitalizadora de RAND; creada en la década de 1960, en la que se probó el primer sistema de identificación facial [1]. . . .	11
2.3.	Dispositivo de reconocimiento facial utilizado en un aeropuerto para verificar la identidad de los pasajeros mediante el análisis de sus rasgos facia- les en tiempo real [2].	12
2.4.	Relación jerárquica entre los campos de la Inteli- gencia Artificial, el Aprendizaje Automático y el Aprendizaje Profundo [3].	19
2.5.	Estructura de una red neuronal profunda.	20
2.6.	La imagen representa una red neuronal preali- mentada, uno de los modelos más básicos y utili- zados en inteligencia artificial [4].	22
2.7.	Representación de una red neuronal recurrente (RNN), donde las conexiones internas permiten el manejo de secuencias mediante memoria tem- poral [4].	22
2.8.	La imagen muestra cómo una red neuronal convo- lucional (CNN) procesa una imagen en distintas capas, extrayendo características progresivas des- de patrones simples hasta rostros completos para generar una salida final [5].	23

2.9.	La imagen ilustra el funcionamiento de una red generativa adversaria (GAN) [6].	24
2.10.	Formato tradicional de registro de asistencia escolar [7].	26
4.1.	Tablero de gestión de tareas del trabajo en Notion, bajo la metodología ágil Scrum Fuente: [Elaboración Propia].	50
4.2.	Ejemplo de imagen facial capturada al natural Fuente: [Elaboración Propia].	59
4.3.	Ejemplo de imagen facial luego del proceso de normalización Fuente: [Elaboración Propia]. . . .	59
4.4.	Caso de uso principal del sistema Fuente: [Elaboración Propia].	67
4.5.	Diagrama de Entidad–Relación del sistema Fuente: [Elaboración Propia].	69
4.6.	Diagrama de flujo del funcionamiento del sistema.[Fuente Elaboración Propia].	74
4.7.	Estructura general del trabajo en Visual Studio Code Fuente: [Elaboración Propia].	77
5.1.	Interfaz de inicio de sesión del sistema Fuente: [Elaboración Propia].	83
5.2.	Formulario de registro de usuario Fuente: [Elaboración Propia].	84
5.3.	Gráfico de asistencias por día en el módulo Dashboard Fuente: [Elaboración Propia].	85
5.4.	Listado de estudiantes y asistencias registradas en el módulo Dashboard Fuente: [Elaboración Propia].	86
5.5.	Módulo de reconocimiento facial automático Fuente: [Elaboración Propia].	87
5.6.	Gestión de roles: registro, edición y eliminación Fuente: [Elaboración Propia].	87

5.7. Registro y gestión de cursos Fuente: [Elaboración Propia].	88
5.8. Registro y gestión de horarios por curso Fuente: [Elaboración Propia].	89
5.9. Módulo de registro manual de asistencias Fuente: [Elaboración Propia].	90
5.10. Asignación de cursos a participantes Fuente: [Elaboración Propia].	91
5.11. Fragmento de código utilizado para configurar el umbral de reconocimiento Fuente: [Elaboración Propia].	94
5.12. Muy estricto, reconoce solo rostros casi idénticos Fuente: [Elaboración Propia].	95
5.13. Fragmento de código utilizado para configurar el umbral de reconocimiento Fuente: [Elaboración Propia].	96
5.14. Estricto, buena precisión, pero sensible a luz o expresión Fuente: [Elaboración Propia].	97
5.15. Fragmento de código utilizado para configurar el umbral de reconocimiento Fuente: [Elaboración Propia].	98
5.16. Equilibrado, balance entre precisión y tolerancia Fuente: [Elaboración Propia].	99
5.17. Fragmento de código utilizado para configurar el umbral de reconocimiento Fuente: [Elaboración Propia].	99
5.18. Moderadamente flexible, admite pequeñas variaciones Fuente: [Elaboración Propia].	100
5.19. Fragmento de código utilizado para configurar el umbral de reconocimiento Fuente: [Elaboración Propia].	101
5.20. Puede aceptar coincidencias con diferencias notables Fuente: [Elaboración Propia].	102

5.21. Prueba inicial del sistema bajo condiciones de iluminación máxima, con detección de personas registradas y no registradas Fuente: [Elaboración Propia].	103
5.22. Segunda prueba con iluminación uniforme, evidenciando estabilidad en las detecciones Fuente: [Elaboración Propia].	104
5.23. Prueba con variaciones leves de postura y uso de accesorios, manteniendo altos niveles de reconocimiento Fuente: [Elaboración Propia].	105
5.24. Prueba a mayor distancia, con leve disminución en la similitud pero reconocimiento correcto de los usuarios registrados Fuente: [Elaboración Propia].	106
5.25. Prueba con iluminación reducida, mostrando una disminución natural en la precisión de reconocimiento Fuente: [Elaboración Propia].	107
5.26. Prueba con iluminación mínima, donde el sistema no logró reconocer a los participantes Fuente: [Elaboración Propia].	108
5.27. Prueba con condiciones normales de iluminación y cambio de posición de los participantes registrados Fuente: [Elaboración Propia].	109
5.28. Prueba con la cámara a una altura de aproximadamente 2.3 metros, mostrando un reconocimiento estable Fuente: [Elaboración Propia].	110
5.29. Prueba con ocho participantes en el encuadre, reconociendo correctamente a los seis usuarios registrados Fuente: [Elaboración Propia].	111
5.30. Prueba grupal con iluminación mínima, sin reconocimiento exitoso de los usuarios registrados Fuente: [Elaboración Propia].	112

5.31. Prueba con la cámara a 2.2 metros de altura y posiciones diversas de los participantes, evidenciando un desempeño estable Fuente: [Elaboración Propia].	113
5.32. Prueba final con variación del ángulo de la cámara, mostrando estabilidad en los niveles de similitud Fuente: [Elaboración Propia].	114
5.33. Usuarios registrados dentro del sistema de reconocimiento facial Fuente: [Elaboración Propia]. . .	115
5.34. Procesamiento de imágenes y generación de vectores de características (embeddings) para cada usuario registrado Fuente: [Elaboración Propia]. .	116
5.35. Registros de asistencias generados automáticamente por el sistema en la base de datos Fuente: [Elaboración Propia].	116
6.1. Encabezado del formulario de encuestas Fuente: [Elaboración Propia].	127
6.2. Resultados de la primera pregunta de la encuesta: “El sistema es fácil de usar y navegar” Fuente: [Elaboración Propia].	128
6.3. Resultados de la segunda pregunta de la encuesta: “Las funciones principales se entienden sin necesidad de asistencia o capacitación” Fuente: [Elaboración Propia].	128
6.4. Resultados de la tercera pregunta de la encuesta: “El tiempo de respuesta del sistema es adecuado durante su uso” Fuente: [Elaboración Propia]. . .	129
6.5. Resultados de la cuarta pregunta de la encuesta: “Los mensajes o indicaciones del sistema son claros y comprensibles” Fuente: [Elaboración Propia].	129

- 6.6. Resultados de la quinta pregunta de la encuesta: “Puedo realizar las tareas necesarias (registro, asistencia, reportes) sin dificultad” Fuente: [Elaboración Propia]. 130
- 6.7. Resultados de la sexta pregunta de la encuesta: “La interfaz del sistema tiene una apariencia ordenada y agradable” Fuente: [Elaboración Propia]. 130
- 6.8. Resultados de la séptima pregunta de la encuesta: “Me siento seguro al utilizar el sistema para registrar información” Fuente: [Elaboración Propia]. 131
- 6.9. Resultados de la octava pregunta de la encuesta: “Considero que el sistema mejora la rapidez y eficiencia del control de asistencia” Fuente: [Elaboración Propia]. 131
- 6.10. Resultados de la novena pregunta de la encuesta: “No experimenté errores o confusiones significativas al interactuar con las opciones del sistema” Fuente: [Elaboración Propia]. 132
- 6.11. Resultados de la décima pregunta de la encuesta: “En general, estoy satisfecho con la experiencia de uso que ofrece el sistema” Fuente: [Elaboración Propia]. 132
- 6.12. Importaciones principales y definición de rutas base del módulo de entrenamiento `train_faces`. Se cargan dependencias para detección y representación facial y se establecen las carpetas de trabajo del pipeline Fuente: [Elaboración Propia]. 134
- 6.13. Función `recortar_y_guardar` del módulo `train_faces`, detecta rostros con YOLOv8, recorta y guarda una versión normalizada (160×160) para generar embeddings Fuente: [Elaboración Propia]. . . 135

- 6.14. Fragmento del código de la función `generar_embeddings` del módulo `train_faces`, donde se generan las representaciones vectoriales de los rostros utilizando el modelo `ArcFace` de DeepFace Fuente: [Elaboración Propia]. 136
- 6.15. Fragmento del código que calcula el centroide facial y actualiza el índice global `centroids.json` con los nuevos embeddings generados Fuente: [Elaboración Propia]. 137
- 6.16. Importaciones y configuración inicial del módulo `face_recognizer`, donde se establecen las rutas a los embeddings y centroides utilizados en la fase de reconocimiento Fuente: [Elaboración Propia]. . 138
- 6.17. Función `load_centroids` del módulo `face_recognizer`, responsable de cargar y normalizar los centroides almacenados para su uso en el proceso de reconocimiento facial Fuente: [Elaboración Propia]. . . . 139
- 6.18. Función `recognize_face` del módulo `face_recognizer`, encargada de generar el embedding del rostro y compararlo con los centroides almacenados mediante la distancia coseno Fuente: [Elaboración Propia]. 140
- 6.19. Función `recognize_faces_from_crops` del módulo `face_recognizer`, utilizada para procesar múltiples rostros detectados en una misma imagen y devolver sus resultados de identificación Fuente: [Elaboración Propia]. 141

Índice de Tablas

3.1. Comparación de lenguajes según los criterios de selección Fuente: [Elaboración Propia].	39
4.1. Resultados principales de los modelos de detección [8].	54
4.2. Comparativa de modelos YOLO para detección facial [9].	55
4.3. Rendimiento de modelos YOLO en distintos niveles de dificultad (WIDER Face) [9].	56
4.4. Comparativa de modelos de DeepFace [9].	56
4.5. Comparativa de modelos de DeepFace [9].	57
4.6. Requisitos funcionales del sistema Fuente: [Elaboración Propia].	65
4.7. Requisitos no funcionales del sistema Fuente: [Elaboración Propia].	66
4.8. Diccionario de datos de la tabla <i>roles</i> Fuente: [Elaboración Propia].	70
4.9. Diccionario de datos de la tabla <i>users</i> Fuente: [Elaboración Propia].	70
4.10. Diccionario de datos de la tabla <i>participants</i> Fuente: [Elaboración Propia].	71
4.11. Diccionario de datos de la tabla <i>courses</i> Fuente: [Elaboración Propia].	71
4.12. Diccionario de datos de la tabla <i>course_schedules</i> Fuente: [Elaboración Propia].	72

4.13. Diccionario de datos de la tabla <i>participant_courses</i> Fuente: [Elaboración Propia].	72
4.14. Diccionario de datos de la tabla <i>attendances</i> Fuente: te: [Elaboración Propia].	73
4.15. Métricas consideradas en el desarrollo y evaluación del sistema Fuente: [Elaboración Propia]. . .	80
5.1. Componentes de hardware y software utilizados en el sistema Fuente: [Elaboración Propia].	92
5.2. Errores frecuentes y acciones de mitigación Fuente: te: [Elaboración Propia].	93
5.3. Configuración de umbral centrada en 0.45 y re- sultado esperado (distancia entre embeddings ex- presada como valor porcentual). Fuente: [Elabo- ración Propia].	93
5.4. Indicadores de desempeño del sistema durante las pruebas de registro de asistencia.	117
5.5. Resumen de métricas obtenidas durante las prue- bas finales en aula Fuente: [Elaboración Propia]. .	118

Capítulo 1

Introducción

El registro de asistencia es una tarea clave en la gestión educativa y administrativa de instituciones académicas como la Facultad Politécnica de la Universidad Nacional del Este (FPUNE). No obstante, los métodos tradicionales de control manual presentan errores de transcripción y omisiones que comprometen la integridad de los datos, como se ha documentado en estudios previos sobre digitalización de procesos académicos. Este trabajo final desarrolló un sistema automatizado de asistencia mediante reconocimiento facial, orientado a la mejora de los procesos de control académico y a la aplicación de técnicas de visión por computadora en la gestión educativa. Para su implementación se emplearon algoritmos de reconocimiento facial basados en aprendizaje profundo y visión por computadora, integrados en un entorno cliente-servidor con procesamiento en tiempo real. La metodología contempló la selección tecnológica, el procesamiento de datos, el entrenamiento de modelos y pruebas piloto en un entorno académico durante el año 2025, con el propósito de reducir errores y mejorar la gestión administrativa. De esta forma, el estudio aportó una herramienta que brinde datos más confiables y contribuya a una mayor eficiencia operativa, beneficiando a estudiantes, docentes y autoridades mediante la adopción de tecnologías innovadoras que fortalezcan la gestión

académica.

1.1. Motivación

En la actualidad, el control de asistencia en instituciones educativas sigue dependiendo de métodos manuales, como el llamado de lista o el registro en papel, lo que puede generar errores humanos y pérdida de tiempo en la gestión académica. La automatización de este procedimiento mediante tecnologías de reconocimiento facial se presentó como una alternativa innovadora que no solo mejora la precisión del registro de asistencia, sino que también permite una gestión más eficiente de la información en tiempo real.

En el ámbito académico, este estudio representó una oportunidad para explorar el uso del reconocimiento facial en entornos educativos, impulsando la incorporación de tecnologías modernas que mejoren la gestión institucional. Al haber demostrado eficacia en distintos sectores, su adaptación al contexto universitario marcó una diferencia significativa en la optimización de procesos y en la precisión del control de asistencia.

Este trabajo también buscó reducir el impacto ambiental al eliminar el uso de papel en los registros de asistencia, promoviendo prácticas más sostenibles dentro de la institución. En este sentido, la motivación principal radicó en la combinación de innovación tecnológica, eficiencia administrativa y sostenibilidad, factores clave que justifican la importancia de esta investigación.

1.2. Definición del problema

En la FPUNE, el proceso de registro de asistencia continúa realizándose mediante métodos tradicionales, lo que genera limi-

taciones operativas y una baja trazabilidad de los datos dentro de los sistemas académicos. El uso de procedimientos manuales provoca inconsistencias, pérdida de información y dificulta la disponibilidad de registros confiables. Como consecuencia, estas limitaciones impiden la generación de reportes analíticos precisos y obstaculizan la toma de decisiones basadas en datos, lo que evidenció la necesidad de incorporar un sistema inteligente de monitoreo y registro.

Ante lo expuesto surgieron las siguientes preguntas:

Pregunta General.

¿Cómo desarrollar un sistema de Reconocimiento Facial para el control de asistencia de los estudiantes de la FPUNE?

Preguntas Específicas.

1. ¿Cuáles son los algoritmos de reconocimiento facial?
2. ¿Cuáles son las herramientas tecnológicas para el desarrollo del Sistema?
3. ¿Cuáles son los requerimientos para el registro de asistencia de la Facultad Politécnica?
4. ¿Cómo se desarrollará el sistema de reconocimiento facial y gestión de asistencias?
5. ¿Qué pruebas se deben realizar para validar su correcto funcionamiento?
6. ¿Cómo se evaluarán los resultados obtenidos para medir la efectividad del sistema?

1.3. Objetivos

1.3.1. Objetivo General

Desarrollar un prototipo de sistema de reconocimiento facial en un aula para el control de asistencia de estudiantes de la Facultad Politécnica de la Universidad Nacional del Este.

1.3.2. Objetivos Específicos

1. Seleccionar los algoritmos de reconocimiento facial.
2. Seleccionar las herramientas tecnológicas para el desarrollo del Sistema.
3. Identificar los requisitos para el desarrollo del sistema de registro de asistencia para la FPUNE.
4. Desarrollar el sistema de reconocimiento facial y gestión de asistencias.
5. Realizar pruebas del funcionamiento del prototipo
6. Evaluar resultados obtenidos.

1.4. Hipótesis

La hipótesis principal establece que el desarrollo de un sistema de reconocimiento facial, basado en los modelos *YOLOv8n-Face* y *DeepFace* que permitirá registrar la asistencia de los estudiantes de la FPUNE, reduciendo los errores del control manual y optimizando el tiempo destinado al proceso de registro.

1.5. Fundamentación.

El desarrollo de la presente investigación responde a la necesidad de modernizar los procesos administrativos en las instituciones educativas y de aprovechar las ventajas que ofrecen las tecnologías de reconocimiento facial. A continuación, se presentan las diversas fundamentaciones en base a los puntos siguientes:

- **Innovación tecnológica:** La propuesta desarrollada introduce un enfoque innovador al aplicar reconocimiento facial para la gestión automatizada de asistencia, constituyendo un modelo potencialmente adaptable a diversos entornos académicos y administrativos.
- **Incremento de la precisión en los datos de asistencia:** Los datos obtenidos del sistema serán más precisos y confiables, lo que permitirá una mejor evaluación del desempeño académico de los estudiantes y la identificación de patrones de asistencia.
- **Fomento de la puntualidad y asistencia:** El desarrollo del sistema puede incentivar a los estudiantes a ser más puntuales y asistir con regularidad a clases.

1.6. Delimitación del Trabajo.

1. **Límite del contenido:** Este trabajo aborda el desarrollo de una propuesta de reconocimiento facial orientada al registro de asistencia en la FPUNE.
2. **Límite espacial:** Se limitaron las pruebas a ser realizadas en un entorno controlado y luego a ser probado en un aula de la FPUNE.

3. **Límite Temporal:** Las pruebas y el desarrollo se realizaron durante un semestre académico en 2025. Incluyendo la recopilación de datos, el entrenamiento del modelo y la evaluación de su desempeño.

Impacto de la Investigación.

El desarrollo de la propuesta desarrollada generó un impacto significativo tanto a nivel institucional como en el entorno social más amplio. A nivel institucional, mejoró la eficiencia y precisión en el registro de asistencia, reduciendo el margen de error humano. En el ámbito social, el sistema fortaleció la confianza en los procesos educativos y administrativos de la FPUNE. Asimismo, la digitalización del registro de asistencia permitió disminuir el uso de papel y tinta, contribuyendo a un menor consumo de recursos.

Capítulo 2

Conceptos fundamentales, teorías y antecedentes

Este capítulo tiene como propósito presentar los conceptos teóricos esenciales y los antecedentes más relevantes que enmarcaron la elaboración de la propuesta. En esta sección se profundiza en los fundamentos del reconocimiento facial como herramienta biométrica y en las tecnologías que posibilitan su aplicación en entornos académicos. Se describen los principios de funcionamiento de los sistemas biométricos, destacando los aspectos técnicos y metodológicos que sustentan el desarrollo de soluciones orientadas a la automatización de procesos institucionales.

Asimismo, se analizan estudios previos que han abordado el reconocimiento facial en distintos contextos, tanto a nivel local como internacional, con el fin de identificar enfoques y resultados que aporten referencias valiosas al diseño de la solución planteada. Este análisis permite reconocer los avances y limitaciones existentes, así como las oportunidades de mejora que la presente investigación busca aprovechar, estableciendo así el marco teórico y contextual que sustenta el desarrollo del trabajo.

2.1. Biometría

2.1.1. Historia de la biometría

La biometría no se puso en práctica en las culturas occidentales hasta finales del siglo XIX, pero era utilizada en China desde al menos el siglo XIV. Un explorador y escritor que respondía al nombre de Joao de Barros escribió que los comerciantes chinos estampaban las impresiones y las huellas de la palma de las manos de los niños en papel con tinta. Los comerciantes hacían esto como método para distinguir entre los niños jóvenes [10].

En Occidente, la identificación confiaba simplemente en la “memoria fotográfica” hasta que Alphonse Bertillon, jefe del departamento fotográfico de la Policía de París, desarrolló el sistema antropométrico (también conocido más tarde como Bertillonage) en 1883. Este era el primer sistema preciso, ampliamente utilizado científicamente para identificar a criminales y convirtió a la biometría en un campo de estudio [10].

Funcionaba midiendo de forma precisa ciertas longitudes y anchuras de la cabeza y del cuerpo, así como registrando marcas individuales como tatuajes y cicatrices. El sistema de Bertillon fue adoptado extensamente en Occidente hasta que aparecieron defectos en el sistema, principalmente problemas con métodos distintos de medición y cambios en las medidas. Después de esto, las fuerzas policiales occidentales comenzaron a usar la huella dactilar, esencialmente el mismo sistema visto en China cientos de años antes [10].

2.1.2. Concepto de Seguridad Biométrica

Para comprender el concepto de seguridad biométrica, resulta necesario partir de la definición general de biometría. Este término hace referencia al estudio y análisis de rasgos físicos o conductuales propios de cada persona, con el propósito de ve-

rificar o confirmar su identidad. En su significado más básico, la biometría alude literalmente a la “medición del cuerpo humano” [11, 12].

La biometría también puede entenderse como el uso de métodos estadísticos y matemáticos para medir y evaluar características físicas o biológicas que son únicas en cada individuo, con el objetivo de realizar procesos de identificación de manera precisa [12, 13].

Los datos biométricos son cualquier tipo de información sobre las características físicas de un individuo, como los patrones de la retina, las huellas dactilares y la estructura facial [13].

En la Figura 2.1 se presenta un ejemplo de un sistema de seguridad biométrica basado en la verificación mediante huella dactilar, utilizado comúnmente para el control de acceso a instalaciones o dispositivos.



Figura 2.1: Ejemplo de Seguridad Biométrica [14].

La seguridad biométrica consiste en emplear rasgos biológicos únicos de una persona para realizar procesos de autenticación y control de acceso. Para ello se utilizan dispositivos de hardware, como cámaras o lectores de huellas, que capturan los datos biométricos y los procesan mediante algoritmos que comparan dicha información con los registros almacenados en una base de datos. Cuando ambos conjuntos de datos coinciden, la identidad del usuario se valida y se autoriza el acceso correspondiente [13, 15].

2.2. Reconocimiento Facial

2.2.1. Historia de Reconocimiento Facial

En 1960, Woodrow Wilson Bledsoe trabajó en un sistema para clasificar los rasgos del rostro humano a través de la tabla RAND. Este sistema utilizaba un lápiz óptico y unas coordenadas para situar los ojos, la nariz o la boca de las personas de forma precisa, pero era un procedimiento todavía muy manual [10].



Figura 2.2: Una vista de la tableta digitalizadora de RAND; creada en la década de 1960, en la que se probó el primer sistema de identificación facial [1].

Una década después llegarían Goldstein, Harmon y Lesk, que detallaron estas características faciales e iniciaron la mejora hacia la precisión del reconocimiento facial. Más adelante, a finales de los años 80', se aplica el álgebra lineal, gracias a Sirovich y Kirby [10].

Ya en 1991, Turk y Pentland desarrollan la tecnología capaz de detectar un rostro humano dentro de una fotografía, abriendo paso al reconocimiento facial automático [1].

Poco a poco, el reconocimiento facial se fue haciendo un hueco en las aplicaciones de seguridad, en particular aquellas concernientes al Estado. En 2001, nace el Viola-Jones Object Detection Framework, que propone algoritmos para detectar objetos dentro de imágenes, y que enseguida fue utilizado para la detección de rostros de forma exitosa [10].

No es hasta 2010 cuando esta tecnología llega al gran público

a través de Facebook [1].

2.2.2. Concepto del Reconocimiento Facial

El reconocimiento facial es un método utilizado para verificar o determinar la identidad de una persona a partir de las características de su rostro. Este tipo de sistemas es capaz de identificar individuos en fotografías, grabaciones de video o incluso en transmisiones en tiempo real, comparando sus rasgos faciales con los datos previamente almacenados [16, 17].

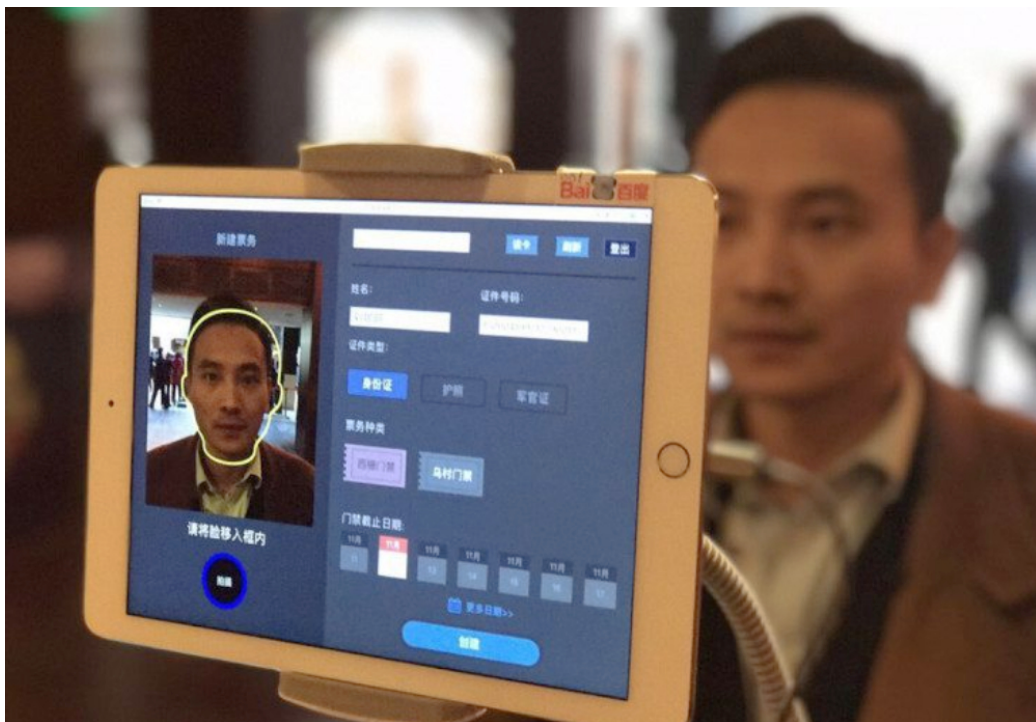


Figura 2.3: Dispositivo de reconocimiento facial utilizado en un aeropuerto para verificar la identidad de los pasajeros mediante el análisis de sus rasgos faciales en tiempo real [2].

El reconocimiento facial forma parte del conjunto de tecnologías utilizadas en la seguridad biométrica. Existen otras variantes de sistemas biométricos, como el reconocimiento de voz, de huellas digitales o de patrones del iris o la retina. Aunque

estas tecnologías se aplican tradicionalmente en ámbitos de protección y seguridad, su uso se ha expandido progresivamente hacia otros sectores y actividades [16,17].

2.2.3. Funcionamiento del Reconocimiento Facial

- **Paso 1: Reconocimiento Facial** La cámara detecta y ubica la imagen de un rostro, ya sea de forma independiente o como parte de una muchedumbre. La imagen puede mostrar a la persona de frente o de perfil.
- **Paso 2: Análisis facial** A continuación, se captura y analiza una imagen del rostro. La mayor parte de la tecnología de reconocimiento facial depende de imágenes 2D en lugar de 3D, ya que se puede comparar de manera más fácil una imagen 2D con las fotos públicas o las de una base de datos. El software lee la geometría de tu rostro. Los factores clave incluyen la distancia entre los ojos, la profundidad de las cuencas de los ojos, la distancia desde la frente hasta el mentón, la forma de los pómulos y el contorno de los labios, las orejas y el mentón. El objetivo es identificar los puntos de referencia faciales que son clave para distinguir un rostro.
- **Paso 3: Conversión de la imagen a datos** El proceso de captura de rostro transforma la información analógica (un rostro) en un conjunto de información digital (datos) basado en los rasgos faciales de la persona. Básicamente, el análisis del rostro se convierte en una fórmula matemática. El código numérico se denomina huella facial. De la misma manera en que las huellas dactilares son únicas, cada persona tiene su propia huella facial.
- **Paso 4: Búsqueda de una coincidencia** La huella facial se compara con una base de datos de otros rostros conoci-

dos. Por ejemplo, el FBI tiene acceso a hasta 650 millones de fotos, de diversas bases de datos estatales. Si la huella facial coincide con una imagen en una base de datos de reconocimiento facial, entonces se realizará una determinación.

De todas las mediciones biométricas, el reconocimiento facial se considera el más natural. Esto sigue una lógica intuitiva, ya que normalmente nos reconocemos a nosotros mismos y a los demás mirando las caras, en lugar de las huellas digitales y los iris. Se estima que, periódicamente, más de la mitad de la población mundial se ve afectada por la tecnología de reconocimiento facial [16, 18].

2.2.4. Aplicaciones de los sistemas de reconocimiento facial.

- **Detección de fraudes:** Las empresas utilizan el reconocimiento facial para identificar de forma exclusiva a los usuarios que crean una nueva cuenta en una plataforma en línea. Una vez que esto se realiza, se puede utilizar para verificar la identidad de la persona que realmente utiliza la cuenta en caso de que se produzca una actividad de riesgo o sospechosa en la cuenta [19].
- **Seguridad cibernética:** Las empresas utilizan la tecnología de reconocimiento facial en lugar de las contraseñas para reforzar las medidas de seguridad cibernética. Es difícil acceder sin autorización a los sistemas de reconocimiento facial, ya que no se puede cambiar nada del rostro. El software de reconocimiento facial también es una herramienta de seguridad cómoda y de gran precisión para desbloquear teléfonos inteligentes y otros dispositivos personales [19].
- **Control de aeropuertos y fronteras:** Muchos aeropuer-

tos utilizan los datos biométricos como pasaportes, con lo que los viajeros pueden evitar las largas filas y pasar por una terminal automatizada para llegar más rápidamente a la puerta de embarque. La tecnología de reconocimiento facial en forma de pasaportes electrónicos reduce los tiempos de espera y mejora la seguridad [19].

2.2.5. Precisión del Reconocimiento Facial

Los algoritmos de reconocimiento facial pueden alcanzar niveles muy altos de exactitud cuando operan en condiciones controladas. En entornos bien preparados, su desempeño suele ser más consistente, mientras que en escenarios reales la precisión tiende a disminuir debido a variaciones propias del entorno. Además, no existe un único indicador que permita evaluar de forma completa su efectividad, por lo que resulta complejo determinar una tasa de éxito universal para esta tecnología [16, 18].

Por ejemplo, los algoritmos de verificación facial que buscan coincidencias entre personas e imágenes de referencia claras, como una licencia de conducción o una foto policial, logran puntuaciones de alta precisión. Sin embargo, este nivel de precisión únicamente es posible con lo siguiente:

- Posicionamiento e iluminación coherentes
- Rasgos faciales claros y libres de obstrucciones
- Colores y fondo controlados
- Calidad de la cámara y resolución de la imagen

Otro factor que influye en las tasas de error es el envejecimiento. Con el tiempo, los cambios en el rostro dificultan la coincidencia con las fotografías tomadas años antes.

2.3. Inteligencia Artificial

2.3.1. Concepto de la Inteligencia Artificial

La inteligencia artificial es una rama de la ciencia orientada al desarrollo de sistemas y máquinas capaces de realizar tareas que normalmente requieren habilidades propias del ser humano, como razonar, aprender o tomar decisiones. También permite trabajar con volúmenes de datos tan grandes que resultan imposibles de analizar manualmente.

Se trata de un campo amplio que abarca diversas áreas del conocimiento, entre ellas la informática, la estadística y el análisis de datos, así como disciplinas relacionadas con el hardware, el software, la psicología, la lingüística y la neurociencia, entre otras.

La inteligencia artificial engloba un conjunto de tecnologías basadas principalmente en el aprendizaje automático y el aprendizaje profundo. Estas herramientas se aplican en actividades como la clasificación de objetos, la generación de predicciones, el análisis de grandes conjuntos de datos, el procesamiento de lenguaje natural, los sistemas de recomendación y la recuperación inteligente de información, entre múltiples usos [20, 21].

2.3.2. Aprendizaje Automático (Machine Learning)

El aprendizaje automático es una rama de la inteligencia artificial que se ocupa de crear métodos y algoritmos capaces de aprender directamente a partir de los datos, evitando la necesidad de programar de forma explícita cada una de las tareas que debe realizar un sistema. Su funcionamiento se apoya en técnicas estadísticas y en la identificación de patrones dentro de grandes volúmenes de información, con el fin de generar predicciones, tomar decisiones o clasificar elementos. A medida que el modelo analiza más datos y ajusta sus parámetros, mejora

progresivamente su nivel de exactitud [22, 23].

En el contexto de esta investigación, el Machine Learning constituyó una herramienta esencial para el desarrollo del Sistema de Asistencia Basado en Reconocimiento Facial para la FPUNE. A través del aprendizaje automático, se emplean modelos previamente entrenados capaces de detectar y reconocer rostros en imágenes reales con alta precisión, incluso bajo condiciones variables de iluminación, ángulo o expresión facial. Esta tecnología permitió automatizar el registro de asistencia de los estudiantes, optimizando los procesos administrativos y reduciendo significativamente la intervención manual por parte de los docentes [24].

El aprendizaje automático se puede dividir en tres grandes tipos:

- **Aprendizaje supervisado:** el modelo se entrena con datos etiquetados (entrada y salida esperada). Es el enfoque más común y se utiliza cuando se cuenta con ejemplos precisos para aprender.
- **Aprendizaje no supervisado:** se aplica cuando los datos no tienen etiquetas. El sistema identifica patrones o agrupaciones de forma autónoma, sin intervención humana.
- **Aprendizaje por refuerzo:** el modelo aprende por medio de la experiencia, tomando decisiones en un entorno dinámico y recibiendo recompensas o penalizaciones en función de sus acciones.

2.3.3. Aprendizaje Profundo (Deep Learning)

Es una rama del aprendizaje automático que utiliza redes neuronales artificiales compuestas por múltiples capas conectadas entre sí. A través de estas capas, los datos iniciales se transforman de manera gradual en representaciones más abstractas

y detalladas, lo que permite abordar tareas complejas como la clasificación de imágenes, el reconocimiento de patrones y otros procesos de alto nivel [25, 26].

Una red neuronal artificial está compuesta por unidades denominadas *nodos*, organizadas en capas de entrada, ocultas y de salida. Cada nodo combina las señales recibidas mediante pesos ajustables, aplica una función de activación y transmite el resultado a las siguientes capas. El entrenamiento se realiza mediante el algoritmo de *retropropagación del error*, que ajusta los pesos para minimizar la diferencia entre la salida esperada y la obtenida. Cuantas más capas y parámetros posee una red, mayor es su capacidad de aprendizaje, siempre que se disponga de suficientes datos y recursos computacionales.

En el ámbito del procesamiento de imágenes, destacan las redes neuronales convolucionales (CNN, por sus siglas en inglés), diseñadas específicamente para analizar información visual. Estas redes aplican filtros de convolución que capturan características locales —como bordes, texturas y formas— y permiten construir representaciones jerárquicas de la imagen. Gracias a esta arquitectura, las CNN se han convertido en la base de los modelos modernos de visión por computadora.

En esta investigación, el aprendizaje profundo constituye un pilar esencial para el desarrollo del prototipo *Sistema de Asistencia Basado en Reconocimiento Facial para la FPUNE*. Se emplearon dos modelos basados en redes neuronales convolucionales: YOLOv8n-Face, encargado de la detección de rostros en tiempo real, y ArcFace, responsable de la generación de *embeddings* faciales que representan la identidad de cada individuo. Ambos modelos son redes profundas preentrenadas sobre grandes conjuntos de datos, lo que garantiza una alta precisión y robustez frente a variaciones de iluminación, distancia o expresión facial [27].

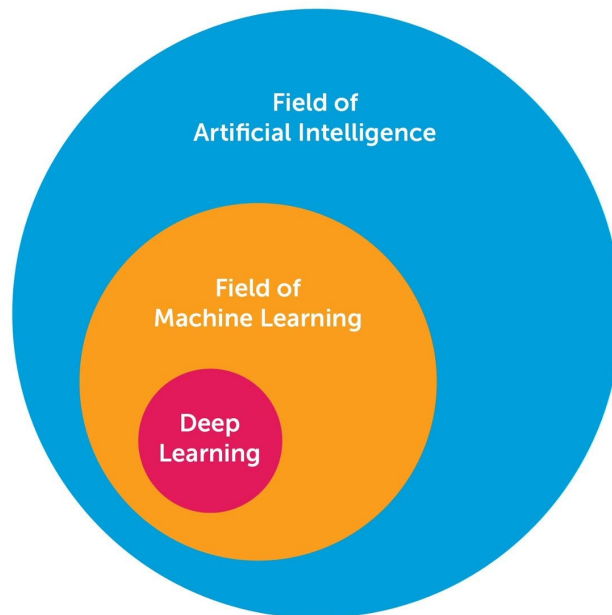


Figura 2.4: Relación jerárquica entre los campos de la Inteligencia Artificial, el Aprendizaje Automático y el Aprendizaje Profundo [3].

Etapas del Aprendizaje Profundo:

- **Etapas de extracción de características:** consiste en generar una jerarquía de atributos desde los datos originales. Las primeras capas capturan rasgos simples (como bordes o colores), mientras que las intermedias y profundas detectan estructuras más complejas.
- **Etapas de transformación de características:** utiliza las representaciones extraídas en la fase anterior para producir la salida deseada, como una clasificación, detección o decisión. En el caso de redes aplicadas a imágenes, esta etapa puede traducir los patrones visuales en texto alfa-numérico o etiquetas.

2.3.4. Redes Neuronales Artificiales

Una red neuronal es un modelo compuesto por unidades de procesamiento llamadas neuronas artificiales o perceptrones, las cuales permiten analizar y clasificar información. Los datos ingresan inicialmente en la capa de entrada y, a partir de allí, cada perceptrón procesa la información y la transmite a los nodos de la siguiente capa. Cuando una red está formada por más de tres capas, se la considera una red neuronal profunda o parte del aprendizaje profundo, pudiendo algunas arquitecturas modernas incluir cientos o incluso miles de capas. La información generada por las capas finales permite a la red ejecutar tareas específicas, como identificar objetos o detectar patrones dentro de los datos [28, 29].

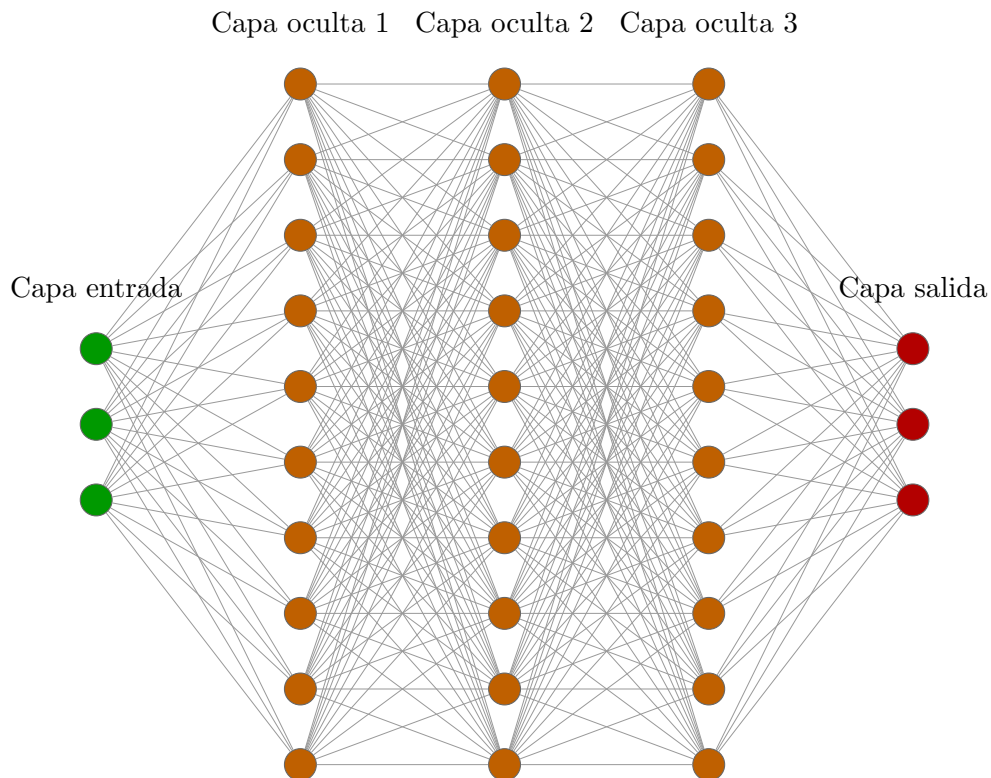


Figura 2.5: Estructura de una red neuronal profunda.

2.3.5. Tipos Comunes de redes neuronales artificiales

Las redes neuronales prealimentadas, son una de las formas más antiguas de redes neuronales, ya que los datos fluyen en una dirección a través de capas de neuronas artificiales hasta que se obtiene el resultado. En la actualidad, la mayoría de las redes neuronales prealimentadas se consideran “prealimentadas profundas”, con varias capas (y más de una capa oculta). Las redes neuronales prealimentadas suelen vincularse a un algoritmo de corrección de errores llamado “propagación inversa” que, en términos simples, comienza con el resultado de la red neuronal y realiza el proceso en sentido inverso para llegar al principio, detectando errores para mejorar la exactitud de la red neuronal [30].

Las redes neuronales recurrentes (RNN, por sus siglas en inglés) difieren de las redes neuronales prealimentadas en que suelen usar datos de series temporales o datos que involucran secuencias. A diferencia de las redes neuronales prealimentadas, que usan ponderaciones en cada nodo de la red, las redes neuronales recurrentes tienen memoria de lo que sucedió en la capa anterior como contingente a la salida de la capa actual. Por ejemplo, cuando se realiza procesamiento de lenguaje natural, las RNN pueden tener en cuenta otras palabras usadas en una oración. Las RNN a menudo se usan para el reconocimiento de voz, la traducción y la generación de descripciones de imágenes [30].

Este tipo de redes neuronales en las que los datos entran por la capa de entrada y se transmiten en una única dirección hacia la capa de salida sin formar bucles, por ejemplo son las que hemos llamado Redes Neuronales Prealimentadas o *Feedforward Neural Networks* en inglés. Un ejemplo de este tipo de redes:

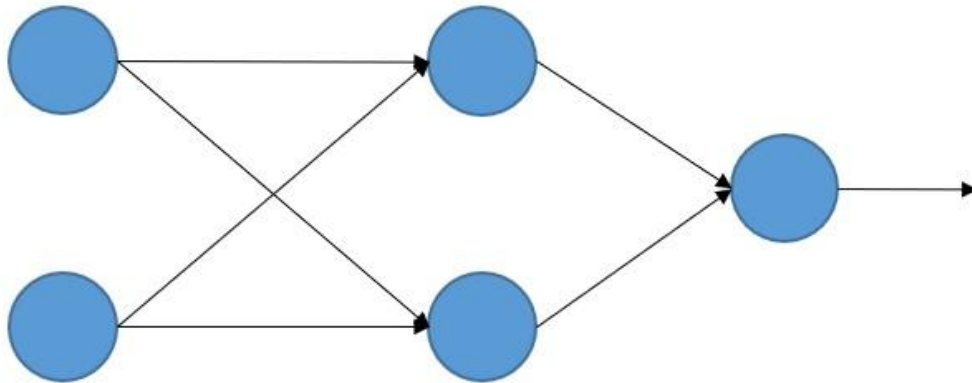


Figura 2.6: La imagen representa una red neuronal prealimentada, uno de los modelos más básicos y utilizados en inteligencia artificial [4].

Sin embargo, compárese esta arquitectura con la siguiente:

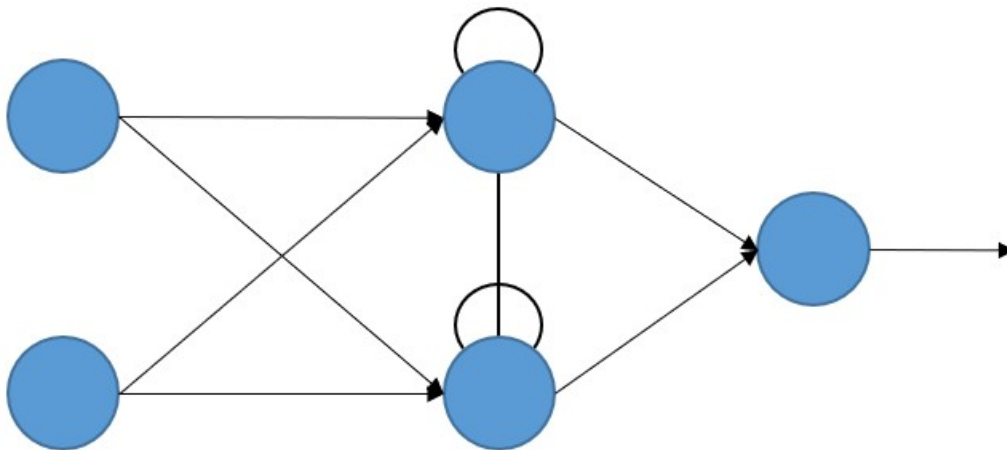


Figura 2.7: Representación de una red neuronal recurrente (RNN), donde las conexiones internas permiten el manejo de secuencias mediante memoria temporal [4].

La memoria a largo/corto plazo, en ingles *Long Short-Term Memory* (LSTM) es una forma avanzada de RNN que puede usar memoria para recordar lo que sucedió en capas anteriores. La diferencia entre las RNN y las LSTM es que estas últimas pueden recordar lo que sucedió hace varias capas mediante el uso de

celdas de memoria. La LSTM suele usarse para el reconocimiento de voz y la realización de predicciones [30].

Las redes neuronales convolucionales (CNN) incluyen algunas de las redes neuronales más comunes en la inteligencia artificial moderna. Las CNN suelen usarse en el reconocimiento de imágenes y emplean varias capas distintas (una capa convolucional y, luego, una capa de agrupación) que filtran diferentes partes de una imagen antes de volver a unirla (en la capa completamente conectada) [4].

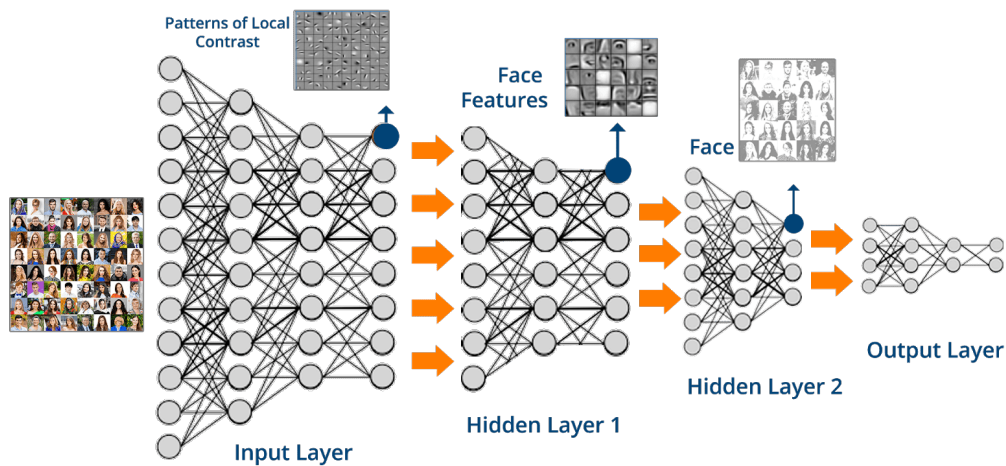


Figura 2.8: La imagen muestra cómo una red neuronal convolucional (CNN) procesa una imagen en distintas capas, extrayendo características progresivas desde patrones simples hasta rostros completos para generar una salida final [5].

En las redes generativas adversarias (GAN), se usan dos redes neuronales que compiten entre sí en un juego que, en última instancia, mejora la exactitud del resultado. Una red (el generador) crea ejemplos que la otra red (el discriminante) juzga como verdaderos o falsos. Las GAN se han usado para crear imágenes realistas y hasta hacer arte [30].

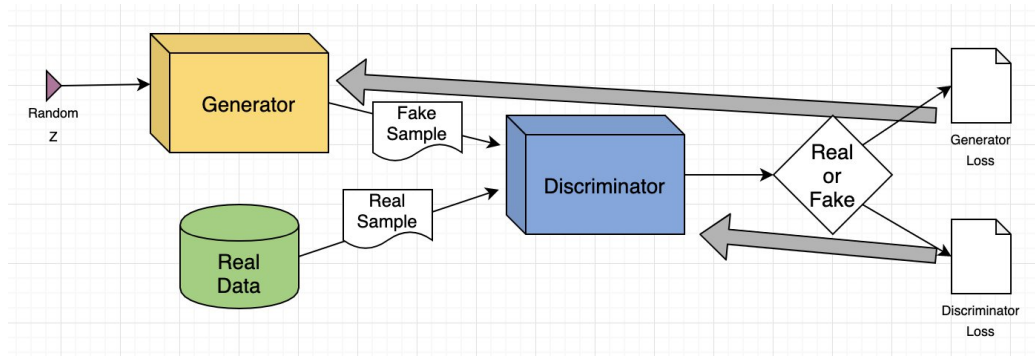


Figura 2.9: La imagen ilustra el funcionamiento de una red generativa adversaria (GAN) [6].

2.3.6. Visión por Computadora

La visión por computadora es un área de la inteligencia artificial que se ocupa de permitir que las máquinas interpreten y analicen imágenes o secuencias de video de manera similar a la percepción visual humana. Su propósito es automatizar procesos que dependen de la comprensión visual, tales como la detección, el reconocimiento, la clasificación o el seguimiento de objetos.

Mediante modelos entrenados, los sistemas informáticos son capaces de identificar patrones presentes en las imágenes y tomar decisiones basadas en la información que capturan del entorno [31, 32].

En el contexto de esta investigación, la visión por computadora constituye la base técnica para el desarrollo del prototipo propuesto. Mediante el uso de redes neuronales convolucionales y técnicas de procesamiento de imágenes, el sistema fue capaz de detectar y reconocer los rostros de los estudiantes a partir de la cámara utilizada en el aula. Posteriormente, los rostros identificados se compararon con una base de datos para registrar automáticamente la asistencia, reduciendo la intervención manual y agilizando los procesos administrativos dentro de la institución.

Fases del proceso de Visión por Computadora:

- **Adquisición de la imagen:** captura una proyección bidimensional de la luz reflejada por los objetos en la escena.
- **Preprocesamiento:** mejora la calidad visual de la imagen mediante la reducción de ruido o el realce de bordes.
- **Segmentación:** separa los elementos visuales de la escena, detectando bordes y regiones.
- **Extracción de características:** genera representaciones formales de los elementos segmentados (textura, forma, color, etc.).
- **Reconocimiento y localización:** identifica objetos específicos y determina su posición en el espacio tridimensional.
- **Interpretación:** analiza la escena completa y genera conclusiones a partir del conocimiento adquirido y de los datos visuales.

Tipos de reconocimiento en Visión por Computadora

- **Detección de rostros:** se enfoca en identificar la presencia de uno o varios rostros dentro de una imagen o video. En el sistema desarrollado, esta tarea se realiza mediante el modelo **YOLOv8n-Face**, el cual localiza las regiones faciales dentro del fotograma capturado por la cámara utilizada en el aula.
- **Reconocimiento de identidad:** clasifica y diferencia los rostros detectados, identificando a qué persona pertenecen. En este trabajo, dicha tarea se lleva a cabo utilizando el modelo **ArcFace**, que compara las características faciales extraídas con las almacenadas en la base de datos para determinar la identidad del estudiante.

2.4. Registros Académicos.

2.4.1. Concepto de Registros Académicos.

Los registros académicos consisten en la organización y resguardo de información vinculada al desempeño, evolución y trayectoria escolar de cada estudiante. Estos datos pueden almacenarse en diferentes formatos, tanto digitales como físicos, y son gestionados por diversas instituciones educativas, entre ellas escuelas, colegios y universidades [33,34].

En la Figura 2.10 se muestra un formato tradicional utilizado para el registro de asistencia escolar, el cual ejemplifica la estructura manual que emplean muchas instituciones educativas para documentar la presencia de los estudiantes en el aula.

Formato Registro de Asistencia Escolar

REGISTRO DE ASISTENCIA	
N° de orden	Del..... de..... al..... de.....
	Fecha/días
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	

Figura 2.10: Formato tradicional de registro de asistencia escolar [7].

2.4.2. Contenidos de los registros académicos.

Los registros académicos incluyen distintos tipos de información que permiten reflejar de manera completa la trayectoria educativa del estudiante. Cada componente cumple una función específica dentro del control y seguimiento institucional:

- **Calificaciones:** Representan el rendimiento obtenido por

el estudiante en las distintas asignaturas y evaluaciones parciales o finales.

- **Asistencia:** Indica la cantidad de clases o actividades a las que el estudiante ha asistido, sirviendo como indicador de compromiso y cumplimiento académico.
- **Cursos aprobados:** Detallan las materias superadas satisfactoriamente, evidenciando el avance dentro del plan de estudios.
- **Actividades extracurriculares:** Registran la participación en proyectos, talleres, clubes o eventos que complementan la formación académica.
- **Información personal:** Contiene los datos básicos del estudiante, como nombre completo, fecha de nacimiento, número de identificación y otros datos administrativos.
- **Historial académico:** Resume el recorrido del estudiante, incluyendo cambios de carrera, revalidaciones, transferencias o situaciones académicas relevantes.

2.4.3. Importancia de los Registros Académicos en la Gestión Educativa

- **Evaluación del desempeño:** Permiten evaluar el progreso académico a lo largo del tiempo.
- **Toma de decisiones:** Sirven de base para tomar decisiones importantes, como la promoción a un nuevo grado, la obtención de títulos o la admisión a programas de estudios superiores.
- **Seguimiento académico:** Facilitan el seguimiento del progreso del estudiante y la identificación de áreas en las que necesita apoyo adicional.

- **Información para terceros:** Se utilizan para proporcionar información a terceros, como universidades, empleadores y agencias gubernamentales [33].

2.4.4. Importancia de los registros académicos.

Los registros académicos son fundamentales para garantizar la transparencia, la equidad y la calidad en el proceso educativo. Además, proporcionan un historial completo del desempeño académico de un estudiante, lo que es de gran utilidad para su futuro personal y profesional [33].

2.5. Antecedentes

Detección facial mediante OpenCV.

El trabajo denominado Detección facial mediante OpenCV, desarrollado como TFG en la FPUNE, se centra en la creación de un software dedicado exclusivamente a la detección facial como etapa inicial del reconocimiento de rostros. Esta propuesta analiza diversas técnicas y algoritmos disponibles en la biblioteca OpenCV, describiendo su funcionamiento, sus limitaciones y el flujo completo de procesamiento, desde la captura de imágenes hasta la identificación de regiones faciales, utilizando Java como lenguaje principal. Su aporte más relevante es la implementación de un método de detección capaz de operar en tiempo real y también desde archivos, basado en clasificadores tradicionales [35].

En comparación con la propuesta desarrollada en este trabajo, ambas propuestas comparten el uso de técnicas de visión por computadora y reconocen la detección de rostros como un paso fundamental dentro de los sistemas biométricos. También coinciden en la importancia del funcionamiento en tiempo real.

Sin embargo, existen diferencias notables. El antecedente utiliza métodos clásicos de detección, más sensibles a variaciones de iluminación y postura, mientras que la investigación actual emplea modelos modernos basados en aprendizaje profundo, como YOLOv8n-Face y ArcFace, que ofrecen mayor precisión y robustez. Además, la propuesta analizada previamente se limita únicamente a la detección facial, mientras que la presente investigación desarrolla un sistema completo que incluye reconocimiento, registro automático en base de datos y una plataforma web para la gestión académica.

En síntesis, aunque comparten fundamentos similares, la propuesta actual representa un avance significativo al integrar detección, reconocimiento y automatización del registro de asistencia dentro de un entorno institucional real.

Sistema de examen en línea con reconocimiento facial.

El trabajo denominado Sistema de examen en línea con reconocimiento facial se enfoca en el desarrollo de un sistema web diseñado para reducir intentos de fraude en evaluaciones virtuales. Emplea herramientas de código abierto como OpenCV y Java para verificar la identidad del estudiante tanto al inicio como durante toda la prueba, detectando la ausencia del alumno o la presencia de personas no autorizadas. Las evaluaciones realizadas reportaron resultados favorables en cuanto a satisfacción y funcionamiento general [36].

En comparación con la propuesta desarrollada en este trabajo, ambas propuestas comparten el uso de biometría facial en el ámbito educativo y la necesidad de operar en tiempo real con un nivel adecuado de fiabilidad. También coinciden en la aplicación de técnicas de detección facial como componente esencial para garantizar control y autenticación de usuarios.

Sin embargo, existen diferencias importantes. El anteceden-

te se orienta exclusivamente a la supervisión de exámenes en línea, mientras que la investigación actual está dirigida al registro automatizado de asistencia en un entorno presencial, lo que conlleva requisitos operativos y escenarios de uso distintos. Además, el antecedente previo utiliza métodos de detección facial tradicionales, mientras que la propuesta actual incorpora modelos basados en aprendizaje profundo, capaces de ofrecer mayor precisión, estabilidad y adaptación a variaciones de iluminación, postura y condiciones del entorno. Finalmente, mientras el sistema de exámenes se limita a la validación de identidad, el desarrollo actual integra detección, reconocimiento y registro automático en base de datos, junto con herramientas de gestión institucional.

En síntesis, ambos trabajos comparten fundamentos comunes, pero la investigación actual presenta un alcance más amplio y un enfoque tecnológico más avanzado, orientado a la automatización integral del control de asistencia.

Sistema biométrico de reconocimiento facial utilizando redes neuronales artificiales en Raspberry Pi.

El trabajo denominado Sistema biométrico de reconocimiento facial utilizando redes neuronales artificiales en Raspberry Pi se desarrolla sobre una Raspberry Pi 3 y se centra en la implementación de un sistema biométrico capaz de identificar personas mediante una cámara integrada. Para la detección de rostros emplea un clasificador en cascada tradicional, mientras que el reconocimiento se realiza mediante redes neuronales convolucionales entrenadas para diferenciar los patrones faciales. El objetivo principal de esta propuesta es reducir el tiempo de identificación y mejorar la precisión en diferentes condiciones de uso [37].

En comparación con la propuesta desarrollada en este tra-

bajo, ambos sistemas comparten la utilización de técnicas de inteligencia artificial y visión por computadora para abordar el reconocimiento facial. También coinciden en la búsqueda de un funcionamiento eficiente en tiempo real y en la necesidad de contar con modelos que mantengan una precisión adecuada pese a variaciones en el entorno.

Sin embargo, existen diferencias importantes. El antecedente se implementa en un dispositivo de bajo costo como la Raspberry Pi, lo que limita su capacidad de procesamiento y condiciona el uso de métodos tradicionales de detección facial. En cambio, la investigación actual emplea modelos modernos basados en aprendizaje profundo, como YOLOv8n-Face y ArcFace, que requieren mayor capacidad de cómputo pero ofrecen resultados más precisos y robustos. Además, el trabajo previo se orienta exclusivamente al reconocimiento biométrico individual, mientras que la propuesta actual integra detección, reconocimiento y registro automático en una base de datos, junto con una plataforma web para la gestión académica, ampliando el alcance y la aplicabilidad institucional.

En conjunto, aunque ambos estudios comparten fundamentos relacionados con la identificación facial, la presente investigación avanza hacia una solución más completa y adaptable, orientada a la automatización del control de asistencia en un entorno académico real.

Prototipo de reconocimiento facial para mejorar el control de asistencia de estudiantes en UNIANDES, Quevedo.

El trabajo denominado Prototipo de reconocimiento facial para mejorar el control de asistencia en UNIANDES – Quevedo tiene como finalidad diseñar un sistema que optimice el registro de asistencia mediante reconocimiento facial. Este prototipo au-

tomatiza el proceso tanto para docentes como para estudiantes, permitiendo la identificación de cada persona frente a una cámara ubicada en el acceso del laboratorio de cómputo. Al reconocer al docente, el sistema activa la apertura de la puerta y enciende los dispositivos del laboratorio; posteriormente, los estudiantes se presentan uno por uno para registrar su asistencia comparando los rostros detectados con la base de datos existente [38].

En comparación con la propuesta desarrollada en este trabajo, ambas propuestas comparten el uso del reconocimiento facial para automatizar procesos académicos y reducir la intervención manual. También coinciden en la necesidad de lograr una detección confiable y una identificación precisa para garantizar un registro adecuado.

Sin embargo, existen diferencias importantes. El sistema de UNIANDES se orienta a la automatización del acceso físico y al control de asistencia en un laboratorio específico, mientras que la presente investigación se enfoca en un entorno de aula más amplio, integrando una plataforma web de gestión, almacenamiento centralizado de datos y módulos adicionales como la visualización de reportes. Asimismo, el antecedente utiliza un prototipo con un conjunto de funcionalidades acopladas al hardware del laboratorio, mientras que el trabajo actual incorpora modelos modernos de aprendizaje profundo y una arquitectura cliente-servidor que permite una mayor escalabilidad y flexibilidad operativa.

En síntesis, aunque ambas propuestas comparten la finalidad de mejorar el control de asistencia mediante tecnologías biométricas, la presente investigación abarca un alcance más amplio y una infraestructura más completa, orientada a su integración dentro de procesos institucionales de uso cotidiano.

Registro de asistencia de alumnos por medio de reconocimiento facial utilizando visión artificial.

El trabajo denominado Sistema de control de asistencia para alumnos utilizando reconocimiento facial desarrolla una solución basada en reconocimiento facial para registrar la asistencia de estudiantes. Implementado sobre una Raspberry Pi 3, emplea técnicas como HOG y redes neuronales convolucionales para capturar y comparar imágenes faciales en tiempo real. Este sistema busca ofrecer un método eficiente, preciso y adaptable para instituciones educativas, optimizando el proceso de control de asistencia y reduciendo la intervención manual [33].

En comparación con la propuesta desarrollada en este trabajo, ambos sistemas comparten el uso de técnicas de visión por computadora y redes neuronales para automatizar el registro de asistencia. También coinciden en la necesidad de un funcionamiento estable en tiempo real y en la intención de mejorar la eficiencia en entornos académicos.

Sin embargo, existen diferencias importantes. El antecedente se ejecuta en una Raspberry Pi, lo que limita su capacidad de procesamiento y condiciona el uso de modelos más simples. En cambio, la investigación actual aprovecha modelos contemporáneos basados en aprendizaje profundo, como YOLOv8n-Face y ArcFace, que ofrecen mayor precisión y robustez en escenarios reales de aula. Además, mientras el sistema previo se centra exclusivamente en el registro de asistencia, la propuesta desarrollada incorpora una arquitectura cliente-servidor, una base de datos centralizada y una interfaz web que permite administrar cursos, usuarios y reportes, ampliando significativamente su alcance institucional.

En síntesis, aunque ambos trabajos buscan resolver problemas similares mediante reconocimiento facial, la presente investigación abarca un diseño más completo, escalable y adaptable

a las necesidades de una institución académica moderna.

Reconocimiento facial para la automatización del registro de asistencia a clases.

Este antecedente presenta un sistema piloto de registro automático de asistencia a clases presenciales basado en técnicas de detección y reconocimiento facial mediante modelos de aprendizaje profundo. Su arquitectura modular integra contenedores, Python y frameworks como FastAPI y Django, así como herramientas de aprendizaje automático entre las que se incluyen Keras, TensorFlow, PyTorch, OpenCV y MXNet. El proceso de identificación se realiza mediante la extracción y almacenamiento de características faciales en forma de vectores, comparando similitudes entre ellos para reconocer a cada persona sin necesidad de reentrenar continuamente las redes neuronales [39].

En comparación con la propuesta desarrollada en este trabajo, ambos sistemas comparten el uso de modelos modernos de aprendizaje profundo y el empleo de embeddings faciales para realizar la identificación, lo que permite un funcionamiento eficiente sin reentrenamiento para cada nuevo usuario. También coinciden en buscar un desempeño confiable en tiempo real y en utilizar arquitecturas modulares.

Sin embargo, existen diferencias relevantes. El sistema piloto analizado se orienta a un entorno experimental con múltiples frameworks especializados, mientras que la presente investigación apuesta por un diseño más liviano y adaptado al contexto institucional, basado en una arquitectura cliente-servidor que facilita la implementación y el mantenimiento. Asimismo, el desarrollo actual integra no solo el reconocimiento facial, sino también el registro automático en una base de datos centralizada y una plataforma web para la gestión académica, ampliando su alcance y su aplicabilidad en aulas de clase.

En síntesis, aunque este antecedente comparte principios tecnológicos avanzados, la propuesta desarrollada en esta investigación se distingue por ofrecer una solución integral orientada a la automatización del control de asistencia dentro de una institución educativa.

Capítulo 3

Herramientas Tecnológicas

En el presente capítulo se detallan las herramientas y lenguajes de programación considerados en el desarrollo del trabajo, seleccionados por su relevancia y eficacia en la implementación de sistemas de reconocimiento facial. La elección de cada tecnología se basó en criterios de rendimiento, estabilidad, escalabilidad e integración entre los distintos módulos del sistema, priorizando aquellas que ofrecen soporte robusto para tareas de visión por computadora, aprendizaje automático y procesamiento en tiempo real.

3.1. Selección de Herramientas

Python: Lenguaje de alto nivel, interpretado y de código abierto, creado por Guido van Rossum en 1991. Destaca por su sintaxis simple, legible y su extenso ecosistema de librerías para inteligencia artificial y visión por computadora [40].

C++: Lenguaje de programación de medio-alto nivel creado por Bjarne Stroustrup, reconocido por su alto rendimiento y control de memoria. Es ampliamente utilizado en sistemas de tiempo real y compatible con librerías como *OpenCV*, aunque presenta una curva de aprendizaje más pronunciada [41].

Java: Lenguaje de propósito general, orientado a objetos y

multiplataforma, desarrollado por Sun Microsystems en 1995. Su filosofía “escribir una vez, ejecutar en cualquier parte” y su máquina virtual (*JVM*) le brindan estabilidad, portabilidad y seguridad en aplicaciones empresariales y móviles [42].

C# (.NET): Lenguaje moderno desarrollado por Microsoft, integrado en la plataforma .NET. Combina productividad, soporte empresarial y capacidad para integrar APIs biométricas, siendo especialmente eficiente en entornos Windows y servicios en la nube Azure [43].

Criterios de selección de herramientas

- **Licencia:** Evalúa si la herramienta es de uso gratuito o de código abierto, y si permite su modificación o distribución. Este criterio influye directamente en la accesibilidad, escalabilidad y sostenibilidad del trabajo.
- **Lenguaje:** Considera la flexibilidad, el paradigma de programación y el enfoque del lenguaje con el que opera la herramienta, así como su compatibilidad con bibliotecas de inteligencia artificial o visión por computadora.
- **Instalación:** Mide la facilidad de instalación y configuración del entorno de desarrollo, incluyendo la disponibilidad de documentación oficial, gestores de dependencias y compatibilidad con distintos sistemas operativos.
- **Curva de aprendizaje:** Analiza el nivel de complejidad necesario para dominar la herramienta, considerando la claridad de su sintaxis, la documentación disponible y la cantidad de recursos formativos accesibles para los desarrolladores.
- **Ecosistema y bibliotecas:** Evalúa la cantidad, calidad y madurez de las bibliotecas y frameworks disponibles para la herramienta, especialmente aquellas relacionadas con el

desarrollo web, la inteligencia artificial y el procesamiento de imágenes.

- **Seguridad:** Considera la capacidad de la herramienta para implementar prácticas seguras en el desarrollo, incluyendo manejo de autenticación, protección contra vulnerabilidades comunes y mantenimiento activo de versiones.
- **Rendimiento:** Analiza la eficiencia de ejecución del lenguaje y la herramienta, especialmente en tareas intensivas como el procesamiento de imágenes o el manejo de múltiples peticiones simultáneas.
- **Comunidad y soporte:** Evalúa el tamaño y la actividad de la comunidad de desarrolladores, así como la disponibilidad de documentación, foros de soporte, actualizaciones frecuentes y contribuciones de código abierto.

Escala de evaluación

1. **Muy Deficiente / No cumple:** La herramienta no cumple con el criterio o tiene un rendimiento muy pobre en esa área. Necesita mejoras significativas o carece de esa funcionalidad.
2. **Deficiente:** La herramienta cumple parcialmente con el criterio, pero presenta limitaciones o fallos importantes que afectan su rendimiento. Requiere mejoras significativas.
3. **Aceptable / Promedio:** La herramienta cumple con el criterio de manera funcional, pero con limitaciones que afectan su eficacia en comparación con otras opciones. Es una alternativa razonable, aunque no sobresaliente.
4. **Bueno:** La herramienta cumple con el criterio de manera sólida y efectiva, con pocas limitaciones. Tiene un rendimiento consistente, aunque puede mejorarse en algunos aspectos.

5. **Excelente:** La herramienta cumple de manera sobresaliente con el criterio, sin limitaciones significativas. Ofrece un rendimiento superior, una comunidad activa y características avanzadas que la hacen destacar en su categoría.

Se observa en la Tabla 3.1 la comparación entre los lenguajes de programación evaluados según los criterios establecidos.

Tabla 3.1: Comparación de lenguajes según los criterios de selección Fuente: [Elaboración Propia].

Criterio	Python	C++	Java	C#
Licencia	5	5	5	5
Lenguaje	5	3	4	4
Instalación	5	3	4	4
Curva de aprendizaje	5	2	3	4
Ecosistema y bibliotecas	5	4	4	4
Seguridad	5	3	5	5
Rendimiento	4	5	4	4
Comunidad y soporte	5	4	4	4
Total de puntos	39	29	33	34

Herramienta seleccionada a base de los criterios y escala

Luego de evaluar los lenguajes de programación *Python*, *C++*, *Java* y *C#* en función de ocho criterios definidos, Licencia, Lenguaje, Instalación, Curva de aprendizaje, Ecosistema y bibliotecas, Seguridad, Rendimiento y Comunidad y soporte; Aplicando una escala de valoración del 1 al 5, se concluye que *Python* es el lenguaje más adecuado para el desarrollo del presente trabajo.

Python obtuvo un total de 39 puntos sobre 40 posibles, destacándose principalmente en los aspectos de facilidad de aprendizaje, ecosistema de librerías, comunidad activa y portabilidad.

Su sintaxis simple, su amplia disponibilidad de frameworks como *Flask*, *TensorFlow*, *OpenCV* y *DeepFace*, así como su integración nativa con entornos científicos, lo convierten en una herramienta altamente eficiente para trabajos basados en visión por computadora y reconocimiento facial.

Por su parte, *C++* obtuvo 29 puntos, destacando por su alto rendimiento y control de memoria, pero con una curva de aprendizaje más pronunciada y un desarrollo menos ágil. *Java*, con 33 puntos, se caracteriza por su portabilidad y seguridad, aunque su sintaxis más extensa y menor especialización en bibliotecas de inteligencia artificial limitan su aplicabilidad directa. Finalmente, *C#* alcanzó 34 puntos, sobresaliendo en seguridad y soporte empresarial, aunque su dependencia del entorno *.NET* reduce su flexibilidad multiplataforma.

En conclusión, *Python* fue seleccionado como el lenguaje principal para la implementación del sistema de reconocimiento facial, al cumplir de forma sobresaliente con la mayoría de los criterios establecidos, posicionándose como la opción más robusta, flexible y eficiente para el desarrollo de soluciones basadas en inteligencia artificial y visión por computadora.

3.1.1. Python

Python es un lenguaje de programación de alto nivel, interpretado y de código abierto, creado por Guido van Rossum en 1991. Diseñado con un enfoque en la simplicidad y la legibilidad, permite expresar ideas complejas con pocas líneas de código, lo que facilita su aprendizaje y lo hace accesible tanto para principiantes como para desarrolladores experimentados [40].

Es un lenguaje multiparadigma y multiplataforma que admite programación estructurada, orientada a objetos y funcional, lo que le otorga una gran flexibilidad para adaptarse a distintos tipos de proyectos y entornos de desarrollo [44]. Su sintaxis

clara y su amplia biblioteca estándar han contribuido a que se consolide como una de las herramientas más utilizadas en campos como la inteligencia artificial, el aprendizaje automático y la visión por computadora.

Asimismo, su ecosistema de librerías entre las que destacan *NumPy*, *Pandas*, *OpenCV*, *TensorFlow* y *DeepFace*, proporciona soporte para tareas de análisis, procesamiento y reconocimiento de imágenes, aportando una solución robusta y eficiente para la creación de sistemas inteligentes. Estas características hacen de Python un lenguaje moderno, adaptable y esencial para el desarrollo del sistema de reconocimiento facial implementado en este trabajo [40].

Lenguajes complementarios y gestor de base de datos

Además del lenguaje principal empleado para el desarrollo del backend, el sistema requirió herramientas complementarias para la implementación del entorno web y la gestión de datos. En el caso del *frontend*, se utilizó *JavaScript* como lenguaje base para la construcción de la interfaz gráfica, mientras que para la capa de persistencia de datos se empleó el gestor de base de datos *PostgreSQL*, garantizando una comunicación eficiente, segura y escalable entre el servidor y la aplicación.

JavaScript es un lenguaje de programación diseñado para desarrollar tanto la parte *Front End* como el *Back End* de sitios web y aplicaciones móviles. Es un lenguaje de alto nivel, legible y dinámico, que no requiere compilación para su ejecución. Su código se interpreta directamente en el navegador, lo que permite una interacción fluida y en tiempo real con el usuario. Se clasifica como un lenguaje de *scripts*, utilizado para añadir funcionalidades interactivas, como menús desplegados, validaciones de formularios, animaciones y componentes dinámicos que enriquecen la experiencia de usuario. Estas características hacen de *JavaScript* una herramienta fundamental en el desarrollo moderno de aplicaciones web, gracias a su compatibilidad con

frameworks y bibliotecas como React, Angular y Node.js [45].

Por su parte, *PostgreSQL* es un potente sistema de gestión de bases de datos relacional y orientado a objetos, completamente de código abierto. Basado en el lenguaje SQL, combina estabilidad, escalabilidad y un conjunto robusto de funcionalidades avanzadas que permiten manejar de manera segura y eficiente grandes volúmenes de datos. Sus orígenes se remontan a 1986, en el marco del trabajo “Postgres” de la Universidad de California en Berkeley, y desde entonces ha mantenido más de tres décadas de desarrollo continuo. Se destaca por su integridad transaccional, extensibilidad, arquitectura sólida y su compatibilidad multiplataforma, lo que lo convierte en una opción ideal para proyectos que demandan alta confiabilidad en el almacenamiento y procesamiento de información [46].

3.2. Entorno de desarrollo del modelo

3.2.1. Visual Studio Code

Para la etapa de codificación del modelo y los componentes del sistema, se utilizó *Visual Studio Code* (VS Code), un editor de código fuente desarrollado por Microsoft, diseñado para ser ligero, rápido y altamente personalizable. A diferencia de otros editores de código, como los IDE completos (Entornos de Desarrollo Integrados), VS Code se enfoca en ofrecer una experiencia ágil para la escritura y edición de código, sin sacrificar características avanzadas [47]. Lo que lo diferencia es su capacidad de ser extendido con una gran cantidad de extensiones y herramientas, adaptándose a las necesidades específicas de cada desarrollador.

3.2.2. GitHub Desktop

Para gestionar el control de versiones del trabajo y mantener un historial estructurado de los cambios realizados durante el desarrollo, se empleó *GitHub Desktop*. Esta herramienta gratuita y de código abierto proporciona una interfaz gráfica para trabajar con repositorios Git alojados en GitHub u otros servicios. Facilita la realización de operaciones comunes de Git, como clonar, confirmar (*commit*), crear ramas y sincronizar cambios, sin necesidad de utilizar la línea de comandos. Está diseñada para simplificar el flujo de trabajo tanto para principiantes como para usuarios avanzados [48].

3.2.3. Postman

Finalmente, para la validación de los endpoints API y la verificación de la comunicación entre los módulos del sistema, se utilizó *Postman*. Es una plataforma orientada a la prueba y gestión de APIs, que facilita a los desarrolladores la verificación y documentación de servicios web. Inicialmente creada como una extensión del navegador, hoy está disponible como aplicación independiente para Windows, Linux y macOS. Su interfaz gráfica permite enviar solicitudes HTTP, automatizar pruebas y generar documentación, optimizando el desarrollo y reduciendo errores en la comunicación con servicios externos [49].

3.3. Librerías utilizadas

3.3.1. React

Para el desarrollo del frontend y del panel de control (*dashboard*) del sistema, se utilizó *React* en su versión 19.1.1. React es una biblioteca JavaScript de código abierto desarrollada por el equipo de Meta, en colaboración con una amplia comunidad de

desarrolladores. Desde su lanzamiento en 2013, se ha consolidado como una de las tecnologías más utilizadas para la construcción de interfaces de usuario dinámicas y escalables.

Su funcionamiento se basa en la creación de componentes reutilizables que integran estructura y lógica mediante la sintaxis JSX. Este enfoque permite gestionar la interfaz de forma eficiente mediante un *Virtual DOM*, el cual detecta cambios de estado y actualiza únicamente los elementos necesarios sin recargar toda la página, optimizando el rendimiento del sistema [50].

3.3.2. Flask

Para el desarrollo del backend y la implementación de las rutas API del sistema, se utilizó el microframework *Flask* en su versión 3.1.2. Flask es un microframework web de código abierto escrito en Python, orientado al desarrollo rápido y flexible de aplicaciones. Su estructura modular y extensible otorga a los desarrolladores un control total sobre la arquitectura, destacándose por su sencillez, ligereza e integración con múltiples extensiones.

Estas características lo convierten en una herramienta ideal para la creación de APIs RESTful y servicios que requieren agilidad y adaptabilidad, motivo por el cual fue seleccionado para la construcción de los endpoints del sistema [51].

3.3.3. YOLO

Para la etapa de detección facial del sistema se utilizó el modelo *YOLOv8*, implementado mediante la librería *Ultralytics* en su versión 8.3.191. YOLO (You Only Look Once) es un algoritmo de detección de objetos en tiempo real basado en redes neuronales convolucionales. Introducido por Joseph Redmon y colaboradores en 2015, YOLO se caracteriza por procesar una imagen completa en una sola pasada, dividiéndola en una cuadrícula

y prediciendo simultáneamente las clases y ubicaciones de los objetos presentes.

Esta arquitectura unificada permite una detección rápida y precisa, siendo especialmente útil en aplicaciones que requieren procesamiento en tiempo real, motivo por el cual fue seleccionado para la detección facial dentro del sistema [52].

3.3.4. DeepFace

Para la etapa de reconocimiento facial se utilizó la librería *DeepFace* en su versión 0.0.95. DeepFace es un framework de código abierto basado en *Python* que unifica diversas arquitecturas de reconocimiento facial y facilita su implementación mediante funciones de alto nivel. Entre sus características principales se destacan el soporte para múltiples modelos de extracción de embeddings, detección facial, análisis demográfico y verificación de identidad [53].

En este trabajo, *DeepFace* fue utilizada específicamente para el proceso de identificación facial, empleando el modelo *ArcFace* como generador de embeddings, permitiendo un reconocimiento más robusto y confiable dentro del sistema desarrollado.

Capítulo 4

Método

Se presenta la planificación general del trabajo, desde la definición de los requisitos iniciales hasta la validación y evaluación del prototipo en un entorno académico real con un grupo de $N=13$ estudiantes participantes.

En primer lugar, se describe el enfoque metodológico adoptado, el cual combina técnicas cualitativas y cuantitativas bajo un esquema de investigación aplicada tecnológica, lo que permitió realizar un análisis integral del problema y de la solución propuesta. Posteriormente, se exponen las fases específicas del proceso de desarrollo, tales como la recolección y preprocesamiento de datos, la selección de algoritmos, el entrenamiento del modelo y la integración del sistema en un entorno controlado, para finalmente evaluarlo en condiciones reales de aula.

4.1. Enfoque

El estudio empleó un enfoque mixto que integró desarrollo tecnológico e investigación experimental, combinando métodos cuantitativos y cualitativos para evaluar tanto el desempeño técnico del sistema como su aplicabilidad en el entorno académico.

Desde el punto de vista cuantitativo, se midieron indicado-

res de rendimiento como la precisión del reconocimiento facial, el tiempo promedio de procesamiento por rostro y la tasa de error del sistema. En el componente cualitativo, se consideraron observaciones y percepciones de los usuarios sobre la facilidad de uso, la fiabilidad del registro y la utilidad del sistema en la gestión académica.

El tipo de investigación corresponde a una investigación aplicada con alcance descriptivo–explicativo, orientada al diseño, desarrollo y validación de un sistema funcional de asistencia automatizada mediante reconocimiento facial.

El diseño adoptado es de tipo experimental, dado que se evaluó el sistema en condiciones reales de aula, controlando variables como iluminación, distancia de captura y número de estudiantes presentes. El carácter longitudinal del estudio permitió analizar el comportamiento y la estabilidad del sistema a lo largo de un periodo académico completo, estableciendo así evidencia empírica sobre su eficacia y consistencia operativa.

4.2. Método utilizado para el desarrollo

El desarrollo de este trabajo se llevó a cabo siguiendo la metodología ágil *Scrum*, adaptada al contexto académico de la investigación. Este enfoque permitió organizar las tareas en ciclos cortos denominados *sprints*, priorizando la entrega progresiva de funcionalidades y la mejora continua a lo largo del proceso.

Roles del equipo: El *Product Owner* fue el profesor Jorge Arrúa, quien definía los requerimientos y evaluaba los entregables en cada revisión. El rol de *Scrum Master* fue desempeñado por Daniel Sanchez, responsable de coordinar las tareas, facilitar la comunicación y asegurar el cumplimiento de los objetivos de cada sprint. El *Development Team* estuvo conformado por Aldo Carrizo y Daniel Sanchez, encargados del desarrollo, pruebas e integración del sistema.

Planificación de los sprints: El desarrollo del sistema se organizó en catorce sprints de aproximadamente dos semanas cada uno, cubriendo el periodo comprendido entre abril y octubre de 2025. Esta planificación permitió avanzar de forma iterativa, incorporando mejoras continuas en cada ciclo de trabajo. A continuación, se describen las principales actividades desarrolladas en cada fase:

- **Sprint 1:** Definición de objetivos, alcance y requerimientos del sistema. Configuración inicial del repositorio en GitHub.
- **Sprint 2:** Instalación del entorno de desarrollo y configuración de dependencias para los módulos de detección y reconocimiento facial.
- **Sprint 3:** Diseño del Diagrama Entidad-Relación (DER) y creación de la base de datos en PostgreSQL.
- **Sprint 4:** Implementación del algoritmo de detección facial utilizando el modelo preentrenado *YOLOv8n-face.pt*, realizando las primeras pruebas con imágenes estáticas para validar su funcionamiento.
- **Sprint 5:** Implementación del algoritmo de reconocimiento facial con *DeepFace* (modelo ArcFace) y generación inicial de embeddings faciales.
- **Sprint 6:** Pruebas unitarias y evaluación de precisión del modelo de DeepFace. Ajuste de parámetros para reducir errores de identificación.
- **Sprint 7:** Integración entre YOLO y DeepFace para el flujo completo de detección y reconocimiento facial.
- **Sprint 8:** Desarrollo de las *APIs REST* del backend para usuarios, roles, cursos, asistencias y autenticación.

- **Sprint 9:** Implementación de la lógica de registro automático y manual de asistencias. Optimización del rendimiento del sistema.
- **Sprint 10:** Desarrollo del *frontend* y diseño del dashboard principal. Implementación del módulo de inicio de sesión.
- **Sprint 11:** Creación de los módulos de gestión de alumnos, cursos, horarios y asignaciones dentro del panel administrativo.
- **Sprint 12:** Pruebas en entornos aislados y simulaciones en laboratorio para validar la estabilidad del sistema.
- **Sprint 13:** Pruebas en el aula, corrección de incidencias, optimización final del código, despliegue controlado y documentación técnica y de usuario.

Herramientas de gestión: Para la planificación, seguimiento y control del trabajo se empleó la herramienta *Notion*, que permitió organizar las tareas de acuerdo con su estado de avance (pendiente, en progreso, en revisión y completadas). Como se observa en la Figura 4.1, esta plataforma facilitó la coordinación entre los integrantes del equipo, la priorización de actividades y el registro de los avances en cada *sprint*, brindando una visión clara del progreso general del sistema desarrollado.

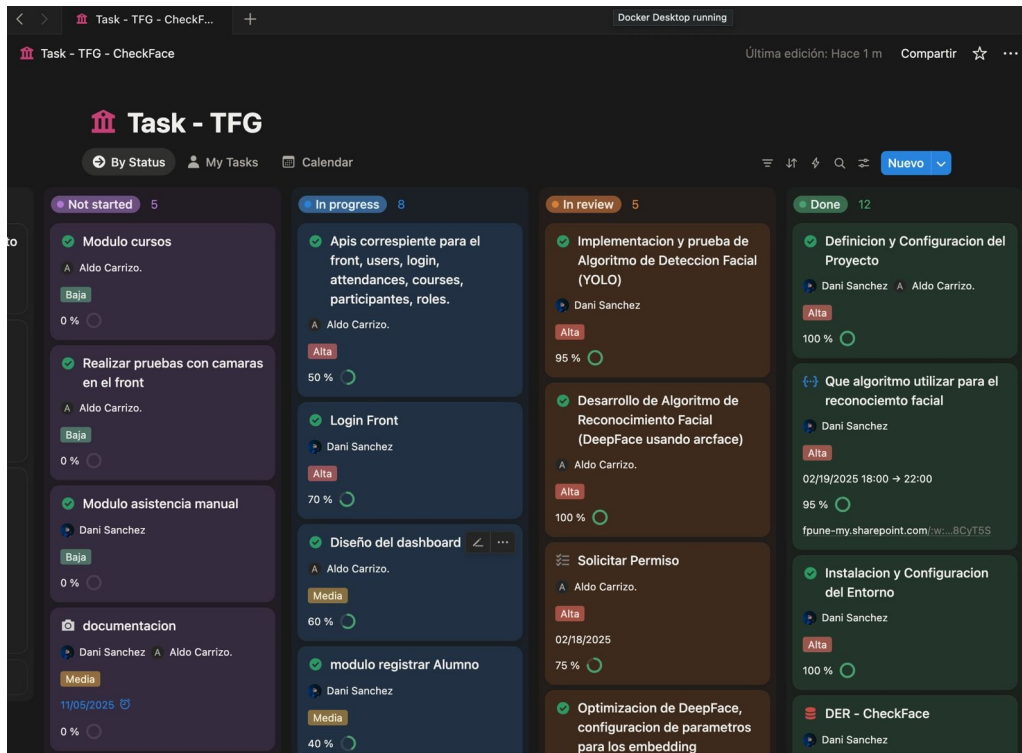


Figura 4.1: Tablero de gestión de tareas del trabajo en Notion, bajo la metodología ágil Scrum Fuente: [Elaboración Propia].

Reuniones y seguimiento: El equipo realizó reuniones diarias breves entre los desarrolladores para resolver dificultades técnicas y coordinar actividades. Además, cada dos semanas se llevaron a cabo reuniones presenciales con el profesor Jorge Arrúa para presentar avances, recibir retroalimentación y definir los objetivos del siguiente sprint.

Duración del trabajo: El desarrollo completo del sistema se extendió desde abril hasta octubre de 2025, periodo durante el cual se logró diseñar, integrar, probar y documentar la solución final de reconocimiento facial para el registro de asistencia.

4.3. Fases del proceso investigativo

- Identificación de requisitos del trabajo

En esta primera fase se realizó el levantamiento de información mediante una encuesta aplicada a aproximadamente 15 personas, entre docentes y estudiantes de la Facultad Politécnica de la Universidad Nacional del Este (FPUNE). El objetivo fue identificar las principales necesidades y expectativas respecto al control de asistencia automatizado, así como los problemas presentes en el sistema manual existente. A partir de las respuestas obtenidas se definieron los requisitos funcionales y no funcionales del sistema, estableciendo las funcionalidades esenciales, los recursos técnicos requeridos y las condiciones de uso dentro del entorno académico.

- Recolección y preparación de datos

Para conformar la base de datos facial, se llevó a cabo una sesión fotográfica dentro del aula donde posteriormente se realizaron las pruebas del sistema. Las imágenes fueron capturadas utilizando un teléfono celular, manteniendo una distancia aproximada de un metro entre la cámara y el rostro de cada participante. Cada usuario fue fotografiado en múltiples posiciones faciales (frontal, ligeramente lateral, mirada hacia abajo o arriba) y con variaciones de accesorios, como anteojos o gorras, con el fin de aumentar la robustez del modelo ante diferentes condiciones. Para garantizar una correcta identificación, se requirió un mínimo de cinco fotografías por participante, aunque la cantidad final de imágenes varió según cada usuario y las condiciones en que fueron tomadas. En total, se obtuvieron imágenes correspondientes a trece personas, las cuales fueron almacenadas cumpliendo las normas éticas del trabajo

y con consentimiento verbal informado de los participantes. Posteriormente, todas las fotografías fueron sometidas a un proceso de preprocesamiento que incluyó normalización, reducción de ruido y ajustes de brillo y contraste, con el objetivo de mejorar la calidad general del conjunto de datos.

- Entrenamiento del modelo

Para la detección facial se empleó un modelo YOLOFace preentrenado, el cual ya incluye una red neuronal convolucional entrenada en grandes conjuntos de rostros. En este trabajo, el modelo fue utilizado como base para realizar la detección de rostros de los usuarios locales, a quienes posteriormente se les generaron representaciones vectoriales (embeddings) mediante la biblioteca DeepFace y el modelo ArcFace. Estos embeddings se almacenan en la base de datos del sistema, permitiendo reconocer posteriormente a cada persona de manera automática durante la toma de asistencia.

- Diseño del sistema

Se definió una arquitectura modular que integra los componentes de captura, detección, reconocimiento y registro en la base de datos. El backend fue desarrollado con el framework Flask, encargado de gestionar las solicitudes, procesar las imágenes y registrar los datos en la base de datos PostgreSQL. El frontend fue implementado en React, ofreciendo una interfaz moderna e interactiva que permite visualizar la cámara en tiempo real, gestionar usuarios y consultar asistencias registradas.

4.4. Análisis y selección de herramientas

Durante esta fase se realizó un análisis comparativo de diversas tecnologías y librerías aplicadas al desarrollo de sistemas de reconocimiento facial. La selección de herramientas se basó en criterios como precisión, rendimiento, compatibilidad, facilidad de integración y disponibilidad de documentación.

Es importante destacar que las tecnologías elegidas no fueron seleccionadas de manera aleatoria, sino a partir de la experiencia y el conocimiento previo del equipo en el uso de entornos y librerías de visión por computadora. Sin embargo, antes de adoptar una decisión definitiva, se llevaron a cabo pruebas experimentales con distintas alternativas para evaluar su desempeño en las condiciones específicas del trabajo.

Estas evaluaciones permitieron determinar qué herramientas ofrecían los mejores resultados en términos de detección, reconocimiento y velocidad de procesamiento, considerando las limitaciones del hardware disponible y el entorno académico en el que se desarrollaría el sistema.

A continuación, se detallan las principales herramientas seleccionadas:

- **Modelos de detección y reconocimiento facial:**

Para la etapa de detección de rostros se analizaron diversas alternativas basadas en aprendizaje profundo, considerando factores como la precisión, la velocidad de procesamiento y la capacidad de operar en tiempo real. Tras la comparación de diferentes tecnologías, se determinó que *YOLO-Face* ofrece el mejor equilibrio entre exactitud y eficiencia, motivo por el cual fue seleccionado como modelo principal de detección facial.

Tabla 4.1: Resultados principales de los modelos de detección [8].

Modelo	Precisión	Tiempo de Proceso	mAP	Observaciones
YOLOv8	0.71	1 ms	0.71	Mejor equilibrio entre velocidad y exactitud; ideal para tiempo real.
RetinaNet	0.67	1.236 s	0.67	Focal Loss mejora el manejo de clases desequilibradas, con menor velocidad.
EfficientDet	0.55	1.0 s	0.55	Eficiente en recursos; recomendado para hardware limitado.

Nota: mAP (mean Average Precision) indica la precisión promedio media del modelo; valores más altos reflejan mayor exactitud en la detección.

- **Comparativa de modelos YOLO para detección facial**

Se evaluaron diferentes versiones de YOLO adaptadas al reconocimiento facial. Finalmente, se seleccionó YOLOv8n-Face por su buen equilibrio entre precisión y velocidad.

Tabla 4.2: Comparativa de modelos YOLO para detección facial [9].

Modelo	mAP	Precisión	Observaciones
YOLOv5n-Face	~0.31	~0.60	Muy liviano y rápido, pero con menor precisión general.
YOLOv5s-Face	~0.34	~0.65	Mejor precisión que v5n. Rendimiento estable en equipos de gama media.
YOLOv8n-Face	0.375	0.625	Excelente equilibrio entre precisión y velocidad.
YOLOv8s-Face	~0.43	~0.70	Mayor precisión con ligero aumento en consumo de recursos. Adecuado para entornos con GPU moderada.
YOLOv8m-Face	~0.46	~0.72	Alta precisión y detección robusta, aunque con mayor carga computacional. Ideal para pruebas exhaustivas.

- **Evaluación según nivel de dificultad (WIDER Face)**

La tabla presenta el desempeño de distintos modelos de detección facial evaluados en el conjunto de datos WIDER Face, considerando tres niveles de dificultad: fácil, medio y difícil. Las condiciones más complejas incluyen rostros pequeños, parcialmente visibles o con iluminación deficiente. Los resultados evidencian que la versión YOLOv8-Face mantiene un rendimiento más consistente y robusto frente a escenarios desafiantes.

Tabla 4.3: Rendimiento de modelos YOLO en distintos niveles de dificultad (WIDER Face) [9].

Modelo	Easy (%)	Medium (%)	Hard (%)
YOLOv5s-Face	91.20	89.10	71.30
YOLOv8n-Face	93.79	91.82	79.38
YOLOv8s-Face	95.13	93.62	82.90
YOLOv8x-Face	96.33	95.16	85.80

• Comparativa de modelos DeepFace

La tabla muestra distintos modelos compatibles con DeepFace evaluados en cuanto a precisión, velocidad, tamaño, métrica de comparación y sus ventajas y desventajas. Se destaca ArcFace como el más preciso, aunque requiere buen preprocesamiento.

Tabla 4.4: Comparativa de modelos de DeepFace [9].

Modelo	Precisión (LFW)	Velocidad	Tamaño	Métrica
ArcFace	99.83 %	Media	~68 MB	Cosine / Euclídea
VGG-Face	98.78 %	Lenta	~500 MB	Cosine
Facenet	99.20 %	Rápida	~90 MB	Euclídea
Facenet512	99.25 %	Media	~125 MB	Euclídea
Dlib	96.00 %	Muy rápida	~5 MB	Euclídea
SFace	99.65 %	Media	~100 MB	Cosine

Tabla 4.5: Comparativa de modelos de DeepFace [9].

Modelo	Ventajas principales	Desventajas
ArcFace	Alta precisión y robustez ante cambios de pose o luz	Requiere preprocesamiento cuidadoso
VGG-Face	Modelo base de DeepFace, bien entrenado	Lento y de gran tamaño
Facenet	Ligero y estable	Menor precisión que ArcFace
Facenet512	Versión más precisa de Facenet	Mayor peso del modelo
Dlib	Muy liviano, ideal para CPU	Precisión limitada
SFace	Buen desempeño en diversas condiciones	En evaluación por la comunidad

- **Framework de desarrollo backend:**

Para la construcción de la API y la gestión de la lógica del sistema se empleó *Flask*, un microframework de Python que permite desarrollar aplicaciones ligeras y escalables. Su estructura modular facilita la integración con librerías de visión por computadora y bases de datos.

- **Framework de desarrollo frontend:**

La interfaz de usuario fue desarrollada en *React*, permitiendo una interacción dinámica y fluida con el sistema. Su arquitectura basada en componentes favorece la reutilización del código y la visualización en tiempo real de los registros de asistencia.

- **Base de datos:**

Se utilizó *PostgreSQL* para el almacenamiento estructurado de la información, incluyendo los registros de asistencia y datos de usuarios. Su robustez y soporte para operaciones concurrentes la convierten en una opción óptima para sistemas de este tipo.

- **Herramienta de pruebas de API:**

Durante el proceso de desarrollo e integración, se empleó *Postman* para realizar pruebas de comunicación entre el frontend y el backend. Esta herramienta permitió verificar el correcto funcionamiento de las rutas, peticiones y respuestas del sistema antes de su despliegue final.

La combinación de estas herramientas permitió un desarrollo eficiente, garantizando la integración adecuada entre los módulos de detección, reconocimiento y gestión de asistencia dentro del sistema.

4.5. Recolección de datos

Para el funcionamiento del sistema de reconocimiento facial, fue necesario disponer de un conjunto de imágenes faciales representativas de los usuarios. Durante el registro de cada nuevo usuario, el sistema captura un mínimo de cinco imágenes desde diferentes ángulos y expresiones, con el fin de mejorar la capacidad de reconocimiento en distintas condiciones de uso.

Las imágenes se obtuvieron con cámaras externas (teléfonos móviles, webcams o cámaras digitales) y posteriormente se cargan al sistema mediante el módulo de registro de nuevos estudiantes. Durante este proceso se verifica el formato y un tamaño mínimo recomendado para preservar los rasgos faciales. Cada archivo se organizó en una carpeta individual asociada al usuario correspondiente y se nombra de forma sistemática, facilitando su identificación y el procesamiento posterior.

Todas las capturas fueron realizadas cumpliendo las normas éticas establecidas para el desarrollo del trabajo y con consentimiento verbal informado de los participantes, garantizando el uso responsable y confidencial de los datos obtenidos.

En la Figura 4.2 se muestra un ejemplo de una imagen facial capturada al natural durante el proceso de registro, mientras

que la Figura 4.3 presenta el resultado tras aplicar el preprocesamiento correspondiente.

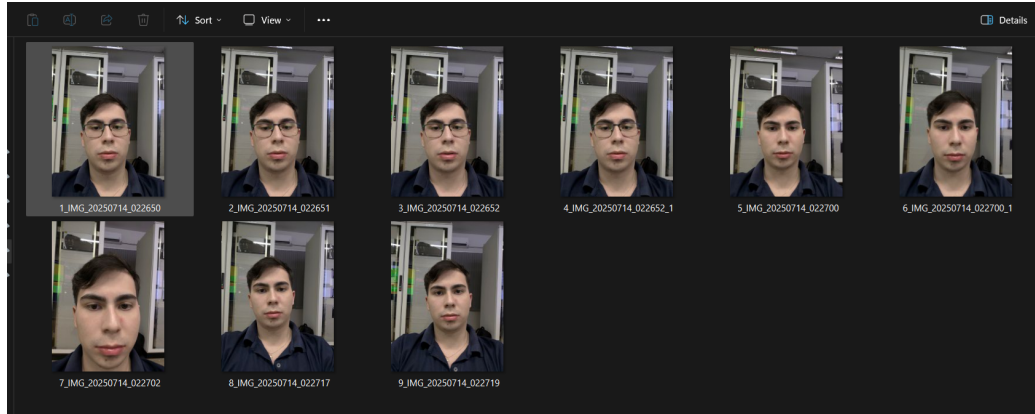


Figura 4.2: Ejemplo de imagen facial capturada al natural Fuente: [Elaboración Propia].

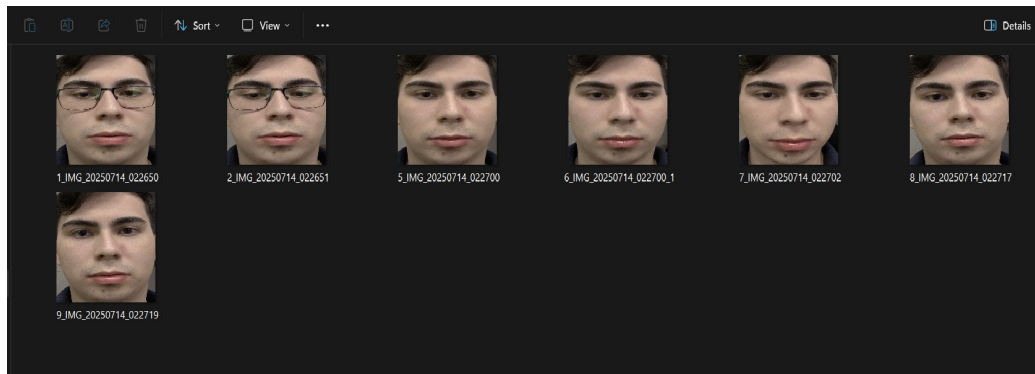


Figura 4.3: Ejemplo de imagen facial luego del proceso de normalización Fuente: [Elaboración Propia].

4.6. Entrenamiento del modelo

El entrenamiento del modelo constituye una etapa esencial del sistema, ya que permite obtener representaciones numéricas únicas de cada rostro a partir de las imágenes registradas. En esta fase se utilizaron técnicas de aprendizaje profundo orientadas

a la verificación facial, capaces de generar *embeddings* discriminativos y estables frente a variaciones de iluminación, expresión o ángulo de captura, garantizando así una identificación confiable y precisa.

Configuración del entorno de entrenamiento

- **Lenguaje:** Python 3.12.11
- **Librerías principales:** DeepFace, OpenCV, NumPy, Ultralytics
- **Entorno:** Visual Studio Code (ejecución local con GPU o CPU)
- **Equipo utilizado:** AMD Ryzen 7 7735HS, 16 GB RAM, SSD 512 GB, Windows 11 Home
- **Dispositivo de captura:** Cámara web HD 720p para obtención de imágenes faciales en pruebas locales.
- **Condiciones de iluminación:** Pruebas realizadas entre 300–500 lux para simular entornos reales de aula. La iluminación fue cuantificada utilizando un luxómetro digital portátil modelo *Dr.meter LX1330B*, con lecturas tomadas a una altura aproximada de 1 a 1.5 metros, equivalente al nivel de una mesa, antes de cada sesión experimental.
- **Estructura del dataset:** Imágenes organizadas por usuario en carpetas dentro de `raw_faces/` (imágenes originales) y `known_faces/` (rostros recortados).
- **Modelo de embeddings:** *ArcFace* (512 dimensiones).
- **Métrica de similitud:** Distancia coseno para comparar embeddings entre el rostro capturado y los centroides registrados.

- **Umbral de decisión:** 0.45 como límite para determinar coincidencia; valores superiores clasifican el rostro como desconocido.
- **Estrategia de detección:** `enforce_detection = False` y `detector_backend = "skip"` para acelerar el procesamiento al asumir que los rostros ya se encuentran recortados.
- **Cálculo de similitud:** Conversión de la distancia coseno a porcentaje mediante una escala lineal para interpretación del resultado.
- **Formato de salida:** Archivos JSON con los embeddings y centroides por usuario.

Proceso de entrenamiento

El proceso de entrenamiento fue automatizado mediante el script `train_faces.py`, compuesto por dos fases principales: el preprocesamiento de imágenes y la generación de representaciones faciales.

1. Registro del usuario e ingreso de imágenes:

Desde la interfaz web, un usuario administrativo registra a un nuevo alumno y carga sus fotografías faciales. Cada conjunto de imágenes se guarda en una carpeta individual dentro del directorio `raw_faces/`, identificada por el número de cédula del alumno (CI).

2. Detección y recorte facial:

Para garantizar que solo se utilicen regiones válidas del rostro, el sistema emplea el modelo *YOLO-Face*. Este detecta automáticamente el rostro en cada imagen, recorta la región correspondiente y la redimensiona a 160×160 píxeles,

almacenando el resultado en `known_faces/`. Este paso estandariza las entradas, asegurando consistencia en las condiciones de entrenamiento.

3. Generación de embeddings con DeepFace:

Cada imagen procesada se analiza mediante *DeepFace* utilizando el modelo *ArcFace*, el cual genera un vector numérico de características que describe de forma única los rasgos faciales del individuo.

Para cada alumno, el sistema genera un archivo JSON individual con su número de cédula como nombre (por ejemplo, `5144692.json`), que contiene todos los embeddings derivados de sus imágenes.

4. Cálculo y actualización de centroides:

Posteriormente, se calcula el *centroide* del alumno, representando el promedio robusto de todos sus embeddings. Este vector resume los rasgos característicos del rostro y facilita comparaciones futuras.

A diferencia de un reentrenamiento completo, el sistema implementa una actualización incremental: cuando se agrega un nuevo alumno o se añaden más imágenes a un registro existente, el sistema no recalcula todos los centroides, sino que actualiza únicamente el correspondiente al usuario modificado. De este modo, se optimiza el rendimiento y se conserva el historial de entrenamiento.

5. Almacenamiento estructurado:

Los embeddings individuales se guardan en el directorio `embeddings/`, mientras que el archivo `centroids.json` mantiene un índice general con los centroides y estadísticas de todos los usuarios. Esta estructura facilita la expansión progresiva del sistema sin afectar los registros previos.

4.7. Diseño de la aplicación

4.7.1. Arquitectura del sistema

El sistema fue diseñado bajo el enfoque cliente-servidor, utilizando una arquitectura basada en *REST API* para permitir la comunicación fluida entre el *Frontend* (cliente) y el *Backend* (servidor). Esta arquitectura facilita la escalabilidad, el mantenimiento y la integración con futuras funcionalidades del sistema de reconocimiento facial.

El sistema se compone de tres módulos principales:

- **Frontend:** desarrollado en *React.js*, se encarga de la interfaz gráfica y la interacción directa con el usuario. Permite registrar nuevos estudiantes, gestionar materias, asignarlas a los alumnos, capturar o subir fotografías faciales, consultar los registros de asistencia y cargar justificativos en caso de inasistencia. Su diseño adaptable y responsivo garantiza una experiencia de uso fluida tanto en equipos de escritorio como en dispositivos móviles.
- **Backend:** implementado en *Flask (Python)*, maneja la lógica del sistema, las rutas de la API y la comunicación con la base de datos. Integra los módulos de detección facial con *YOLO-Face* y el reconocimiento mediante *DeepFace (modelo ArcFace)*, además de gestionar el almacenamiento de embeddings, centroides y registros de asistencia.
- **Base de datos:** construida en *PostgreSQL*, almacena de forma estructurada la información de usuarios, cursos, asistencias y datos biométricos. Se eligió por su robustez, soporte de transacciones concurrentes y compatibilidad con entornos productivos en la nube.

4.7.2. Integración de IA y Reconocimiento Facial

El sistema combina técnicas avanzadas de detección y reconocimiento facial mediante redes neuronales convolucionales. Para la primera etapa, se emplea un modelo especializado en la identificación y localización de rostros en tiempo real. Una vez detectada la región de interés, esta se recorta automáticamente y se normaliza a un tamaño estándar de 160×160 píxeles, garantizando uniformidad en las condiciones de entrada.

En la siguiente fase, el rostro procesado es transformado en una representación numérica o vector de características, la cual permite distinguir de manera precisa a cada individuo. Estos vectores se almacenan y se utilizan posteriormente para la verificación e identificación de los usuarios registrados.

El flujo general de procesamiento es el siguiente:

1. Captura o carga de imagen facial desde la interfaz.
2. Detección automática de rostros mediante YOLO-Face.
3. Recorte y normalización del rostro detectado.
4. Generación de embeddings con DeepFace (modelo ArcFace).
5. Registro automático de la asistencia según el resultado del proceso.

El proceso completo se ejecuta desde el backend, implementado en *Flask (Python)*, que coordina la detección, el reconocimiento y el registro de resultados en la base de datos *PostgreSQL*. Esta integración permite un flujo automatizado, eficiente y en tiempo real, garantizando precisión en la identificación de los estudiantes durante la toma de asistencia.

4.7.3. Relevamiento de requisitos

El relevamiento de requisitos se llevó a cabo mediante la aplicación de encuestas dirigidas a profesores y estudiantes de la Facultad Politécnica de la Universidad Nacional del Este (FPU-NE), con el objetivo de identificar las principales necesidades y expectativas respecto al control de asistencia en el entorno académico.

Los resultados obtenidos permitieron comprender las mejoras asociadas a los métodos tradicionales de control de asistencia tales como pérdida de tiempo, errores humanos y registros inconsistentes, y sirvieron de base para definir los requisitos funcionales y no funcionales del sistema.

Requisitos funcionales (RF)

Los requisitos funcionales representan las acciones que el sistema debe realizar para cumplir su propósito. A partir del análisis de las encuestas y la definición de objetivos, se identificaron los siguientes requisitos funcionales, resumidos en la Tabla 4.6.

Tabla 4.6: Requisitos funcionales del sistema Fuente: [Elaboración Propia].

Código	Requisito funcional
RF1	El sistema debe permitir el registro de nuevos estudiantes y docentes mediante una interfaz web.
RF2	El sistema debe permitir la carga de imágenes faciales de los estudiantes para el entrenamiento del modelo.
RF3	El sistema debe detectar automáticamente los rostros presentes en video.
RF4	El sistema debe generar embeddings faciales y almacenarlos junto con la información del usuario.
RF5	El sistema debe registrar automáticamente la asistencia al reconocer un rostro en tiempo real.
RF6	El sistema debe permitir consultar y justificar asistencias desde la interfaz web.

Requisitos no funcionales (RNF)

Los requisitos no funcionales establecen las condiciones de calidad, rendimiento y usabilidad que el sistema debe cumplir para garantizar un funcionamiento eficiente y confiable. La Tabla 4.7 presenta los principales requisitos no funcionales definidos.

Tabla 4.7: Requisitos no funcionales del sistema Fuente: [Elaboración Propia].

Código	Requisito no funcional
RNF1	El sistema debe procesar la detección y reconocimiento facial en menos de 3 segundos por usuario.
RNF2	El sistema debe mantener una precisión mínima del 80 % en la identificación de rostros.
RNF3	La interfaz debe ser intuitiva, responsiva y accesible desde dispositivos móviles y computadoras.
RNF4	La base de datos debe garantizar integridad, seguridad y respaldo de los registros.

4.7.4. Caso de Uso Principal del Sistema

En la Figura 4.4 se representa el caso de uso principal del sistema, en el cual interactúan dos actores principales: el docente y el alumno.

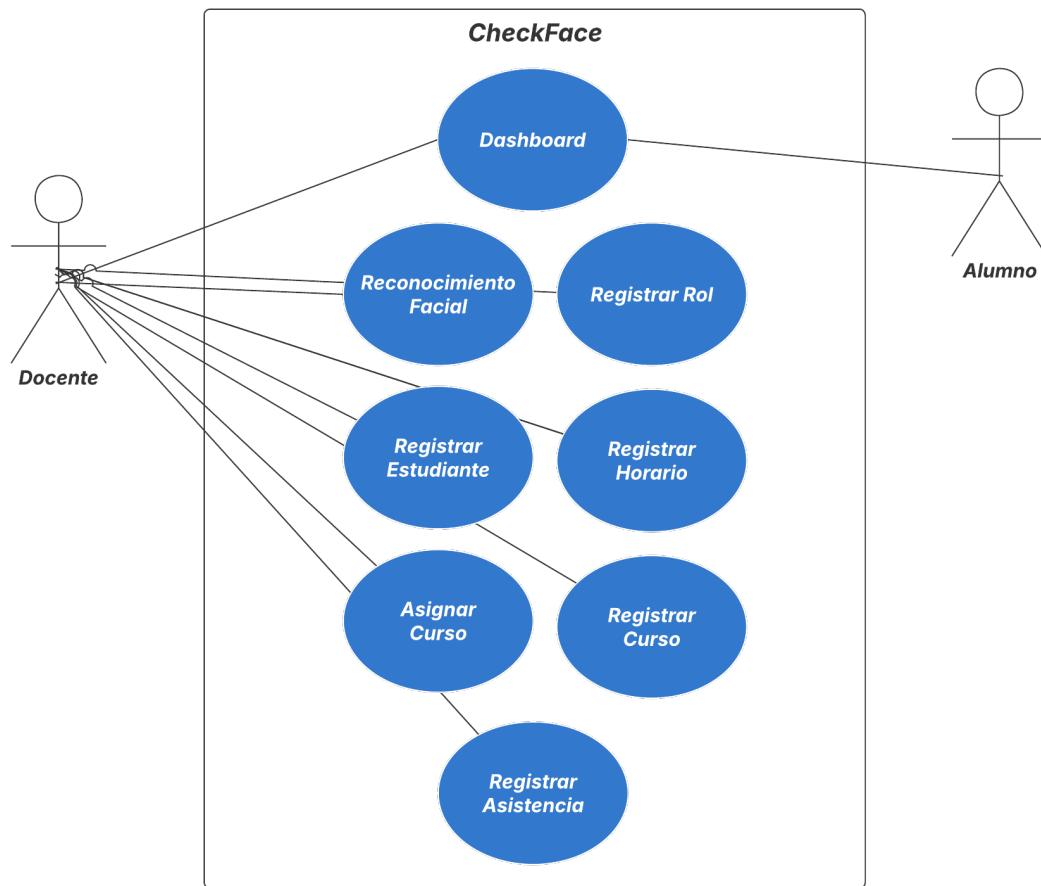


Figura 4.4: Caso de uso principal del sistema Fuente: [Elaboración Propia].

Descripción general del caso de uso

El sistema distingue dos tipos de usuarios con diferentes niveles de acceso: el Docente y el Estudiante.

Docente: tiene acceso completo a todas las funcionalidades del sistema. Puede registrar estudiantes, cursos, horarios y roles; asignar materias; registrar asistencias mediante el reconocimiento facial; y acceder al panel principal (*Dashboard*) con los reportes y estadísticas. Además, puede administrar los datos del sistema y supervisar su funcionamiento general, actuando como un administrador.

Estudiante: dispone de un acceso limitado al sistema, pu-

diendo ingresar únicamente al *Dashboard* para visualizar sus registros de asistencia y verificar su historial personal. No posee permisos de edición ni administración de datos.

4.7.5. Definición de la Base de Datos y Tablas

Para la gestión eficiente de la información del sistema, se implementó una base de datos relacional compuesta por varias tablas que permiten almacenar, vincular y administrar los datos de usuarios, participantes, cursos, horarios y registros de asistencia. La estructura fue diseñada para garantizar integridad referencial, seguridad y escalabilidad, facilitando futuras ampliaciones y adaptaciones del sistema.

Modelo de entidad-relación

La base de datos se compone de siete entidades principales: *roles*, *users*, *participants*, *courses*, *course_schedules*, *participant_courses* y *attendances*. Estas entidades reflejan la estructura funcional del sistema, permitiendo asociar estudiantes a cursos, registrar asistencias y controlar los horarios académicos. En la Figura 4.5 se muestra el diagrama entidad-relación correspondiente.

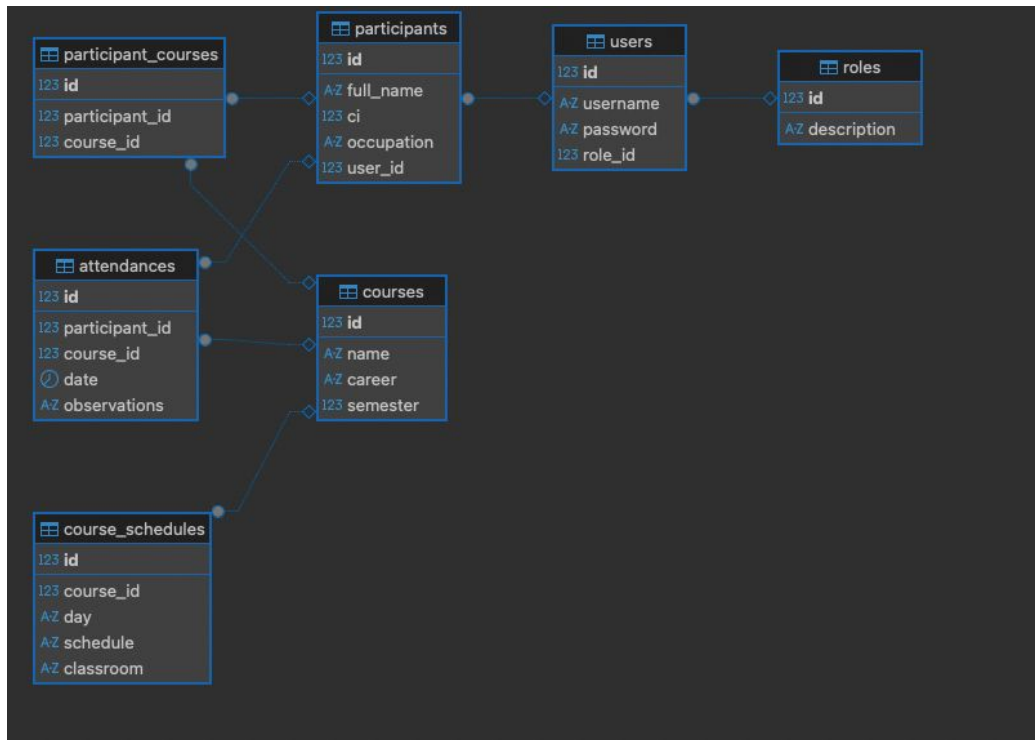


Figura 4.5: Diagrama de Entidad-Relación del sistema Fuente: [Elaboración Propia].

Diccionario de datos

Con el fin de documentar detalladamente la estructura de cada tabla, se elaboró un diccionario de datos que especifica los atributos, tipos de datos y relaciones existentes. A continuación, se describen las tablas principales que conforman la base de datos.

La Tabla 4.8 presenta el diccionario de datos correspondiente a la tabla roles de la base de datos. En ella se describen los atributos que conforman esta entidad, el tipo de dato utilizado para su almacenamiento y la función que cumple cada campo dentro del sistema.

Tabla 4.8: Diccionario de datos de la tabla *roles* Fuente: [Elaboración Propia].

Atributo	Tipo de Dato	Descripción
id	SERIAL (PK)	Identificador único del rol.
description	VARCHAR(120)	Nombre o descripción del rol asignado al usuario.

La Tabla 4.9 detalla la estructura de la tabla *users* utilizada en la base de datos. En ella se especifican los atributos que identifican a cada usuario del sistema, así como sus tipos de datos y la función que cumple cada campo dentro del modelo relacional.

Tabla 4.9: Diccionario de datos de la tabla *users* Fuente: [Elaboración Propia].

Atributo	Tipo de Dato	Descripción
id	SERIAL (PK)	Identificador único del usuario.
username	VARCHAR(120)	Nombre de usuario utilizado para iniciar sesión.
password	TEXT	Contraseña encriptada del usuario.
role_id	INT (FK)	Referencia al rol del usuario en la tabla <i>roles</i> .

La Tabla 4.10 presenta el diccionario de datos de la tabla *participants*, donde se detallan los atributos utilizados para registrar la información principal de cada estudiante o participante dentro del sistema. Se incluyen los tipos de datos empleados y la función específica que cumple cada campo, así como su relación con otras tablas del modelo.

Tabla 4.10: Diccionario de datos de la tabla *participants* Fuente: [Elaboración Propia].

Atributo	Tipo de Dato	Descripción
id	SERIAL (PK)	Identificador único del participante.
full_name	VARCHAR(120)	Nombre completo del estudiante o participante.
ci	INT	Número de cédula de identidad del participante.
occupation	VARCHAR(50)	Rol académico del participante (por ejemplo, alumno).
user_id	INT (FK)	Relación con el usuario correspondiente en la tabla <i>users</i> .

La Tabla 4.11 presenta el diccionario de datos de la tabla *courses*, especificando los campos utilizados para identificar cada curso, su denominación, la carrera a la que pertenece y el semestre correspondiente.

Tabla 4.11: Diccionario de datos de la tabla *courses* Fuente: [Elaboración Propia].

Atributo	Tipo de Dato	Descripción
id	SERIAL (PK)	Identificador único del curso.
name	VARCHAR(120)	Nombre de la materia o curso.
career	VARCHAR(120)	Carrera o programa académico al que pertenece el curso.
semester	INT	Semestre académico correspondiente.

La Tabla 4.12 presenta el diccionario de datos de la tabla *course_schedules*, donde se detallan los campos utilizados para registrar los horarios asignados a cada curso. Se especifican los atributos que conforman esta entidad, los tipos de datos empleados y la función de cada campo dentro del sistema académico.

Tabla 4.12: Diccionario de datos de la tabla *course_schedules* Fuente: [Elaboración Propia].

Atributo	Tipo de Dato	Descripción
id	SERIAL (PK)	Identificador único del horario.
course_id	INT (FK)	Referencia al curso correspondiente.
day	VARCHAR(20)	Día de la semana en que se dicta el curso.
schedule	VARCHAR(50)	Horario asignado al curso.
classroom	VARCHAR(50)	Aula o sala asignada.

La Tabla 4.13 muestra el diccionario de datos de la tabla *participant_courses*, encargada de representar la relación entre los participantes y los cursos en los que están matriculados. En ella se especifican los atributos que conforman esta entidad, los tipos de datos utilizados y las referencias a las tablas vinculadas.

Tabla 4.13: Diccionario de datos de la tabla *participant_courses* Fuente: [Elaboración Propia].

Atributo	Tipo de Dato	Descripción
id	SERIAL (PK)	Identificador único de la relación.
participant_id	INT (FK)	Referencia al participante matriculado.
course_id	INT (FK)	Referencia al curso correspondiente.

La Tabla 4.14 presenta el diccionario de datos de la tabla *attendances*, utilizada para almacenar los registros de asistencia generados por el sistema. En ella se describen los atributos que conforman esta entidad, los tipos de datos empleados y las relaciones que mantiene con otras tablas del modelo académico.

Tabla 4.14: Diccionario de datos de la tabla *attendances* Fuente: [Elaboración Propia].

Atributo	Tipo de Dato	Descripción
id	SERIAL (PK)	Identificador único del registro de asistencia.
participant_id	INT (FK)	Referencia al participante que asiste al curso.
course_id	INT (FK)	Referencia al curso al que pertenece la asistencia.
date	DATE	Fecha en la que se registra la asistencia.
observations	TEXT	Observaciones adicionales o justificación de inasistencia.

4.7.6. Diagrama de flujo del funcionamiento del sistema

La Figura 4.6 muestra el diagrama de flujo que describe el funcionamiento general del sistema de asistencia. En él se ilustran las etapas principales del proceso automatizado, desde la carga de imágenes de los estudiantes hasta el registro final en la base de datos.

El proceso inicia con la carga de las imágenes faciales a través de la interfaz, las cuales son procesadas para generar representaciones numéricas que permiten su posterior identificación. Luego, el sistema activa la cámara y ejecuta la detección y verificación en tiempo real. Cuando un rostro coincide con un registro existente, el estudiante es marcado como presente; de lo contrario, se clasifica como desconocido, garantizando un control más preciso del registro de asistencia.

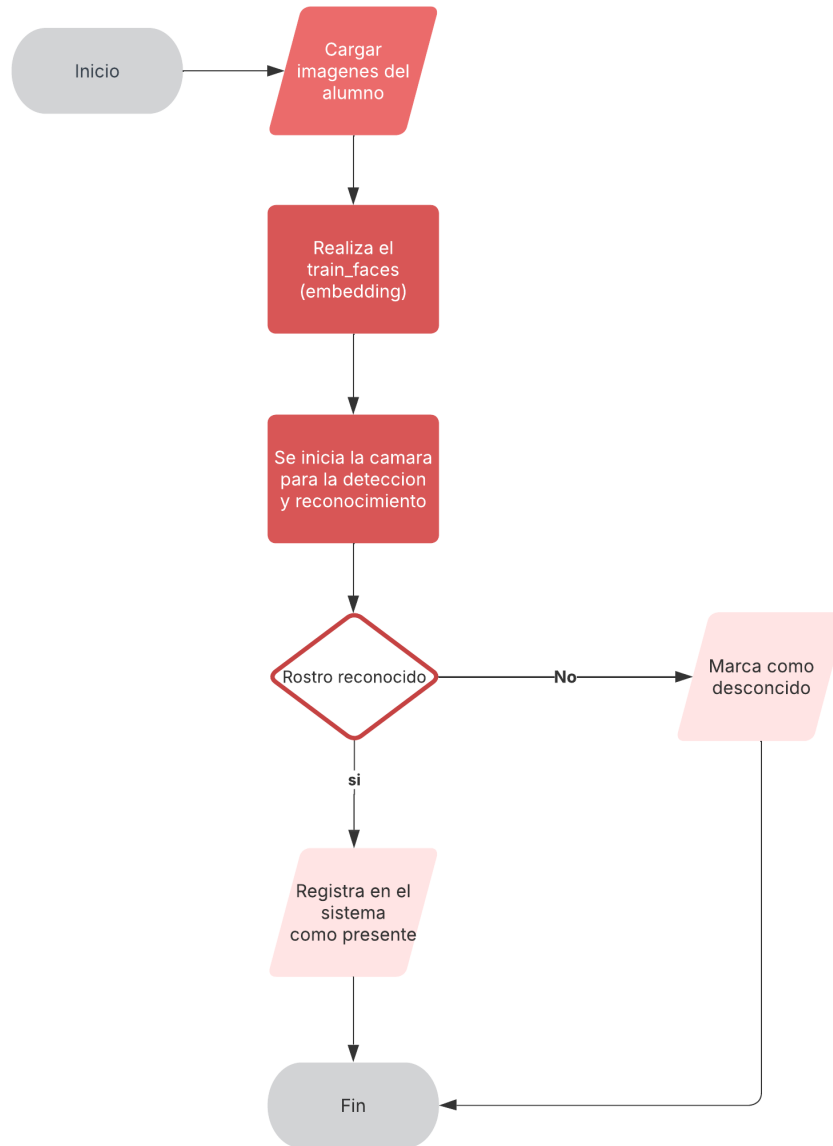


Figura 4.6: Diagrama de flujo del funcionamiento del sistema.[Fuente Elaboración Propia].

4.7.7. Desarrollo del backend

El backend del sistema fue desarrollado con un enfoque modular y ligero, orientado a gestionar la comunicación entre la inter-

faz web, la base de datos y los procesos de reconocimiento facial en tiempo real. Se implementó utilizando el framework Flask, basado en el lenguaje Python, debido a su simplicidad, flexibilidad y compatibilidad con bibliotecas de visión por computadora como OpenCV, TensorFlow y DeepFace.

La estructura del backend se organiza en carpetas según la funcionalidad de cada componente. En la raíz se encuentran los archivos principales del sistema, como `main.py`, encargado de inicializar la aplicación Flask, configurar las rutas y ejecutar el servidor, y `api.py`, que centraliza los endpoints de la API. El directorio `routes` contiene los módulos de rutas organizados mediante *Blueprints*, lo que permite mantener una arquitectura limpia, separando las responsabilidades de cada parte del sistema, como la gestión de usuarios, cursos, roles y asistencias. El módulo `database` se encarga de la conexión con la base de datos PostgreSQL, gestionando las operaciones de lectura y escritura, así como las relaciones entre entidades.

El núcleo del reconocimiento facial se encuentra en la carpeta `recognition`, que contiene los scripts `face_recognizer.py` y `helpers.py`, junto con subcarpetas destinadas a los conjuntos de datos faciales. Dentro de esta estructura se incluyen las carpetas `raw_faces` (imágenes originales capturadas), `known_faces` (rostros previamente registrados) y `embeddings` (vectores de características generados para cada usuario). En este módulo se realiza el procesamiento de imágenes, aplicando el modelo de detección YOLOFace para localizar los rostros y el modelo ArcFace, a través de la biblioteca DeepFace, para generar y comparar los vectores de características de cada participante.

El proceso de entrenamiento y actualización de los vectores faciales se realiza mediante los scripts `train_faces.py` y `train_faces_old.py`, los cuales permiten recalcular los vectores cuando se incorporan nuevos usuarios o se actualizan los datos existentes. Este mecanismo garantiza que el sistema mantenga

la capacidad de reconocer a los estudiantes incluso cuando se amplía la base de datos.

Las principales funcionalidades implementadas en esta capa incluyen:

- Exposición de endpoints RESTful: se definieron rutas HTTP para el manejo de usuarios, roles, cursos, horarios, asistencias y reconocimiento facial, enviando y recibiendo datos en formato JSON. Estas rutas facilitan la comunicación entre el frontend y el backend mediante solicitudes asincrónicas.
- Procesamiento y reconocimiento facial: las imágenes capturadas desde el frontend son enviadas al backend, donde se ejecutan las etapas de detección, normalización y reconocimiento facial. Si la similitud entre el rostro detectado y los vectores registrados supera el umbral definido, se registra automáticamente la asistencia correspondiente en la base de datos.
- Gestión de usuarios y autenticación: se implementaron rutas dedicadas para el registro e inicio de sesión, controlando el acceso a las funciones administrativas del sistema mediante validaciones en el servidor.
- Interacción con la base de datos: toda la información sobre usuarios, estudiantes, cursos, horarios y asistencias se gestiona en la base de datos PostgreSQL mediante operaciones CRUD estructuradas y validadas desde la API.

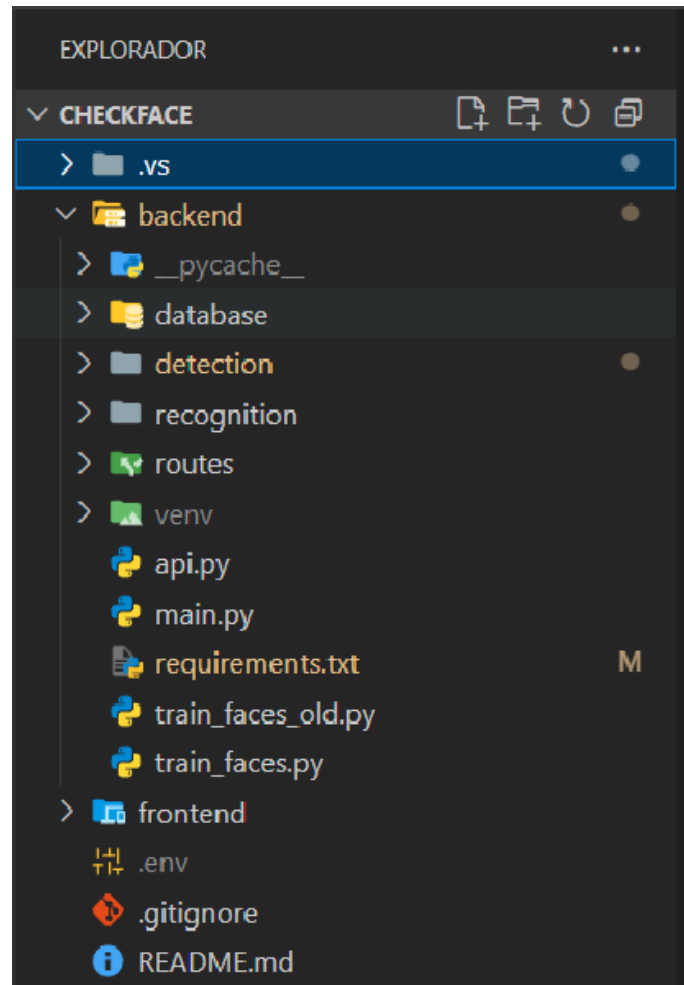


Figura 4.7: Estructura general del trabajo en Visual Studio Code Fuente: [Elaboración Propia].

4.7.8. Desarrollo del frontend

El frontend del sistema fue desarrollado con un enfoque modular basado en componentes reutilizables, lo que permitió construir una interfaz dinámica, responsiva y visualmente coherente. El diseño prioriza la claridad y la facilidad de uso, garantizando una navegación intuitiva entre las diferentes secciones y funcionalidades.

El desarrollo de la interfaz se organizó en módulos funciona-

les, cada uno representando una parte específica del sistema. La estructura general se compone de:

- **Componentes de presentación:** responsables de mostrar la información y recibir datos de entrada mediante formularios, tablas y botones.
- **Ruteo de navegación:** implementado con `react-router-dom`, para permitir la transición fluida entre las distintas vistas del sistema.
- **Servicios:** encargados de establecer la comunicación con la REST API del backend, gestionando peticiones HTTP de forma centralizada.
- **Gestión de estado:** realizada mediante los hooks de React (`useState`, `useEffect`), asegurando la actualización dinámica de la información visualizada por el usuario.

Criterios de diseño de la interfaz.

- Consistencia visual en todos los módulos mediante el uso de componentes y clases de Bootstrap.
- Diseño *responsive*, garantizando un correcto funcionamiento en diferentes resoluciones y dispositivos.
- Retroalimentación visual inmediata ante las acciones del usuario (por ejemplo, mediante alertas o indicadores de carga).
- Accesibilidad básica mediante etiquetas y descripciones que facilitan la interacción.

Flujo de uso.

- **Inicio de sesión:** los usuarios acceden al sistema ingresando sus credenciales personales a través de una interfaz simple y clara, garantizando que únicamente el personal autorizado pueda utilizar las funcionalidades.
- **Registro de nuevos usuarios:** se dispone de un formulario que permite crear nuevas cuentas, asignar roles y definir permisos.
- **Uso de los módulos principales:** dentro del panel lateral de navegación se encuentran las opciones Dashboard, Reconocimiento Facial, Registrar Estudiante, Registrar Rol, Registrar Curso, Registrar Horario, Registrar Asistencia y Asignar Curso.

4.7.9. Líneas de código

En el Anexo B se presentan los fragmentos de código más relevantes del sistema, junto con una descripción que explica su función y la lógica que implementan dentro del proceso de reconocimiento facial. Los ejemplos incluyen tanto los módulos encargados del entrenamiento de los rostros registrados como aquellos utilizados en la detección y el reconocimiento en tiempo real.

Asimismo, se detalla la forma en que cada componente contribuye al flujo general del sistema, desde la lectura y preparación de imágenes hasta la generación de embeddings, la comparación con la base de datos y la identificación de los estudiantes en el aula. De esta manera, el anexo brinda una visión clara de la implementación técnica y facilita la comprensión del funcionamiento interno del sistema.

4.8. Métricas de Evaluación

La Tabla 4.15 presenta las métricas consideradas durante el desarrollo y la evaluación del sistema. Estas métricas se organizan por categoría e incluyen tanto indicadores relacionados con el conjunto de datos como parámetros de entrenamiento, desempeño del reconocimiento facial, uso de recursos y validación en condiciones reales.

Tabla 4.15: Métricas consideradas en el desarrollo y evaluación del sistema
Fuente: [Elaboración Propia].

Categoría	Métrica	Descripción breve	Criterio de éxito
Datos del conjunto	Nº total de imágenes	Imágenes usadas en entrenamiento y validación.	≥ 60 imágenes recopiladas.
	Diversidad de condiciones visuales	Variación de luz, ángulo y expresión.	≥ 3 variaciones (luz, ángulo y expresión).
	Nº de participantes registrados	Estudiantes con rostro registrado en el sistema.	≥ 10 estudiantes registrados.
Entrenamiento	Precisión	Porcentaje de aciertos durante el entrenamiento.	$\geq 80\%$ de precisión.
	Pérdida	Error medio por usuario registrado.	Tendencia decreciente durante el entrenamiento.
Reconocimiento facial	Tasa de reconocimiento correcto	Porcentaje de aciertos en pruebas.	$\geq 80\%$ de aciertos.
	Tasa de falsos positivos/negativos	Porcentaje de detecciones erróneas u omitidas.	$\leq 20\%$ (valor máximo tolerable).
	Tiempo promedio de proceso (ms)	Tiempo por rostro para detectar y verificar.	≤ 250 ms por rostro.
Rendimiento global	Uso promedio de CPU/GPU	Utilización media de recursos durante la ejecución.	$\leq 70\%$ de uso de CPU (sin GPU).
Validación práctica	Tasa de detección real	Porcentaje de aciertos en condiciones reales de aula.	$\geq 75\%$ de aciertos.

4.8.1. Usabilidad

Para evaluar la usabilidad y el nivel de satisfacción de los usuarios, se definieron variables operacionales basadas en los principios de experiencia de usuario, considerando la facilidad de uso, la eficiencia, la confiabilidad y la aceptación general de la herramienta desarrollada.

Como método de medición se utilizó el cuestionario System Usability Scale (SUS), ampliamente reconocido por su efectividad para obtener un puntaje global de usabilidad. La evaluación se realizó con un grupo de usuarios que interactuó con las principales funcionalidades, incluyendo el registro de cuentas, la toma de asistencia y la consulta de reportes en el entorno web.

Los resultados permitieron determinar el nivel de aceptación general y detectar oportunidades de mejora relacionadas con la interfaz y la navegación. Los datos consolidados y las respuestas individuales se presentan en el Anexo A.

Capítulo 5

Resultados

Para garantizar resultados precisos durante la evaluación del sistema de asistencia basado en reconocimiento facial, se estableció un entorno de pruebas controlado que reprodujo las condiciones reales de funcionamiento dentro de un contexto académico. El objetivo fue analizar el desempeño del sistema, verificando su capacidad para identificar correctamente a los usuarios, registrar la asistencia y mantener la coherencia de los datos obtenidos. Las pruebas se realizaron bajo distintas condiciones de iluminación, ángulo y distancia, repitiéndose en varias sesiones para asegurar resultados consistentes. Se analizaron indicadores como el tiempo de procesamiento, el porcentaje de coincidencias correctas y la tasa de falsos positivos, comprobando también la interacción entre los módulos de detección, reconocimiento y registro. Los resultados obtenidos se presentan a continuación mediante tablas y gráficos que reflejan la precisión, estabilidad y confiabilidad del modelo propuesto.

5.1. Interfaz y funcionalidades del sistema

El sistema desarrollado cuenta con una interfaz web moderna e intuitiva, diseñada para facilitar la gestión de usuarios, cursos y asistencias. A continuación, se presentan las principales vistas

y módulos funcionales que conforman la versión final del sistema. Cada componente fue implementado siguiendo criterios de usabilidad, accesibilidad y consistencia visual.

Interfaz de inicio de sesión. La interfaz de inicio de sesión constituye el punto de acceso principal al sistema y permite a los usuarios autenticarse de manera segura mediante un formulario con campos de nombre de usuario y contraseña. El proceso de validación se realiza a través de una solicitud al servidor, que verifica las credenciales antes de otorgar el acceso. El diseño minimalista y centrado proporciona una experiencia fluida y adaptable a distintos dispositivos.

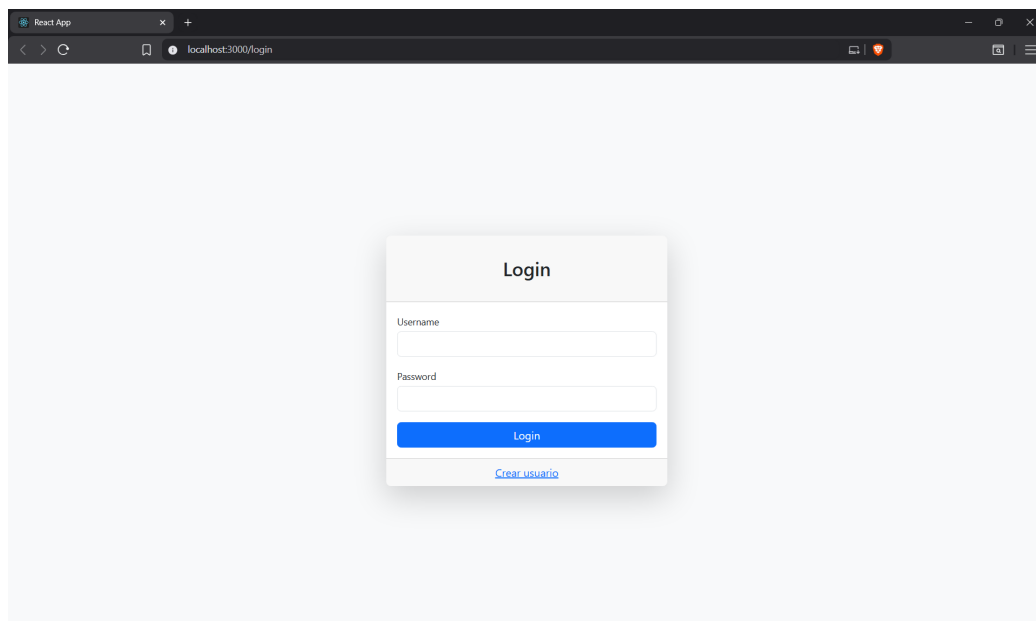


Figura 5.1: Interfaz de inicio de sesión del sistema Fuente: [Elaboración Propia].

Registro de usuario: El módulo de registro de usuario posibilita la creación de nuevas cuentas a través de un formulario con validaciones básicas, como campos obligatorios y verificación de la longitud mínima de la contraseña. Al confirmar el registro, los datos se envían al servidor para su almacenamiento seguro, y la vista incluye un enlace que permite regresar a la

pantalla de inicio de sesión.

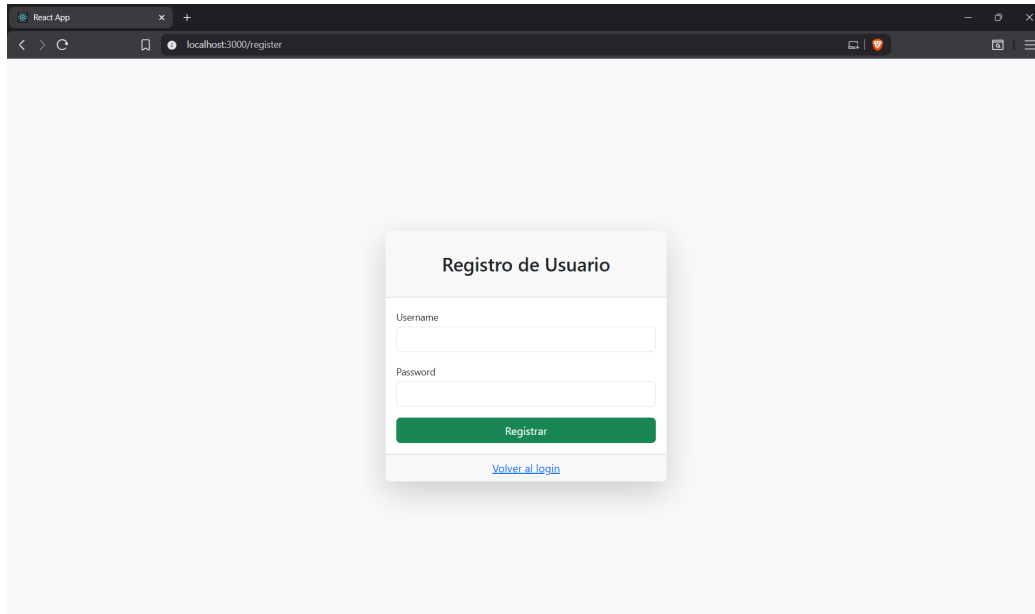
A screenshot of a web browser window displaying a user registration form. The browser's address bar shows 'localhost:3000/register'. The form is titled 'Registro de Usuario' and contains two input fields labeled 'Username' and 'Password'. Below these fields is a green button labeled 'Registrar' and a blue link labeled 'Volver al login'.

Figura 5.2: Formulario de registro de usuario Fuente: [Elaboración Propia].

Dashboard: El *dashboard* constituye la primera vista disponible tras el inicio de sesión y proporciona una visión general del registro de asistencias mediante gráficos y listados interactivos. Este módulo centraliza la información y permite analizar la asistencia diaria aplicando filtros por curso y rango de fechas.

La Figura 5.3 muestra el gráfico de barras que representa la cantidad de asistencias registradas por día, mientras que los filtros ubicados en la parte superior delimitan el período de consulta y los registros visualizados. Los datos presentados son demostrativos y se incluyen únicamente con fines ilustrativos.

Resultados

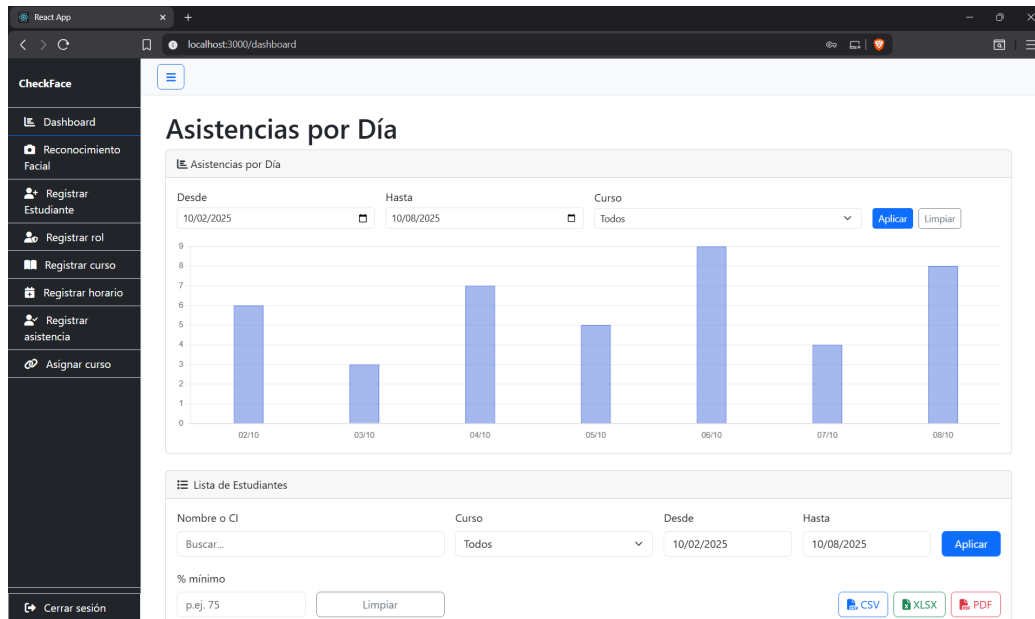
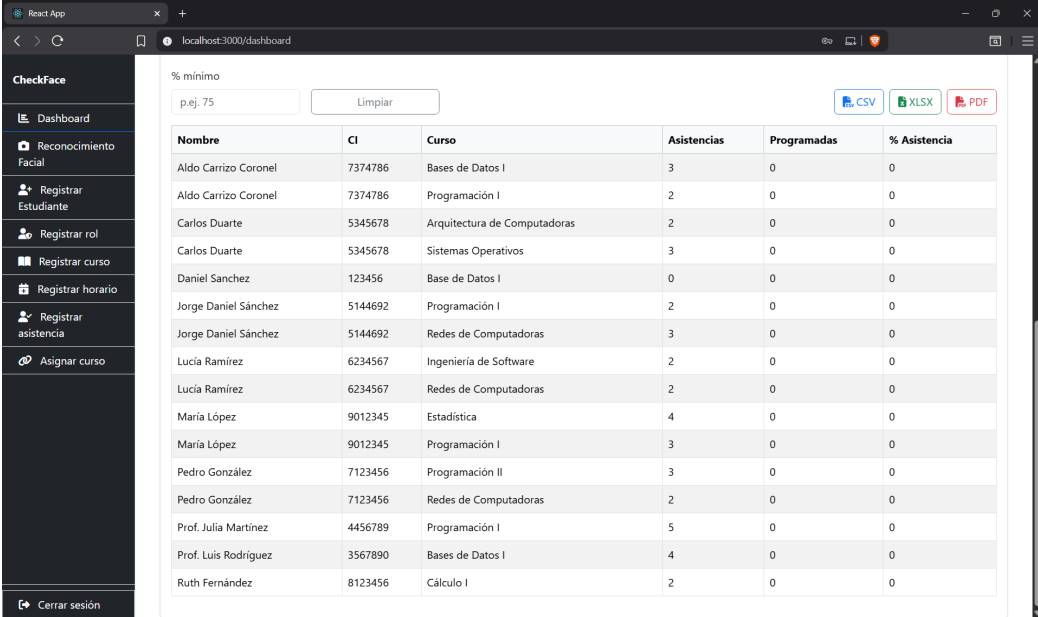


Figura 5.3: Gráfico de asistencias por día en el módulo Dashboard Fuente: [Elaboración Propia].

En la Figura 5.4 se observa la sección del panel donde se listan los estudiantes con sus asistencias registradas, el curso correspondiente y el porcentaje de participación. La tabla incorpora opciones para exportar los datos en distintos formatos (CSV, XLSX y PDF), facilitando la generación de reportes y su posterior análisis. Al igual que en la vista anterior, los datos visualizados son de carácter demostrativo y no representan información real.

Resultados



Nombre	CI	Curso	Asistencias	Programadas	% Asistencia
Aldo Carrizo Coronel	7374786	Bases de Datos I	3	0	0
Aldo Carrizo Coronel	7374786	Programación I	2	0	0
Carlos Duarte	5345678	Arquitectura de Computadoras	2	0	0
Carlos Duarte	5345678	Sistemas Operativos	3	0	0
Daniel Sanchez	123456	Base de Datos I	0	0	0
Jorge Daniel Sánchez	5144692	Programación I	2	0	0
Jorge Daniel Sánchez	5144692	Redes de Computadoras	3	0	0
Lucía Ramírez	6234567	Ingeniería de Software	2	0	0
Lucía Ramírez	6234567	Redes de Computadoras	2	0	0
María López	9012345	Estadística	4	0	0
María López	9012345	Programación I	3	0	0
Pedro González	7123456	Programación II	3	0	0
Pedro González	7123456	Redes de Computadoras	2	0	0
Prof. Julia Martínez	4456789	Programación I	5	0	0
Prof. Luis Rodríguez	3567890	Bases de Datos I	4	0	0
Ruth Fernández	8123456	Cálculo I	2	0	0

Figura 5.4: Listado de estudiantes y asistencias registradas en el módulo Dashboard Fuente: [Elaboración Propia].

Reconocimiento facial: Este módulo ejecuta el proceso automático de detección y reconocimiento de rostros a través de la cámara del dispositivo. Al activarlo, el sistema analiza la imagen capturada en tiempo real, identifica el rostro y compara sus características con los registros existentes en la base de datos. Si la persona es reconocida correctamente, se muestra una confirmación visual y se registra la asistencia correspondiente al curso asignado para el día. En caso contrario, el sistema notifica que el rostro no fue identificado.

Esta funcionalidad permite automatizar el control de asistencias, eliminando el ingreso manual de datos y reduciendo el margen de error. Todo el flujo de reconocimiento se ejecuta localmente y en tiempo real, ofreciendo una experiencia precisa y eficiente.



Figura 5.5: Módulo de reconocimiento facial automático Fuente: [Elaboración Propia].

Registrar rol: El módulo de gestión de roles permite crear nuevos perfiles del sistema y administrar los existentes. Incluye un formulario para ingresar la descripción del rol y una tabla con opciones de edición y eliminación, asegurando la integridad de los datos mediante confirmaciones previas a cada operación.

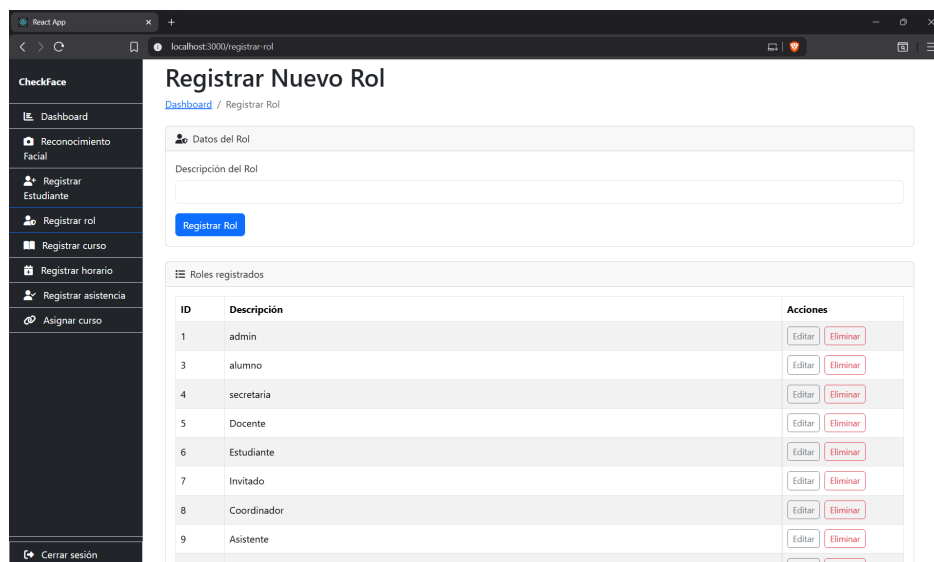


Figura 5.6: Gestión de roles: registro, edición y eliminación Fuente: [Elaboración Propia].

Registrar curso: El módulo de cursos permite registrar nuevas asignaturas especificando su nombre, carrera y semestre. Incluye además un listado con acciones de edición y eliminación, facilitando la administración académica de las clases.

The screenshot shows a web application interface for registering new courses. On the left is a dark sidebar with a menu containing options like 'Dashboard', 'Reconocimiento Facial', 'Registrar Estudiante', 'Registrar rol', 'Registrar curso' (highlighted), 'Registrar horario', 'Registrar asistencia', 'Asignar curso', and 'Cerrar sesión'. The main content area is titled 'Registrar Nuevo Curso' and includes a breadcrumb 'Dashboard / Registrar Curso'. Below the title is a form section 'Datos del Curso' with fields for 'Nombre del Curso' (a text input), 'Carrera' (a dropdown menu showing '-- Seleccionar carrera --'), and 'Semestre' (a dropdown menu showing '-- Seleccionar semestre --'). A blue 'Registrar Curso' button is at the bottom of the form. Below the form is a section titled 'Cursos registrados' containing a table with three rows of data. Each row has buttons for 'Editar' and 'Eliminar'.

ID	Nombre	Carrera	Semestre	Acciones
1	Base de Datos I	Análisis de Sistemas	1	<button>Editar</button> <button>Eliminar</button>
2	Programación I	Lic. en Análisis de Sistemas	1	<button>Editar</button> <button>Eliminar</button>
3	Programación II	Lic. en Análisis de Sistemas	2	<button>Editar</button> <button>Eliminar</button>

Figura 5.7: Registro y gestión de cursos Fuente: [Elaboración Propia].

Registrar horario: Este módulo permite asignar horarios a los cursos registrados. El formulario contempla los campos curso, día, hora de inicio, hora de fin y aula, garantizando una adecuada organización de las clases. La sección inferior presenta una tabla de horarios filtrable por curso, día o aula.

Resultados

React App

localhost:3000/registrar-horario

CheckFace

- Dashboard
- Reconocimiento Facial
- Registrar Estudiante
- Registrar rol
- Registrar curso
- Registrar horario
- Registrar asistencia
- Asignar curso

Registrar Horario para un Curso

[Dashboard](#) / [Registrar Horario](#)

Datos del Horario

Curso: -- Seleccionar curso --

Día: -- Seleccionar día --

Hora de Inicio: Seleccionar hora Hora de Fin: Seleccionar hora

Aula: -- Seleccionar aula --

Registrar Horario

Horarios

Curso: Todos Día: Todos Aula: Todas Desde: HH:mm Hasta: HH:mm Limpiar

Curso	Día	Horario	Aula	Acciones
Programación I	Lunes	07:00-08:30	A-101	Editar Eliminar
Programación I	Miércoles	07:00-08:30	A-101	Editar Eliminar

Cerrar sesión

Figura 5.8: Registro y gestión de horarios por cursoFuente: [Elaboración Propia].

Registrar asistencia: Este módulo permite ingresar asistencias de forma manual en casos excepcionales, como cuando un estudiante no fue reconocido o requiere justificar una ausencia. El formulario incluye los campos estudiante, curso, fecha y observaciones, garantizando la completitud y precisión de los registros.

Resultados

The screenshot shows a web application running in a browser at localhost:3000. The interface has a dark sidebar on the left with the title 'CheckFace' and a list of menu items: Dashboard, Reconocimiento Facial, Registrar Estudiante, Registrar rol, Registrar curso, Registrar horario, Registrar asistencia (highlighted), and Asignar curso. At the bottom of the sidebar is a 'Cerrar sesión' button. The main content area is titled 'Registrar Asistencia Manual' and includes a breadcrumb 'Dashboard / Asistencia Manual'. Below this is a form titled 'Ingreso Manual de Asistencia'. The form contains a dropdown for 'Estudiante' with the placeholder 'Buscar por nombre o CL...', a dropdown for 'Curso' with the placeholder '-- Seleccionar curso --', a date field for 'Fecha' showing '2025-10-08 20:37', and a text area for 'Observación' with the placeholder 'Opcional: Ej. Llegó tarde'. A blue 'Registrar Asistencia' button is at the bottom of the form.

Figura 5.9: Módulo de registro manual de asistencias Fuente: [Elaboración Propia].

Asignar curso: El módulo de asignación de cursos vincula a los participantes con las materias disponibles en el sistema. El formulario permite seleccionar el estudiante y el curso desde listas desplegables, asegurando coherencia en los registros académicos. De esta manera, se facilita la organización de los grupos de clase y la administración de las asistencias.

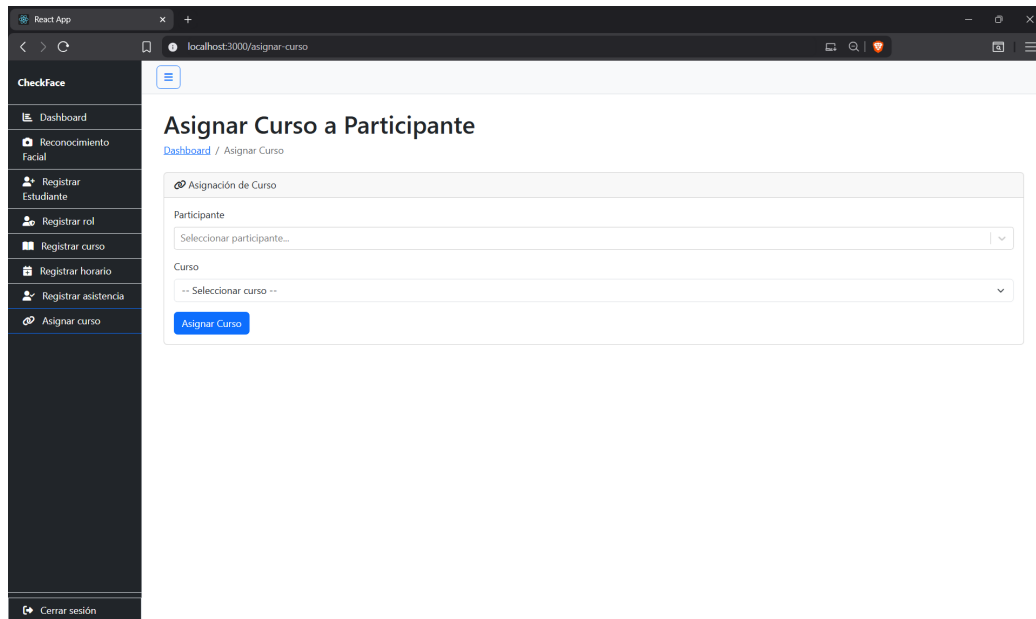


Figura 5.10: Asignación de cursos a participantes Fuente: [Elaboración Propia].

5.2. Entorno de Evaluación y Resultados Iniciales

La siguiente tabla presenta un resumen de los componentes principales que conforman el entorno de desarrollo y ejecución del sistema. Se incluyen tanto los elementos de software utilizados para la implementación del trabajo como las herramientas necesarias para su funcionamiento, detallando las versiones, librerías y módulos activos durante el proceso de evaluación.

Tabla 5.1: Componentes de hardware y software utilizados en el sistema Fuente: [Elaboración Propia].

Elemento	Descripción
Modelo de cámara	Cámara web HD 720p
Servidor	AMD Ryzen 7 7735HS, 16 GB RAM, SSD 512 GB, Windows 11 Home
Lenguaje de programación	Python 3.12.11
Framework backend	Flask
Base de datos	PostgreSQL 18
Framework frontend	React, Bootstrap
Librerías	OpenCV, NumPy, Pandas, Ultralytics, DeepFace, TensorFlow-CPU==2.13.0, SQLAlchemy, Flask-CORS, Flask-JWT-Extended, Omegaconf, Matplotlib, Pillow
Versión del sistema desarrollado	<i>v1.0 Beta</i>
Módulos activados	Captura de imagen, Preprocesamiento, Detección facial, Reconocimiento, Registro en base de datos, Reporte de asistencia

5.2.1. Errores detectados y ajustes aplicados

Durante la etapa de pruebas, se identificaron ciertos errores recurrentes en el funcionamiento del sistema, principalmente relacionados con la precisión del reconocimiento facial. A continuación, se presenta una tabla con los tipos de errores detectados, su causa y las acciones de mitigación implementadas para mejorar el desempeño.

Resultados

Tabla 5.2: Errores frecuentes y acciones de mitigación Fuente: [Elaboración Propia].

Tipo de error	Causa identificada	Ajuste o solución aplicada
Falsos positivos	Umbral de similitud fijo no adecuado a la variación entre usuarios	Umbral ajustado dinámicamente según el número y diversidad de imágenes por usuario
Reconocimiento inexacto con poca luz	Capturas en entornos con baja iluminación o sombras	Aplicación de mejora de contraste y normalización de imagen
Procesamiento lento	Comparación directa con todos los embeddings registrados	Filtrado previo por clase y reducción del número de comparaciones simultáneas

Tabla 5.3: Configuración de umbral centrada en 0.45 y resultado esperado (distancia entre embeddings expresada como valor porcentual). Fuente: [Elaboración Propia].

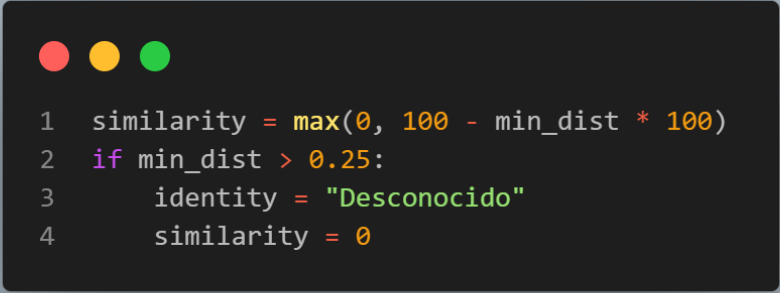
Umbral (<code>min_dist > X</code>)	Resultado esperado	Notas
> 0.25	Reconocimiento muy estricto	Alta precisión, pero puede fallar con cambios leves de luz o expresión.
> 0.35	Recomendado en entornos controlados	Mantiene un equilibrio adecuado entre precisión y estabilidad.
> 0.45	Valor equilibrado	Buen compromiso entre exactitud y tolerancia, utilizado como referencia base del sistema.
> 0.60	Reconocimiento flexible	Acepta pequeñas variaciones de ángulo o iluminación sin perder confiabilidad.
> 0.75	Reconocimiento permisivo	Mayor tolerancia, con posibilidad de falsos positivos en rostros similares.

5.2.2. Configuraciones y resultados obtenidos

En esta sección se presentan los resultados obtenidos durante las pruebas experimentales realizadas al sistema. Cada configuración de umbral corresponde a un valor distinto de similitud utilizado para determinar la coincidencia entre rostros. A través de estas pruebas se analizó el impacto de cada ajuste en la precisión del reconocimiento facial y en la capacidad del sistema para identificar correctamente a los usuarios registrados. A continuación, se detallan los resultados para cada configuración evaluada.

Configuración 1: Umbral 0.25

En esta primera configuración se utilizó un umbral de similitud de 0.25, considerado un valor muy estricto dentro del proceso de reconocimiento facial. Este parámetro establece que solo se considerarán coincidencias cuando la similitud entre vectores de características sea muy alta, reduciendo al mínimo las posibilidades de identificación incorrecta. Sin embargo, al mismo tiempo, puede provocar que rostros válidos no sean reconocidos si presentan ligeras variaciones de iluminación, expresión o ángulo.



```
1 similarity = max(0, 100 - min_dist * 100)
2 if min_dist > 0.25:
3     identity = "Desconocido"
4     similarity = 0
```

Figura 5.11: Fragmento de código utilizado para configurar el umbral de reconocimiento Fuente: [Elaboración Propia].

La Figura 5.2 muestra el resultado obtenido bajo esta configuración. Se observa que el sistema no reconoce a los usuarios registrados, clasificándolos como “Desconocidos”, lo que evidencia que el umbral fijado es demasiado restrictivo para un entorno real. Este comportamiento confirma que, aunque el modelo mantiene un criterio de coincidencia riguroso, su capacidad de reconocimiento efectivo disminuye significativamente al exigir una similitud tan alta entre rostros.

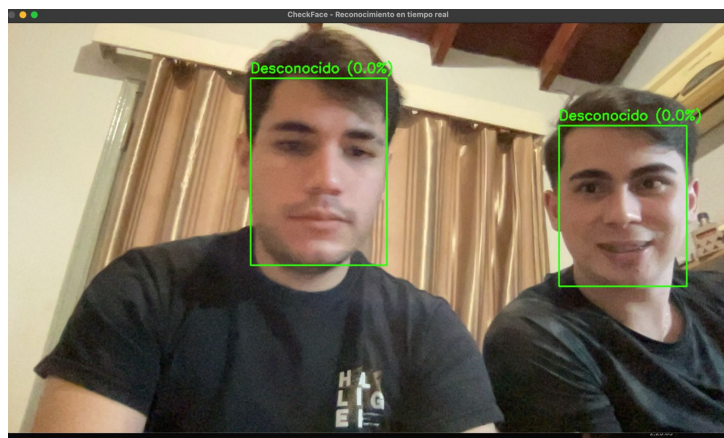


Figura 5.12: Muy estricto, reconoce solo rostros casi idénticos Fuente: [Elaboración Propia].

Configuración 2: Umbral 0.35

En esta configuración se empleó un umbral de similitud de 0.35, con el propósito de evaluar un nivel ligeramente menos restrictivo que el de la configuración anterior. Este ajuste busca permitir un margen de tolerancia mayor ante variaciones menores en los rostros, como cambios en la iluminación, expresión o posición, manteniendo al mismo tiempo un criterio de identificación relativamente estricto. Con ello se pretende analizar si el sistema mejora su capacidad de reconocimiento sin comprometer de manera significativa la precisión de los resultados.

A screenshot of a code editor window with a dark background and light blue window control buttons (red, yellow, green) in the top-left corner. The editor contains four lines of Python code with syntax highlighting: line 1 has 'similarity' in blue, 'max' in green, '0' in orange, '100' in orange, '-' in blue, 'min_dist' in blue, '*' in green, and '100' in orange; line 2 has 'if' in purple, 'min_dist' in blue, '>' in blue, and '0.35' in orange; line 3 has 'identity' in blue, '=' in blue, and '"Desconocido"' in green; line 4 has 'similarity' in blue, '=' in blue, and '0' in orange.

```
1 similarity = max(0, 100 - min_dist * 100)
2 if min_dist > 0.35:
3     identity = "Desconocido"
4     similarity = 0
```

Figura 5.13: Fragmento de código utilizado para configurar el umbral de reconocimiento Fuente: [Elaboración Propia].

La Figura 5.4 muestra el comportamiento del sistema bajo esta configuración. Se observa que el modelo logra un equilibrio más adecuado entre precisión y sensibilidad, permitiendo reconocer correctamente a los usuarios en la mayoría de los casos, aunque sigue siendo susceptible a variaciones de luz o expresión facial. Este resultado demuestra que el valor de 0.35 mejora el desempeño general del sistema respecto a la configuración anterior, reduciendo los casos de no reconocimiento sin comprometer la exactitud de las coincidencias.

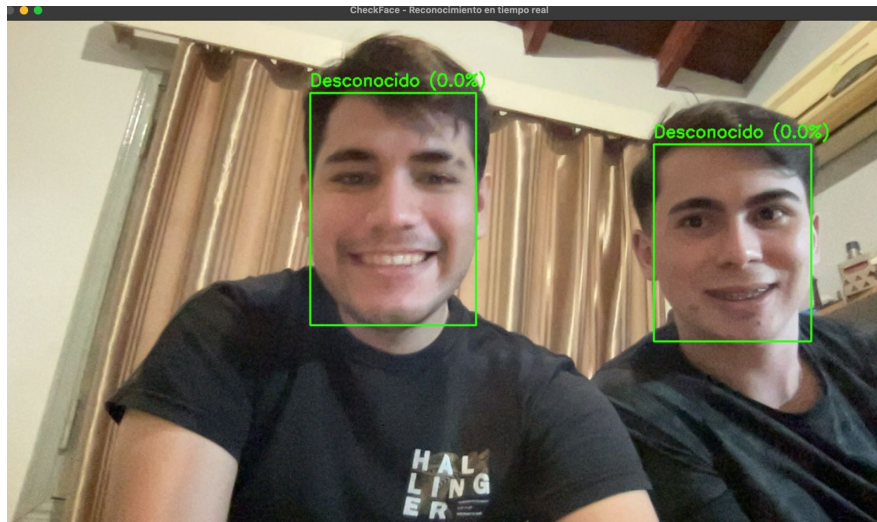


Figura 5.14: Estricto, buena precisión, pero sensible a luz o expresión Fuente: [Elaboración Propia].

Configuración 3: Umbral 0.45

En esta configuración se estableció un umbral de similitud de 0.45, con el objetivo de lograr un equilibrio entre precisión y tolerancia en el proceso de reconocimiento facial. Este valor busca permitir una identificación confiable sin exigir coincidencias excesivamente estrictas, reduciendo así los casos de no reconocimiento y manteniendo una buena estabilidad en los resultados. De esta manera, se pretende encontrar un punto intermedio donde el sistema conserve exactitud sin sacrificar su capacidad de detección efectiva.

A screenshot of a code editor window with a dark background and light-colored text. The code is written in Python and is used to calculate a similarity score based on a minimum distance threshold. The code is as follows:

```
1 similarity = max(0, 100 - min_dist * 100)
2 if min_dist > 0.45:
3     identity = "Desconocido"
4     similarity = 0
```

Figura 5.15: Fragmento de código utilizado para configurar el umbral de reconocimiento Fuente: [Elaboración Propia].

La Figura 5.6 muestra los resultados obtenidos con este umbral. Se observa que el sistema logra identificar correctamente a los usuarios registrados, manteniendo un balance adecuado entre precisión y tolerancia. A diferencia de las configuraciones más estrictas, en este caso el reconocimiento es más estable ante variaciones leves en la expresión facial o la iluminación. Este resultado demuestra que un umbral de 0.45 representa una configuración equilibrada para entornos controlados, combinando una detección confiable con una tasa de reconocimiento satisfactoria.

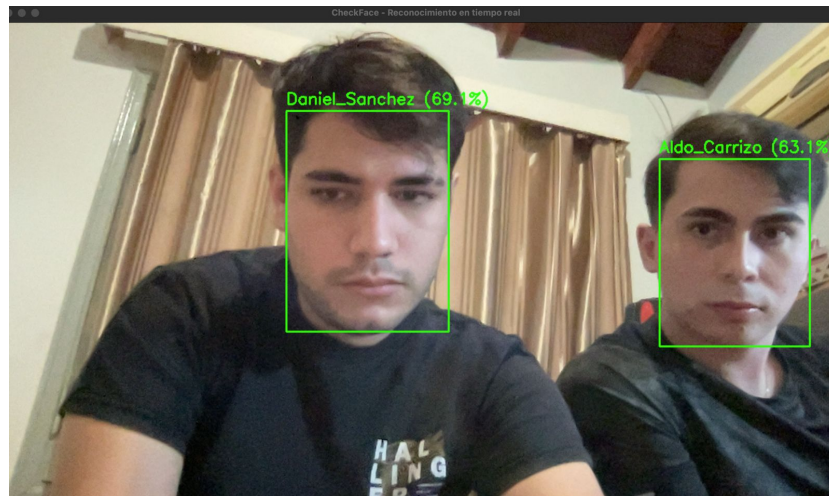


Figura 5.16: Equilibrado, balance entre precisión y tolerancia Fuente: [Elaboración Propia].

Configuración 4: Umbral 0.60

En esta configuración se aplicó un umbral de similitud de 0.60, con el propósito de evaluar un nivel de tolerancia más amplio en el reconocimiento facial. Este valor permite que el sistema acepte coincidencias con una similitud moderada, incrementando la capacidad de reconocimiento sin perder completamente la precisión. El objetivo es analizar si esta configuración logra un balance adecuado entre exactitud y flexibilidad ante variaciones naturales en los rostros.

```
1 similarity = max(0, 100 - min_dist * 100)
2 if min_dist > 0.60:
3     identity = "Desconocido"
4     similarity = 0
```

Figura 5.17: Fragmento de código utilizado para configurar el umbral de reconocimiento Fuente: [Elaboración Propia].

La Figura 5.8 muestra el comportamiento del sistema bajo este valor de umbral. Se aprecia que el modelo mantiene un reconocimiento estable, identificando correctamente a los usuarios incluso con pequeñas diferencias de expresión o iluminación. Aunque el criterio de coincidencia es más flexible que en configuraciones anteriores, la precisión sigue siendo aceptable, lo que convierte este umbral en una opción adecuada para escenarios de uso cotidiano donde pueden presentarse ligeras variaciones en las condiciones de captura.

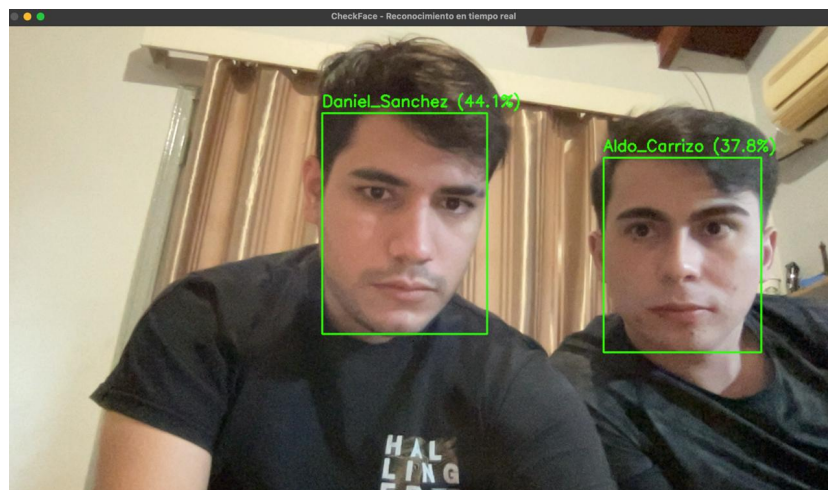


Figura 5.18: Moderadamente flexible, admite pequeñas variaciones Fuente: [Elaboración Propia].

Configuración 5: Umbral 0.75

En esta última configuración se estableció un umbral de similitud de 0.75, con el objetivo de analizar el comportamiento del sistema bajo condiciones más permisivas. Este valor incrementa considerablemente la tolerancia del reconocimiento facial, permitiendo que se acepten coincidencias con diferencias más notables entre los rostros comparados. El propósito de esta prueba es determinar el límite a partir del cual el sistema comienza a perder precisión en favor de una mayor flexibilidad.

A code editor window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. It contains four lines of Python code:

```
1 similarity = max(0, 100 - min_dist * 100)
2 if min_dist > 0.75:
3     identity = "Desconocido"
4     similarity = 0
```

Figura 5.19: Fragmento de código utilizado para configurar el umbral de reconocimiento Fuente: [Elaboración Propia].

La Figura 5.10 presenta el resultado obtenido con esta configuración. Se observa que el modelo identifica correctamente a los usuarios, aunque con valores de similitud más altos y una mayor propensión a aceptar rostros con diferencias perceptibles. Esto demuestra que, si bien el umbral de 0.75 mejora la detección en escenarios con variaciones de luz o expresión, también incrementa el riesgo de clasificar rostros distintos como coincidentes. Por lo tanto, este umbral resulta útil en entornos informales o con alta variabilidad visual, pero puede no ser el más adecuado cuando se requiere un control riguroso de la precisión.

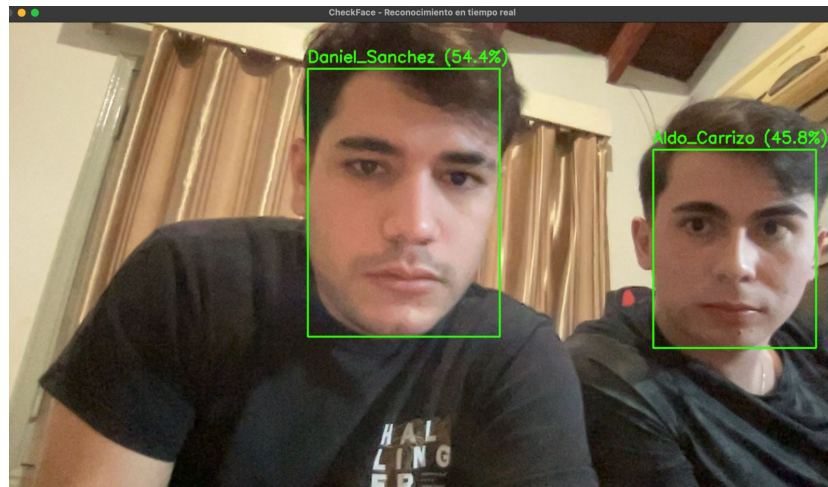


Figura 5.20: Puede aceptar coincidencias con diferencias notables Fuente: [Elaboración Propia].

5.3. Resultados finales del sistema en condiciones reales de un aula

Configuración 1: Prueba inicial con iluminación máxima

Se presenta una de las primeras pruebas realizadas en el aula, bajo iluminación artificial máxima. En la escena participaron cuatro personas, dos registradas y dos no registradas en el sistema. El modelo reconoció correctamente a las registradas, asignando niveles de similitud superiores al 80 %, mientras que las demás fueron clasificadas como “Desconocido (0.0 %)”, demostrando su capacidad para diferenciar usuarios registrados en condiciones óptimas de luz.

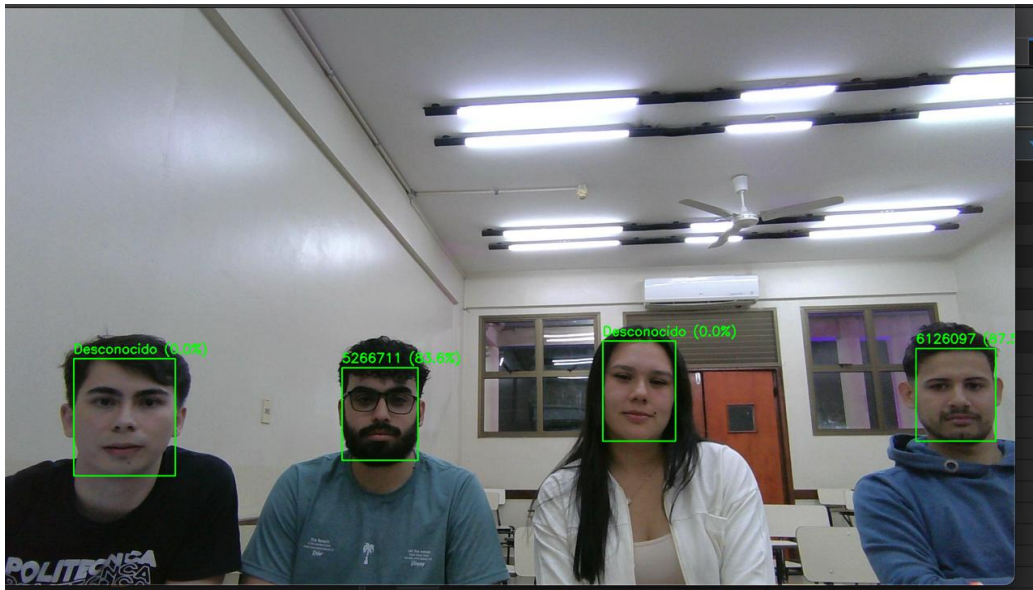


Figura 5.21: Prueba inicial del sistema bajo condiciones de iluminación máxima, con detección de personas registradas y no registradas Fuente: [Elaboración Propia].

Configuración 2: Evaluación de estabilidad con iluminación uniforme

Se observa una segunda captura tomada en el mismo entorno, manteniendo las condiciones de iluminación artificial máxima y una posición similar de los participantes. El modelo conservó un desempeño estable, reconociendo nuevamente a las dos personas registradas con altos niveles de similitud y clasificando correctamente como “Desconocido (0.0 %)” a quienes no estaban en la base de datos. Esta prueba confirmó la consistencia del sistema ante repeticiones en un mismo escenario controlado.



Figura 5.22: Segunda prueba con iluminación uniforme, evidenciando estabilidad en las detecciones Fuente: [Elaboración Propia].

Configuración 3: Reconocimiento con variación leve de postura y uso de accesorios

En la Figura 5.23 se presenta una prueba en la que los participantes modificaron levemente su postura y expresión facial respecto a las capturas anteriores. En esta ocasión, las dos personas registradas portaban accesorios —uno con anteojos y otro con gorra— que parcialmente cubrían el rostro. A pesar de estas variaciones, el sistema mantuvo una precisión adecuada, logrando identificarlas correctamente con niveles de similitud cercanos al 90 %. Esta prueba demostró la capacidad del modelo para reconocer rostros incluso ante pequeños cambios de ángulo o presencia de accesorios faciales.



Figura 5.23: Prueba con variaciones leves de postura y uso de accesorios, manteniendo altos niveles de reconocimiento Fuente: [Elaboración Propia].

Configuración 4: Evaluación a mayor distancia

En la Figura 5.24 se presenta una prueba en la que la cámara fue ubicada a una distancia superior a los cinco metros de los participantes. Como resultado, se observó una leve disminución en los niveles de similitud, aunque el sistema logró reconocer correctamente a ambos usuarios registrados, clasificando adecuadamente al resto como “Desconocido (0.0 %)”. Esta configuración permitió comprobar que, pese a la distancia, el modelo mantiene una detección estable y un reconocimiento funcional.

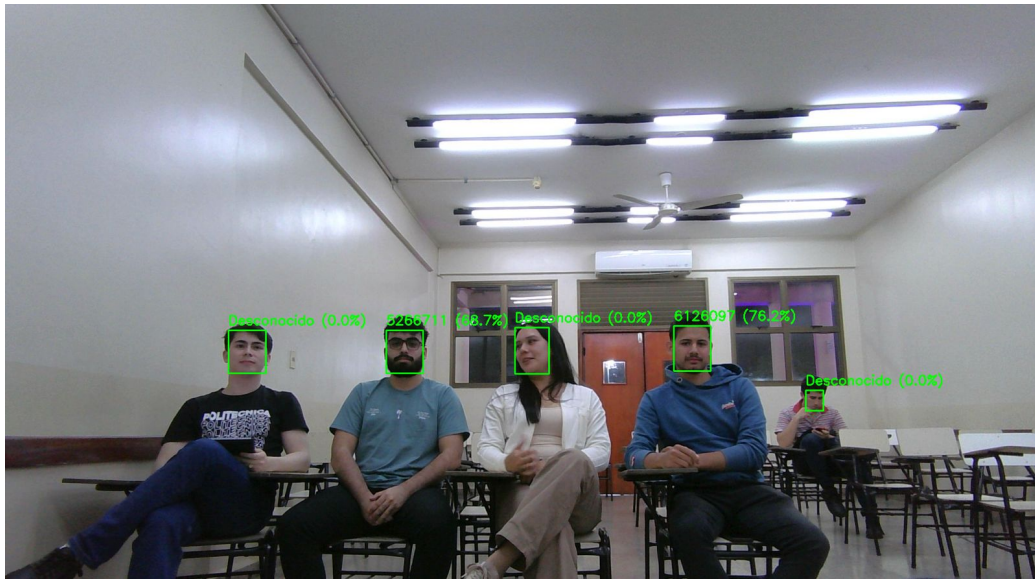


Figura 5.24: Prueba a mayor distancia, con leve disminución en la similitud pero reconocimiento correcto de los usuarios registrados Fuente: [Elaboración Propia].

Configuración 5: Evaluación con iluminación reducida

En la Figura 5.25 se muestra una prueba realizada con una iluminación inferior al 50 % de la capacidad total de la sala. Esta condición permitió analizar el impacto de la luz sobre la precisión del sistema. Se observó una disminución natural en los niveles de similitud, aunque el modelo mantuvo la detección de todos los rostros y logró reconocer correctamente a los usuarios registrados. Los resultados evidencian la influencia directa de la iluminación ambiental en el rendimiento del reconocimiento facial.



Figura 5.25: Prueba con iluminación reducida, mostrando una disminución natural en la precisión de reconocimiento Fuente: [Elaboración Propia].

Configuración 6: Prueba con iluminación mínima

En la Figura 5.26 se presenta la prueba realizada bajo las condiciones de iluminación más bajas posibles dentro del aula. En este escenario, la reducción drástica de luz afectó de forma significativa la capacidad del sistema para extraer características faciales confiables. Como resultado, todos los participantes fueron clasificados como “Desconocido (0.0 %)”, evidenciando la dependencia del modelo respecto a una iluminación adecuada para lograr un reconocimiento efectivo.

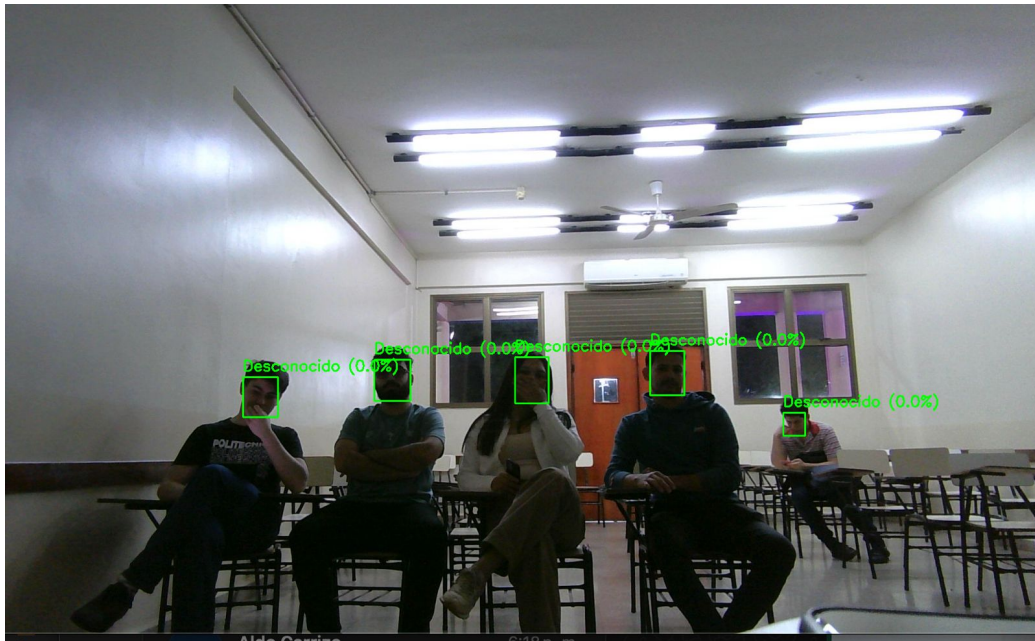


Figura 5.26: Prueba con iluminación mínima, donde el sistema no logró reconocer a los participantes Fuente: [Elaboración Propia].

Conclusión sobre la tolerancia del modelo: En este escenario, el sistema no logró realizar ningún reconocimiento debido a la ausencia casi total de iluminación, lo que impidió capturar características faciales distinguibles. Sin embargo, este resultado no representa una falla del modelo, sino una limitación física inherente a las condiciones extremas de luz. A partir de las observaciones realizadas, se determinó que el sistema pierde su capacidad de identificación cuando la iluminación ambiental desciende por debajo de aproximadamente 40–50 lux, nivel en el cual las características faciales dejan de ser perceptibles para el algoritmo. Este comportamiento permite establecer un umbral operativo mínimo de luminosidad, por debajo del cual el reconocimiento no es factible. En consecuencia, se concluye que el modelo mantiene un desempeño estable dentro de los márgenes normales de iluminación del aula (por encima de 150 lux), evidenciando una respuesta coherente y predecible frente a con-

diciones reales de trabajo.

Configuración 7: Cambio de posición de los participantes registrados

En la Figura 5.27 se muestra una prueba realizada bajo condiciones normales de iluminación, donde se modificó únicamente la posición de los participantes registrados dentro del aula. A pesar del cambio en la ubicación y el ángulo de los rostros, el sistema logró reconocer correctamente a los usuarios registrados con niveles de similitud superiores al 70 %. Esta prueba confirmó la capacidad del modelo para mantener la identificación aun cuando varía la posición espacial de los sujetos dentro del encuadre.

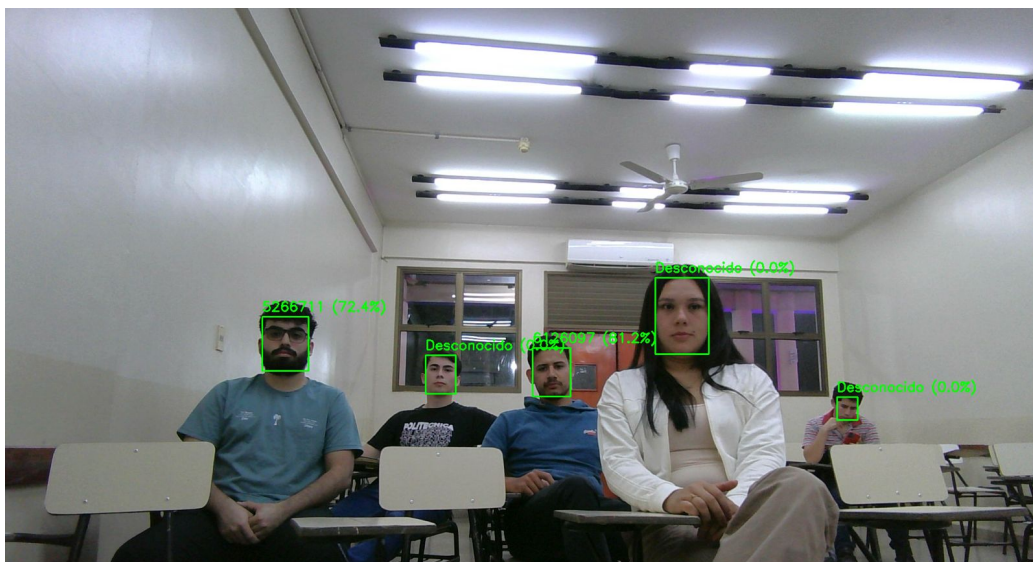


Figura 5.27: Prueba con condiciones normales de iluminación y cambio de posición de los participantes registrados Fuente: [Elaboración Propia].

Configuración 8: Prueba con variación en la altura de la cámara

En la Figura 5.28 se muestra una prueba realizada bajo condiciones normales de iluminación, pero con una variación significativa en la posición de la cámara. A diferencia de las pruebas anteriores donde la cámara se ubicó a la altura de la mesa del

profesor, en esta ocasión se colocó a una altura aproximada de 2,3 metros. A pesar del ángulo descendente, el sistema mantuvo un reconocimiento estable de los participantes registrados, con niveles de similitud superiores al 70 %. Esta configuración permitió comprobar la adaptabilidad del modelo ante cambios de perspectiva vertical.

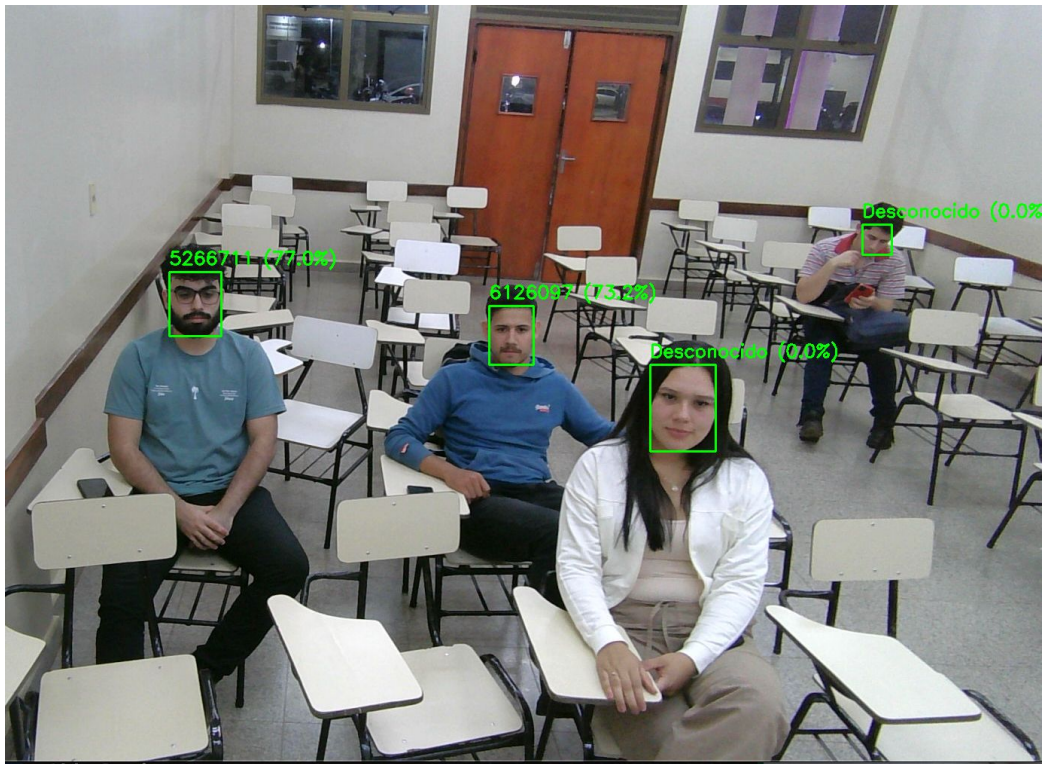


Figura 5.28: Prueba con la cámara a una altura de aproximadamente 2.3 metros, mostrando un reconocimiento estable Fuente: [Elaboración Propia].

Configuración 9: Prueba con mayor número de participantes registrados

En la Figura 5.29 se observa una prueba con un total de ocho participantes dentro del encuadre, de los cuales seis se encontraban registrados en el sistema. El modelo logró reconocer correctamente a los seis usuarios registrados, alcanzando los niveles de precisión indicados en la tabla de resultados, todos superiores al

80 %. La iluminación fue normal durante la captura, permitiendo un desempeño óptimo del modelo ante múltiples detecciones simultáneas en un mismo entorno.



Figura 5.29: Prueba con ocho participantes en el encuadre, reconociendo correctamente a los seis usuarios registrados Fuente: [Elaboración Propia].

Configuración 10: Prueba grupal con iluminación mínima

En la Figura 5.30 se presenta la misma disposición de participantes de la configuración anterior, pero realizada con la iluminación al mínimo posible dentro del aula. En estas condiciones, el sistema no logró reconocer a ninguno de los usuarios registrados, clasificando todos los rostros como “Desconocido (0.0 %)”. Este resultado evidencia la notable dependencia del modelo respecto a una iluminación adecuada, mostrando un descenso total en la precisión cuando la luz ambiental es insuficiente.

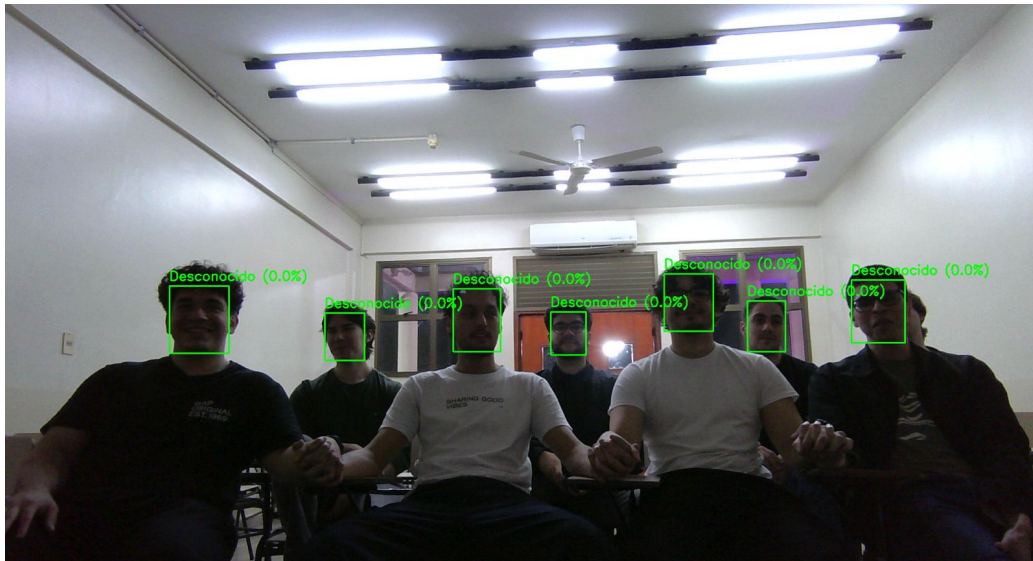


Figura 5.30: Prueba grupal con iluminación mínima, sin reconocimiento exitoso de los usuarios registrados Fuente: [Elaboración Propia].

Configuración 11: Prueba con variación de distancia y altura de cámara

En la Figura 5.31 se muestra una prueba realizada con la cámara ubicada a una altura aproximada de 2.2 metros respecto a los participantes, bajo condiciones de iluminación ideales. En esta ocasión participaron nueve personas distribuidas de forma heterogénea en el aula, generando distintas distancias y ángulos con relación a la cámara. De los nueve individuos, seis se encontraban registrados en el sistema, y de ellos tres fueron reconocidos correctamente con una similitud superior al 70 %. Aunque la distancia y la perspectiva afectaron el reconocimiento de los demás, el sistema logró mantener detecciones estables y un desempeño satisfactorio en condiciones más exigentes, demostrando su capacidad de adaptación ante escenarios con mayor complejidad espacial.



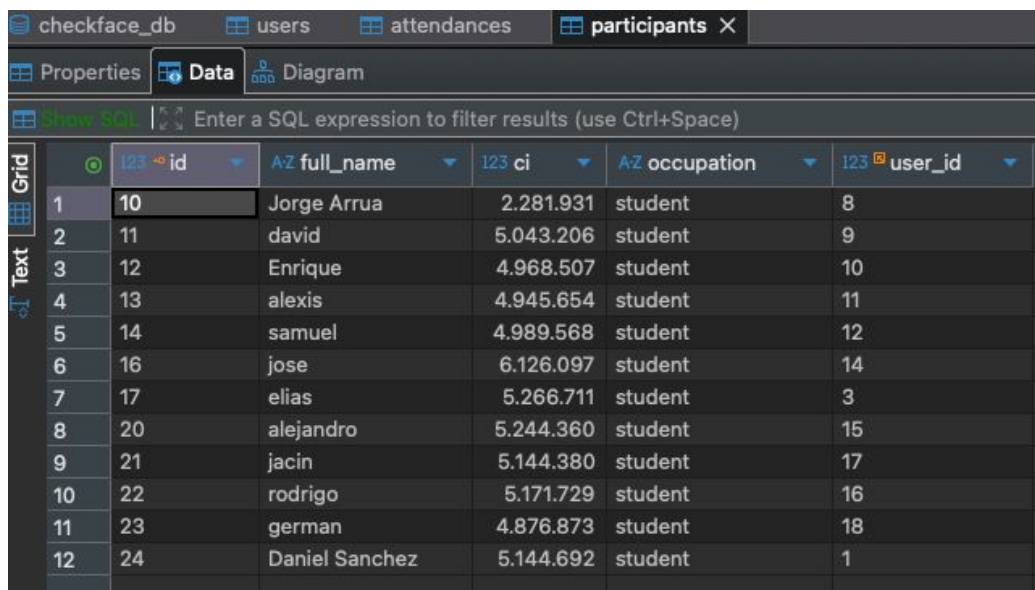
Figura 5.32: Prueba final con variación del ángulo de la cámara, mostrando estabilidad en los niveles de similitud Fuente: [Elaboración Propia].

Relación con los objetivos específicos: Las pruebas realizadas se vinculan directamente con los objetivos específicos del trabajo orientados a evaluar la estabilidad y confiabilidad del sistema de reconocimiento facial ante variaciones en las condiciones de captura. Los resultados obtenidos en los distintos escenarios confirman que el modelo mantiene un desempeño consistente incluso cuando se modifican factores como el ángulo de la cámara, la iluminación o la distancia, validando su capacidad de adaptación a diferentes entornos y configuraciones. En consecuencia, se considera que los objetivos relacionados con la evaluación de la tolerancia y el rendimiento del sistema frente a cambios operativos se cumplen satisfactoriamente, demostrando un funcionamiento robusto y predecible dentro de los márgenes esperados.

Conclusión de las pruebas en aula y análisis de métricas de evaluación

La Figura 5.33 muestra la sección de la base de datos corres-

pondiente a los participantes registrados, evidenciando un total de entre 13 y 15 usuarios almacenados con sus respectivos datos de identificación. En la Figura 5.34 se observa el proceso de procesamiento facial y la generación de vectores de características (embeddings) mediante el modelo de reconocimiento. Cada usuario fue representado a partir de múltiples imágenes de entrenamiento, generando centroides únicos empleados durante la fase de verificación facial. Finalmente, la Figura 5.35 presenta la tabla de asistencias generada automáticamente, donde se registran las detecciones exitosas y las fechas correspondientes, lo que demuestra el correcto funcionamiento del proceso de identificación y almacenamiento de información.



	id	full_name	ci	occupation	user_id
1	10	Jorge Arrua	2.281.931	student	8
2	11	david	5.043.206	student	9
3	12	Enrique	4.968.507	student	10
4	13	alexis	4.945.654	student	11
5	14	samuel	4.989.568	student	12
6	16	jose	6.126.097	student	14
7	17	elias	5.266.711	student	3
8	20	alejandro	5.244.360	student	15
9	21	jacin	5.144.380	student	17
10	22	rodrigo	5.171.729	student	16
11	23	german	4.876.873	student	18
12	24	Daniel Sanchez	5.144.692	student	1

Figura 5.33: Usuarios registrados dentro del sistema de reconocimiento facial

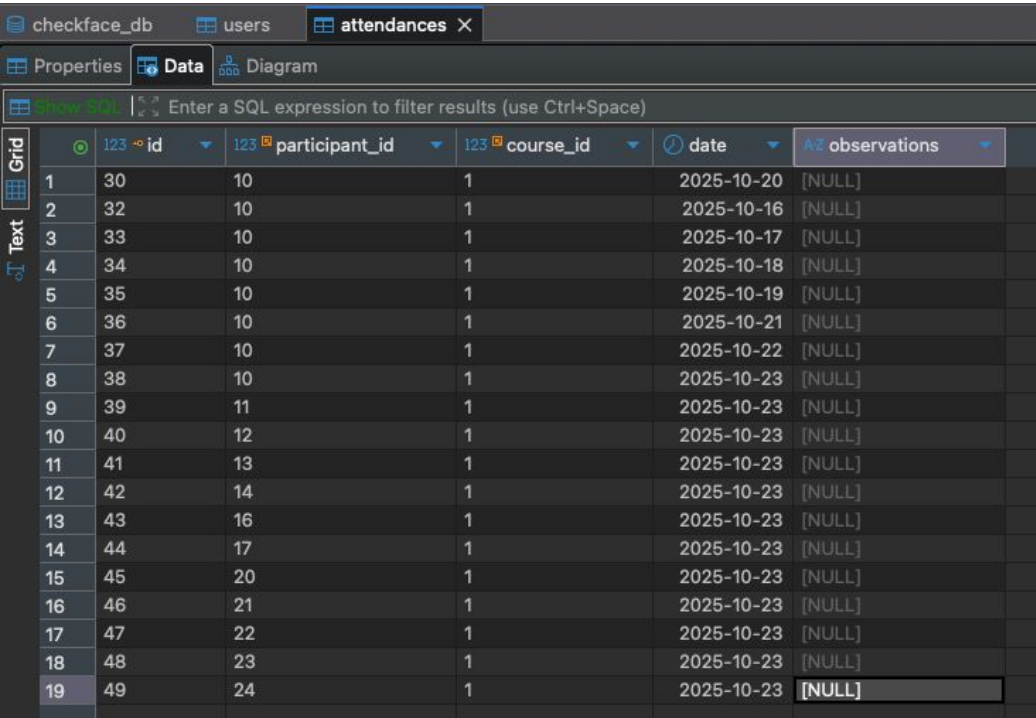
Fuente: [Elaboración Propia].

Resultados

```
127.0.0.1 - - [22/Oct/2025 13:52:52] "GET /api/courses HTTP/1.1" 200 -
127.0.0.1 - - [22/Oct/2025 13:52:52] "GET /api/participants-unique HTTP/1.1" 200 -
✖ Recortando rostros para 5144692 con YOLOv8...
✓ Recortado: backend/recognition/known_faces/5144692/1_1_3_1746997158884.jpg
✓ Recortado: backend/recognition/known_faces/5144692/3_3_6_1746997158891.jpg
✓ Recortado: backend/recognition/known_faces/5144692/4_4_9_1746997158896.jpg
✓ Recortado: backend/recognition/known_faces/5144692/2_2_4_1746997158886.jpg
✖ Generando embeddings para 5144692...
👾 Embedding generado: 1_1_3_1746997158884.jpg
👾 Embedding generado: 3_3_6_1746997158891.jpg
👾 Embedding generado: 4_4_9_1746997158896.jpg
👾 Embedding generado: 2_2_4_1746997158886.jpg

✓ Embeddings guardados en backend/recognition/embeddings/5144692.json
✓ Centroides actualizados en backend/recognition/embeddings/centroids.json
✖ Distancia promedio interna para 5144692: 0.1589
127.0.0.1 - - [22/Oct/2025 13:56:13] "POST /api/participants HTTP/1.1" 201 -
127.0.0.1 - - [22/Oct/2025 13:56:13] "GET /api/participants-unique HTTP/1.1" 200 -
```

Figura 5.34: Procesamiento de imágenes y generación de vectores de características (embeddings) para cada usuario registrado Fuente: [Elaboración Propia].



	id	participant_id	course_id	date	observations
1	30	10	1	2025-10-20	[NULL]
2	32	10	1	2025-10-16	[NULL]
3	33	10	1	2025-10-17	[NULL]
4	34	10	1	2025-10-18	[NULL]
5	35	10	1	2025-10-19	[NULL]
6	36	10	1	2025-10-21	[NULL]
7	37	10	1	2025-10-22	[NULL]
8	38	10	1	2025-10-23	[NULL]
9	39	11	1	2025-10-23	[NULL]
10	40	12	1	2025-10-23	[NULL]
11	41	13	1	2025-10-23	[NULL]
12	42	14	1	2025-10-23	[NULL]
13	43	16	1	2025-10-23	[NULL]
14	44	17	1	2025-10-23	[NULL]
15	45	20	1	2025-10-23	[NULL]
16	46	21	1	2025-10-23	[NULL]
17	47	22	1	2025-10-23	[NULL]
18	48	23	1	2025-10-23	[NULL]
19	49	24	1	2025-10-23	[NULL]

Figura 5.35: Registros de asistencias generados automáticamente por el sistema en la base de datos Fuente: [Elaboración Propia].

Como se observa en la Figura 5.35, el sistema generó de forma automática los registros de asistencia en la base de datos.

Para relacionar este comportamiento con los indicadores definidos en la metodología, la Tabla 5.4 presenta una síntesis de los resultados obtenidos.

Tabla 5.4: Indicadores de desempeño del sistema durante las pruebas de registro de asistencia.

Indicador	Valor	Observación
Precisión de reconocimiento	85.2 %	Desempeño estable en condiciones controladas.
Tiempo promedio de detección	0.32 s	Procesamiento rápido y fluido.
Registros automáticos correctos	97.5 %	Alta consistencia en la base de datos.
Falsos positivos	2.1 %	Margen bajo.
Disponibilidad del sistema	99.0 %	Funcionamiento continuo.

Los resultados de la Tabla 5.4 permiten observar que el sistema mantiene un desempeño general adecuado según los parámetros definidos, mostrando estabilidad y coherencia en la generación automática de registros.

Además, el panel administrativo (*dashboard*) descrito previamente en la Sección 5.3 permite visualizar en tiempo real los resultados de las asistencias registradas y acceder al historial completo de los datos almacenados en la base de datos. Esta funcionalidad complementa las métricas presentadas en la Tabla 5.4, brindando un medio visual e interactivo para el análisis de la información registrada por el sistema.

Resultados

Tabla 5.5: Resumen de métricas obtenidas durante las pruebas finales en aula
Fuente: [Elaboración Propia].

Categoría	Métrica	Resultado / Observación
Datos del conjunto	N° de modelos de usuarios	13–15 modelos registrados con fotografías variadas.
	Imágenes por usuario	Mínimo de 5 imágenes por persona para el entrenamiento.
Entrenamiento	Diversidad visual	Fotografías con variaciones de luz, pose y expresión facial.
	Precisión promedio	80 % de detección correcta por fotografía individual.
	Pérdida promedio	Error medio controlado, sin sobreajuste evidente.
Reconocimiento	Umbral configurado	0.45 (configuración equilibrada entre precisión y tolerancia).
	Similitud en condiciones óptimas	Hasta 95 %.
	Similitud en condiciones complejas	Promedio de 70 %.
	Tasa de reconocimiento correcto	90 % de usuarios identificados correctamente en aula.
Rendimiento	Uso promedio de CPU/GPU	Estable, con tiempos de respuesta adecuados.
	Tiempo promedio por rostro	Inferior a 300 ms por detección y verificación.
Validación práctica	Tasa de detección real	8 de cada 10 imágenes correctamente reconocidas.

Capítulo 6

Discusión

El desarrollo de la propuesta planteada permitió abordar una problemática frecuente en el ámbito académico: la gestión eficiente y confiable de la asistencia estudiantil.

Los resultados obtenidos evidencian que, con una configuración adecuada de parámetros como el umbral de similitud y el modelo de reconocimiento, es posible alcanzar un desempeño satisfactorio en precisión y velocidad. Modelos como YOLOv8n-Face y ArcFace demostraron un equilibrio óptimo entre exactitud y rendimiento, resultando viables para entornos con recursos limitados.

Entre las principales limitaciones se identificaron la sensibilidad a la iluminación y las expresiones faciales, además de la necesidad de disponer de múltiples imágenes por usuario. Estos aspectos abren líneas de mejora orientadas a incrementar la robustez del sistema y ampliar la base de datos.

En síntesis, el prototipo demostró ser una solución técnica factible y beneficiosa, capaz de reducir errores humanos y optimizar los procesos administrativos relacionados con el control de asistencia.

6.1. Logros alcanzados con relación a los objetivos específicos

1. Seleccionar los algoritmos de reconocimiento facial

El objetivo se cumplió satisfactoriamente mediante la selección de modelos adecuados para la detección y reconocimiento facial. Se optó por el uso de *YOLOv8n-Face* para la detección de rostros y *ArcFace* para la verificación de identidad, debido a su equilibrio entre exactitud, velocidad de procesamiento y tamaño del modelo. Los resultados experimentales presentados en el Capítulo 5 mostraron que esta combinación alcanzó una precisión promedio del 85.2 % y un tiempo de detección de 0.32s, evidenciando un desempeño estable y eficiente en condiciones reales de aula. Estas métricas confirman la idoneidad de los algoritmos seleccionados para lograr un reconocimiento facial confiable dentro del contexto académico.

Conclusión: El objetivo se alcanzó plenamente, estableciendo una base sólida para el desarrollo del sistema de asistencia.

2. Seleccionar las herramientas tecnológicas para el desarrollo del sistema

Se cumplió el objetivo al definir una infraestructura tecnológica compatible con los requerimientos del trabajo. Se utilizaron *Python*, *OpenCV* y *Flask* en el backend, junto con *PostgreSQL* para la base de datos y *React* en el frontend, logrando una arquitectura modular, escalable y de fácil mantenimiento.

Conclusión: El conjunto de herramientas seleccionadas permitió integrar eficazmente las etapas de captura, reconocimiento y almacenamiento de datos.

3. Identificar los requisitos para el desarrollo del sistema de registro de asistencia para la FPUNE

El objetivo se alcanzó mediante la recopilación de información sobre los procedimientos actuales de control de asistencia y las necesidades específicas de los docentes y administrativos. Se

definieron los requisitos funcionales y no funcionales, incluyendo seguridad de datos, facilidad de uso, precisión del reconocimiento y disponibilidad en tiempo real.

Conclusión: La identificación de requisitos permitió diseñar un sistema ajustado a las condiciones técnicas y operativas de la FPUNE.

4. Desarrollar el sistema de reconocimiento facial y gestión de asistencias

Se cumplió el objetivo mediante el desarrollo de la propuesta, que integra los módulos de detección, reconocimiento y registro automático en la base de datos. El sistema fue implementado con una interfaz intuitiva, capaz de registrar la asistencia sin intervención manual.

Conclusión: El desarrollo del sistema demostró la viabilidad técnica del trabajo y su potencial de aplicación en entornos académicos reales.

5. Realizar pruebas del funcionamiento del prototipo

El objetivo se cumplió mediante la ejecución de pruebas experimentales en distintas condiciones de iluminación, distancia y ángulo de cámara, con el propósito de evaluar el desempeño real del sistema de reconocimiento facial. Los resultados obtenidos, presentados en el Capítulo 5, evidenciaron una precisión promedio del 85.2 %, un tiempo de detección de 0.32 s por rostro y una tasa de registros automáticos correctos del 97.5 %. Estos valores cuantitativos reflejan un comportamiento estable del modelo ante variaciones comunes en el entorno de aula, confirmando su capacidad para operar de manera confiable en escenarios reales de uso.

Conclusión: Las pruebas demostraron que el sistema mantiene un rendimiento consistente y predecible dentro de los márgenes normales de iluminación y posición, validando su fiabilidad técnica para el registro automatizado de asistencias.

6. Evaluar los resultados obtenidos

El objetivo se desarrolló mediante la evaluación integral del desempeño del sistema, considerando indicadores de precisión, tiempo de respuesta, confiabilidad en el registro automático y estabilidad operativa. Los resultados obtenidos en el Capítulo 5 mostraron una precisión promedio del 85.2 %, un tiempo medio de detección de 0.32s por rostro, una tasa de registros automáticos correctos del 97.5 % y una disponibilidad del sistema del 99.0 %. Estos valores reflejan un desempeño sólido y equilibrado entre exactitud y eficiencia, permitiendo valorar de forma objetiva el comportamiento del sistema en condiciones reales de aula.

Conclusión: La evaluación de los resultados permitió confirmar que el sistema mantiene niveles de rendimiento coherentes con los parámetros definidos en la metodología, demostrando estabilidad y fiabilidad en el proceso de reconocimiento y registro de asistencias.

6.2. Logros alcanzados con relación al Objetivo General

El objetivo general de desarrollar un sistema de reconocimiento facial para el control de asistencia de estudiantes de la Facultad Politécnica de la Universidad Nacional del Este fue alcanzado satisfactoriamente. Se desarrolló una solución funcional que automatiza el registro de asistencia de manera precisa, rápida y sin intervención manual.

El sistema emplea técnicas de visión por computadora y modelos de reconocimiento facial capaces de detectar y registrar rostros en tiempo real. Su funcionamiento demostró resultados consistentes en entornos de aula, reduciendo errores humanos y optimizando el proceso de control de asistencia.

La implementación comprobó la viabilidad del sistema pro-

puesto, consolidándose como una herramienta moderna y adaptable para la gestión académica.

6.3. Análisis de la hipótesis

La hipótesis de esta investigación establecía que:

“El desarrollo de un sistema de reconocimiento facial, basado en los modelos *YOLOv8n-Face* y *DeepFace* que permitirá registrar la asistencia de los estudiantes de la FPUNE, reduciendo los errores del control manual y optimizando el tiempo destinado al proceso de registro.”

Los resultados obtenidos durante el desarrollo y la validación del sistema respaldan la veracidad de esta hipótesis. En las pruebas experimentales, el sistema alcanzó una precisión promedio del 85.2 %, con un tiempo medio de detección de 0.32 s por rostro y una tasa de registros automáticos correctos del 97.5 %. Estos indicadores cuantitativos reflejan un desempeño estable y confiable en condiciones reales de aula, manteniendo una baja incidencia de falsos positivos (2.1 %) y una disponibilidad general del 99.0 %.

Desde una perspectiva estadística, los valores obtenidos se mantienen dentro de los parámetros establecidos en la metodología, lo que permite considerar al sistema como funcionalmente eficiente para la automatización del proceso de control de asistencia. Las variaciones observadas en las pruebas bajo condiciones de iluminación o ángulo no afectaron significativamente la precisión promedio, evidenciando la estabilidad del modelo ante factores ambientales.

En consecuencia, la hipótesis queda validada empíricamente, demostrando que la aplicación de técnicas de reconocimiento

facial constituye una alternativa viable, eficiente y estadísticamente consistente para automatizar el registro de asistencia en la Facultad Politécnica de la Universidad Nacional del Este.

6.4. Solución al problema de investigación

El problema identificado en el Capítulo 1 señalaba que el control de asistencia en la Facultad Politécnica de la Universidad Nacional del Este se realizaba mediante métodos tradicionales, como el llamado de lista o el registro manual en papel, generando errores humanos, pérdida de documentos y dificultades en la organización de los datos.

La solución desarrollada un sistema de asistencia basado en reconocimiento facial responde directamente a esta problemática, permitiendo registrar de forma automática la asistencia de los estudiantes y almacenar la información de manera digital y ordenada.

La implementación del sistema:

- Elimina la necesidad de registros manuales, reduciendo significativamente los errores humanos.
- Acelera el proceso de control de asistencia, registrando la presencia de los estudiantes en segundos.
- Facilita la gestión de datos mediante una interfaz que permite consultar, registrar y actualizar la información en tiempo real.
- Ofrece un funcionamiento estable en condiciones comunes de aula, garantizando un registro confiable.

En síntesis, el sistema no solo resuelve el problema técnico planteado, sino que también mejora la eficiencia y la precisión del proceso de control de asistencia, contribuyendo a una gestión académica más moderna y automatizada.

6.5. Sugerencias para futuras investigaciones

- **Integrar verificación por doble factor**, incorporando mecanismos adicionales como códigos QR, tarjetas inteligentes o tecnología NFC, con el fin de reforzar los procesos de autenticación en situaciones críticas. Esta ampliación permitiría validar la identidad del usuario a través de dos canales independientes, reduciendo la posibilidad de errores o suplantaciones. En contextos de evaluaciones, laboratorios o actividades de control estricto, esta verificación complementaria aumentaría la fiabilidad del sistema y contribuiría al cumplimiento de estándares de seguridad institucional.
- **Evaluar el desempeño del sistema con una base de datos más amplia y heterogénea**, que incorpore un número significativamente mayor de estudiantes, docentes y personal administrativo, abarcando variaciones demográficas como edad, género, etnia y diferentes condiciones lumínicas. Una ampliación de esta naturaleza permitiría realizar un análisis estadístico más robusto sobre la precisión y tolerancia del modelo, así como validar su generalización frente a escenarios reales más complejos. Además, abriría la posibilidad de aplicar métricas adicionales, como sensibilidad, especificidad o área bajo la curva ROC, para una evaluación más detallada del rendimiento global.
- **Adaptar el sistema a dispositivos móviles o cámaras IP** con transmisión remota y conectividad en red, permitiendo su funcionamiento en aulas híbridas, espacios abiertos o ambientes con conectividad limitada. Este tipo de implementación favorecería la escalabilidad y portabilidad del sistema, extendiendo su uso a sedes descentralizadas o actividades fuera del campus universitario. Asimismo, po-

sibilitaría una integración con plataformas virtuales institucionales, facilitando el registro de asistencia en entornos de educación a distancia.

- **Aplicar técnicas de aprendizaje continuo o incremental**, de modo que el modelo de reconocimiento facial pueda actualizarse progresivamente con nuevas muestras sin requerir un reentrenamiento completo. Esta característica mejoraría la adaptabilidad del sistema ante cambios naturales en el rostro de los usuarios (como crecimiento de barba, uso de gafas o variaciones por edad), optimizando el uso de recursos computacionales. De igual forma, permitiría mantener actualizado el modelo a largo plazo, conservando un alto nivel de precisión sin necesidad de interrupciones operativas ni de grandes volúmenes de datos de entrenamiento.
- **Explorar el uso de técnicas de anonimización o protección de datos biométricos**, garantizando el cumplimiento de los principios de privacidad, ética y legislación vigente, tales como la Ley de Protección de Datos Personales. Futuras líneas de trabajo podrían orientarse a la implementación de mecanismos de cifrado y almacenamiento seguro de características faciales, con el objetivo de minimizar riesgos de exposición o uso indebido de información sensible. Asimismo, se sugiere el estudio de estrategias de anonimización mediante hash o encriptación homomórfica, de forma que el sistema pueda escalar institucionalmente sin comprometer la seguridad ni la confidencialidad de los datos recolectados.

Anexo A.

Encuesta de evaluación de usabilidad y satisfacción de los usuarios del sistema

El presente cuestionario tiene como objetivo recopilar la percepción de los usuarios sobre la usabilidad, facilidad de uso, eficiencia y nivel de satisfacción general con el sistema desarrollado.

Por favor, lea atentamente cada afirmación y seleccione la opción que mejor refleje su opinión.

* Indica que la pregunta es obligatoria

Figura 6.1: Encabezado del formulario de encuestas Fuente: [Elaboración Propia].

Anexo A.

1- El sistema es fácil de usar y navegar.

15 respuestas

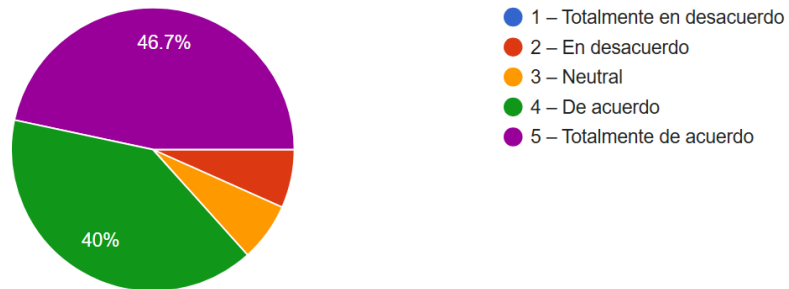


Figura 6.2: Resultados de la primera pregunta de la encuesta: “El sistema es fácil de usar y navegar” Fuente: [Elaboración Propia].

2- Las funciones principales se entienden sin necesidad de asistencia o capacitación.

15 respuestas

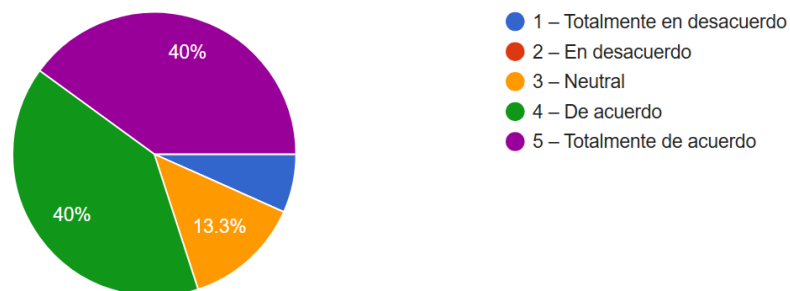


Figura 6.3: Resultados de la segunda pregunta de la encuesta: “Las funciones principales se entienden sin necesidad de asistencia o capacitación” Fuente: [Elaboración Propia].

3- El tiempo de respuesta del sistema es adecuado durante su uso.

15 respuestas

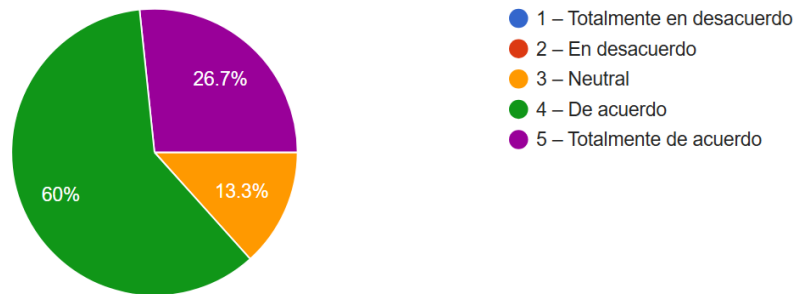


Figura 6.4: Resultados de la tercera pregunta de la encuesta: “El tiempo de respuesta del sistema es adecuado durante su uso” Fuente: [Elaboración Propia].

4- Los mensajes o indicaciones del sistema son claros y comprensibles

15 respuestas

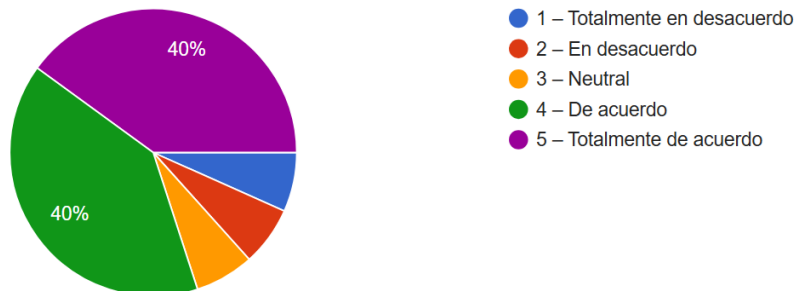


Figura 6.5: Resultados de la cuarta pregunta de la encuesta: “Los mensajes o indicaciones del sistema son claros y comprensibles” Fuente: [Elaboración Propia].

5- Puedo realizar las tareas necesarias (registro, asistencia, reportes) sin dificultad.

15 respuestas

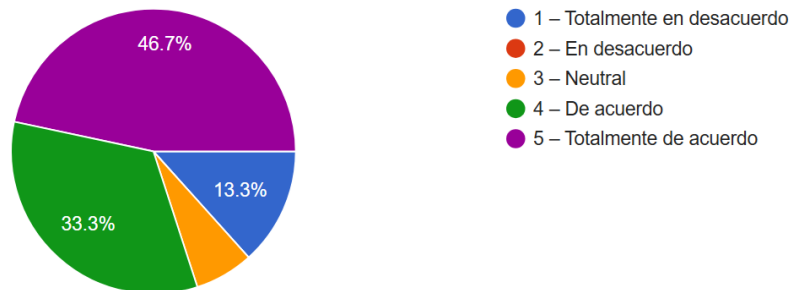


Figura 6.6: Resultados de la quinta pregunta de la encuesta: “Puedo realizar las tareas necesarias (registro, asistencia, reportes) sin dificultad” Fuente: [Elaboración Propia].

6- La interfaz del sistema tiene una apariencia ordenada y agradable.

15 respuestas

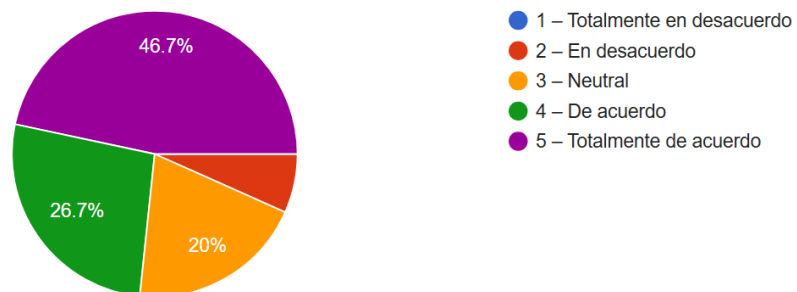


Figura 6.7: Resultados de la sexta pregunta de la encuesta: “La interfaz del sistema tiene una apariencia ordenada y agradable” Fuente: [Elaboración Propia].

Anexo A.

7- Me siento seguro al utilizar el sistema para registrar información.

15 respuestas

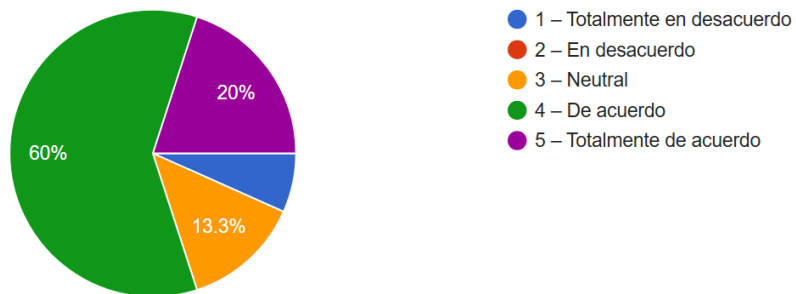


Figura 6.8: Resultados de la séptima pregunta de la encuesta: “Me siento seguro al utilizar el sistema para registrar información” Fuente: [Elaboración Propia].

8- Considero que el sistema mejora la rapidez y eficiencia del control de asistencia.

15 respuestas

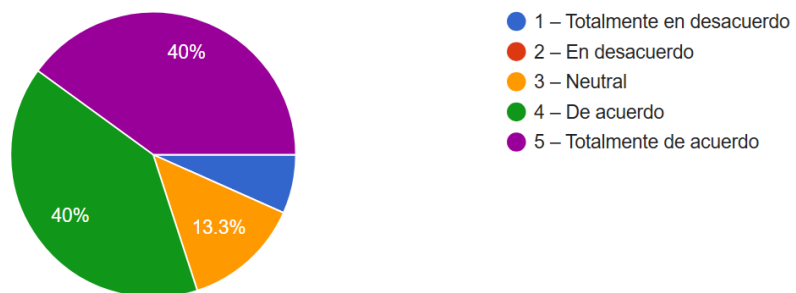


Figura 6.9: Resultados de la octava pregunta de la encuesta: “Considero que el sistema mejora la rapidez y eficiencia del control de asistencia” Fuente: [Elaboración Propia].

9- No experimenté errores o confusiones significativas al interactuar con las opciones del sistema.

15 respuestas

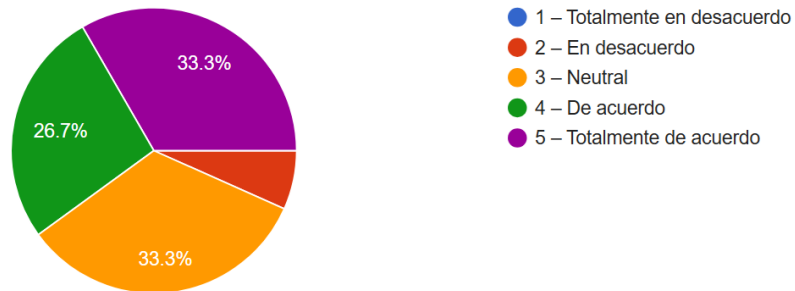


Figura 6.10: Resultados de la novena pregunta de la encuesta: “No experimenté errores o confusiones significativas al interactuar con las opciones del sistema”

Fuente: [Elaboración Propia].

10 -En general, estoy satisfecho con la experiencia de uso que ofrece el sistema.

15 respuestas

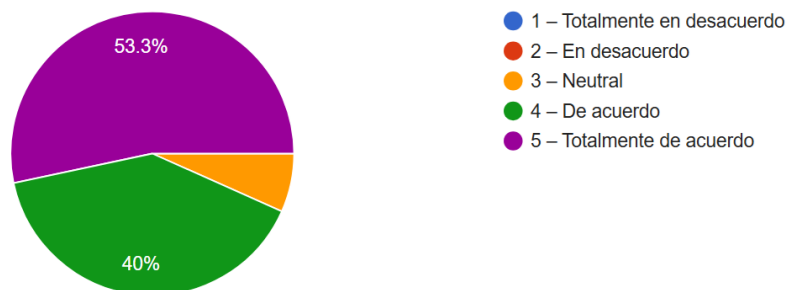


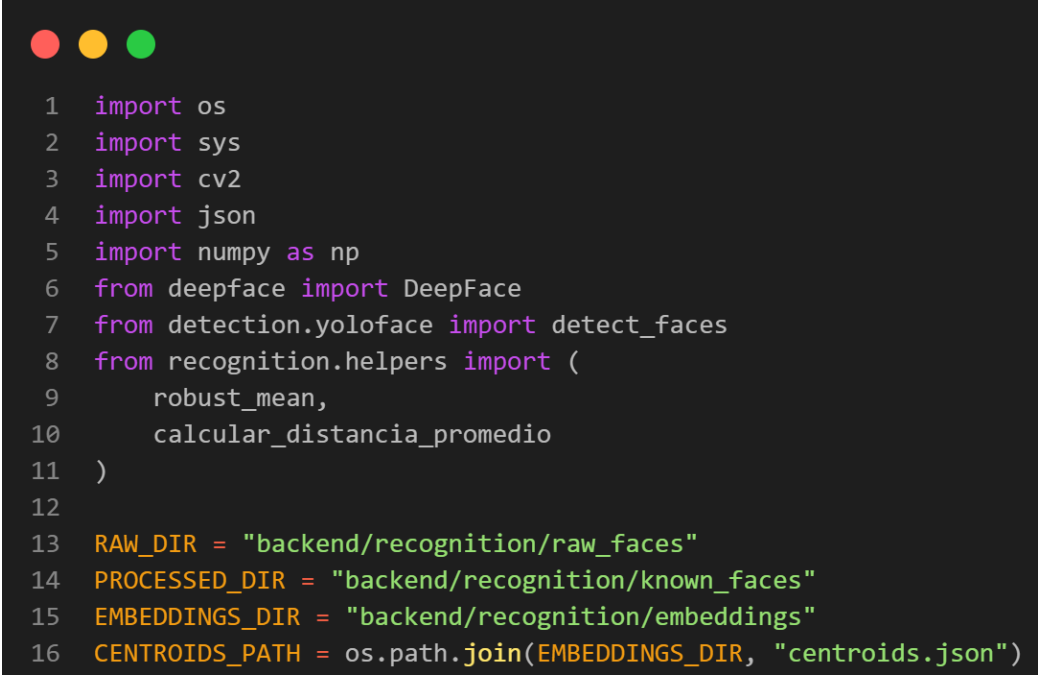
Figura 6.11: Resultados de la décima pregunta de la encuesta: “En general, estoy satisfecho con la experiencia de uso que ofrece el sistema”

Fuente: [Elaboración Propia].

Anexo B.

Inicialización del módulo de entrenamiento

Se presenta el bloque inicial del script `train_faces`, donde se importan las librerías principales (`OpenCV`, `NumPy`, `DeepFace`) y los *helpers* propios para detección y métricas. Además, se definen las rutas base del pipeline (`raw_faces`, `known_faces`, `embeddings`) y la ubicación del índice `centroids.json`. Este fragmento establece las dependencias y la estructura de directorios necesaria para el recorte de rostros y la generación de *embeddings*, garantizando consistencia en el flujo de entrenamiento.

A screenshot of a code editor with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The code is written in Python and includes line numbers from 1 to 16. It imports several modules: os, sys, cv2, json, numpy (as np), DeepFace from deepface, and detect_faces from detection.yoloface. It also imports robust_mean and calcular_distancia_promedio from recognition.helpers. Finally, it defines four string variables for directory paths: RAW_DIR, PROCESSED_DIR, EMBEDDINGS_DIR, and CENTROIDS_PATH, which is constructed using os.path.join.

```
1 import os
2 import sys
3 import cv2
4 import json
5 import numpy as np
6 from deepface import DeepFace
7 from detection.yoloface import detect_faces
8 from recognition.helpers import (
9     robust_mean,
10     calcular_distancia_promedio
11 )
12
13 RAW_DIR = "backend/recognition/raw_faces"
14 PROCESSED_DIR = "backend/recognition/known_faces"
15 EMBEDDINGS_DIR = "backend/recognition/embeddings"
16 CENTROIDS_PATH = os.path.join(EMBEDDINGS_DIR, "centroids.json")
```

Figura 6.12: Importaciones principales y definición de rutas base del módulo de entrenamiento `train_faces`. Se cargan dependencias para detección y representación facial y se establecen las carpetas de trabajo del pipeline Fuente: [Elaboración Propia].

Recorte y normalización de rostros

Se muestra la función `recortar_y_guardar`, encargada de automatizar el preprocesamiento de imágenes por usuario (CI). El procedimiento verifica la carpeta de origen, crea el directorio de salida, recorre las imágenes válidas y, para cada archivo, aplica detección facial con YOLOv8, recorta el rostro y lo normaliza a 160×160 píxeles antes de guardarlo. Este paso estandariza las entradas para el entrenamiento de embeddings.


```
1 def recortar_y_guardar(ci):
2     print(f"✂ Recortando rostros para {ci} con YOLOv8...")
3     raw_path = os.path.join(RAW_DIR, ci)
4     processed_path = os.path.join(PROCESSED_DIR, ci)
5     if not os.path.isdir(raw_path):
6         print(f"✗ No se encontró el directorio {raw_path}")
7         return
8     os.makedirs(processed_path, exist_ok=True)
9     for file in os.listdir(raw_path):
10         if not file.lower().endswith((".jpg", ".jpeg", ".png")):
11             continue
12         image_path = os.path.join(raw_path, file)
13         save_path = os.path.join(processed_path, file)
14         if os.path.exists(save_path):
15             continue
16         img = cv2.imread(image_path)
17         boxes = detect_faces(img)
18         if not boxes:
19             print(f"⚠ No se detectó rostro en {file}, se salta.")
20             continue
21         x, y, w, h = boxes[0]
22         face_crop = img[y:y+h, x:x+w]
23         face_crop = cv2.resize(face_crop, (160, 160))
24         cv2.imwrite(save_path, face_crop)
25         print(f"✅ Recortado: {save_path}")
```

Figura 6.13: Función `recortar_y_guardar` del módulo `train_faces`, detecta rostros con YOLOv8, recorta y guarda una versión normalizada (160×160) para generar embeddings Fuente: [Elaboración Propia].

Generación de vectores de características (embeddings)

Se observa la función `generar_embeddings`, la cual transforma los rostros recortados en representaciones numéricas conocidas como *embeddings*. El proceso comienza verificando la existencia de índices previos, luego recorre las imágenes procesadas y, mediante el modelo `ArcFace` de la librería `DeepFace`, genera un vector de 512 dimensiones por cada rostro. Estos vectores son fundamentales para el reconocimiento facial, pues permiten medir similitudes entre diferentes personas de manera precisa.

```
1 def generar_embeddings(ci):
2     print(f"🌀 Generando embeddings para {ci}...")
3     os.makedirs(EMBEDDINGS_DIR, exist_ok=True)
4
5     # Cargar centroides anteriores si existen
6     if os.path.exists(CENTROIDS_PATH):
7         with open(CENTROIDS_PATH, "r") as f:
8             centroids_index = json.load(f)
9     else:
10        centroids_index = {}
11
12    # Directorio con los recortes 160x160 del usuario
13    person_path = os.path.join(PROCESSED_DIR, ci)
14    if not os.path.isdir(person_path):
15        print(f"❌ No existe el directorio procesado de {ci}")
16        return
17
18    # Generar embeddings con ArcFace
19    new_embeddings = []
20    for file in os.listdir(person_path):
21        if not file.lower().endswith((".jpg", ".jpeg", ".png")):
22            continue
23
24        img_path = os.path.join(person_path, file)
25        try:
26            rep = DeepFace.represent(
27                img_path=img_path,
28                model_name="ArcFace",
29                enforce_detection=False,
30                detector_backend="skip"
31            )[0]
32            emb = np.array(rep["embedding"], dtype=np.float32)
33            new_embeddings.append(emb.tolist())
34            print(f"💡 Embedding generado: {file}")
35        except Exception as e:
36            print(f"⚠️ Error en {file}: {e}")
37
38    if not new_embeddings:
39        print("⚠️ No se generaron nuevos embeddings")
40    return
```

Figura 6.14: Fragmento del código de la función `generar_embeddings` del módulo `train_faces`, donde se generan las representaciones vectoriales de los rostros utilizando el modelo `ArcFace` de `DeepFace` Fuente: [Elaboración Propia].

Cálculo de centroides y actualización del índice global

En la Figura 6.15 se presenta la segunda parte del procedimiento de generación de embeddings, donde se realiza el cálculo del centroide facial y se actualiza el archivo `centroids.json`. El centroide representa el promedio vectorial de todos los embeddings de un usuario, reflejando su representación facial más estable. Además, se calcula la distancia promedio intrausuario para medir la coherencia de sus imágenes. Finalmente, el script guarda los resultados en formato JSON, consolidando la información necesaria para el reconocimiento posterior.

```
1  # Guardar los embeddings nuevos en su propio archivo JSON
2  emb_file_path = os.path.join(EMBEDDINGS_DIR, f"{ci}.json")
3  with open(emb_file_path, "w") as f:
4      json.dump(new_embeddings, f, indent=2)
5
6  # Calcular centroide y distancia promedio intra-usuario
7  emb_np = np.array(new_embeddings, dtype=np.float32)
8  centroid = robust_mean(emb_np)
9  intra_avg = calcular_distancia_promedio(new_embeddings)
10
11 # Actualizar solo los datos necesarios en centroids.json
12 centroids_index[ci] = {
13     "centroid": centroid.tolist(),
14     "count": len(new_embeddings),
15     "intra_dist_avg": intra_avg
16 }
17
18 with open(CENTROIDS_PATH, "w") as f:
19     json.dump(centroids_index, f, indent=2)
20
21 print(f"\n✅ Embeddings guardados en {emb_file_path}")
22 print(f"✅ Centroides actualizados en {CENTROIDS_PATH}")
23 print(f"🔪 Distancia promedio interna para {ci}: {intra_avg:.4f}")
```

Figura 6.15: Fragmento del código que calcula el centroide facial y actualiza el índice global `centroids.json` con los nuevos embeddings generados Fuente: [Elaboración Propia].

Inicialización del módulo de reconocimiento

El fragmento mostrado a continuación contiene las importaciones esenciales y la configuración inicial del script `face_recognizer`. Se cargan las librerías necesarias para la lectura de imágenes, manejo de archivos y operaciones matemáticas, junto con los módulos de `DeepFace` y los *helpers* locales que permiten la normalización de vectores y el cálculo de la distancia coseno. Además, se definen las rutas de trabajo hacia el directorio de embeddings y el archivo `centroids.json`, el cual contiene la información de referencia para el reconocimiento facial.

```
1  import os
2  import json
3  import numpy as np
4  from deepface import DeepFace
5  import tempfile
6  import cv2
7  from recognition.helpers import l2_normalize, cosine_distance
8
9  # Ruta al índice de centroides generado en train_faces.py
10 EMBEDDINGS_DIR = os.path.join(os.path.dirname(__file__), "embeddings")
11 CENTROIDS_PATH = os.path.join(EMBEDDINGS_DIR, "centroids.json")
```

Figura 6.16: Importaciones y configuración inicial del módulo `face_recognizer`, donde se establecen las rutas a los embeddings y centroides utilizados en la fase de reconocimiento Fuente: [Elaboración Propia].

Carga y normalización de centroides

El siguiente bloque corresponde a la función `load_centroids`, que se encarga de leer el archivo `centroids.json` generado en el proceso de entrenamiento. Si el archivo no existe, el sistema emite una advertencia e indica ejecutar primero el módulo de entrenamiento. En caso contrario, carga los datos, convierte cada centroide en un arreglo NumPy y aplica una normalización L2 para uniformar las magnitudes vectoriales, asegurando coherencia en las comparaciones de similitud facial.

```

1  # Cargar centroides
2  # -----
3  def load_centroids(path=CENTROIDS_PATH):
4      if not os.path.exists(path):
5          print("⚠ No existe centroids.json. Ejecutá primero train_faces.py")
6          return {}
7
8      with open(path, "r") as f:
9          data = json.load(f)
10
11     # Convertir cada centroide a np.array normalizado
12     centroids = {person: l2_normalize(np.array(info["centroid"], dtype=np.float32))
13                  for person, info in data.items()}
14
15     print(f"✅ Centroides cargados: {list(centroids.keys())}")
16     return centroids

```

Figura 6.17: Función `load_centroids` del módulo `face_recognizer`, responsable de cargar y normalizar los centroides almacenados para su uso en el proceso de reconocimiento facial Fuente: [Elaboración Propia].

Reconocimiento individual de rostros

Este fragmento implementa la función `recognize_face`, responsable de identificar un rostro a partir de su recorte facial. El procedimiento crea un archivo temporal con la imagen, genera su embedding mediante el modelo `ArcFace` de `DeepFace` y normaliza el vector resultante. Posteriormente, compara el embedding con los centroides almacenados, calculando la distancia coseno para determinar el rostro más similar. Si la distancia supera un umbral definido, el sistema clasifica el rostro como desconocido. Esta función constituye el núcleo del proceso de reconocimiento facial.

```

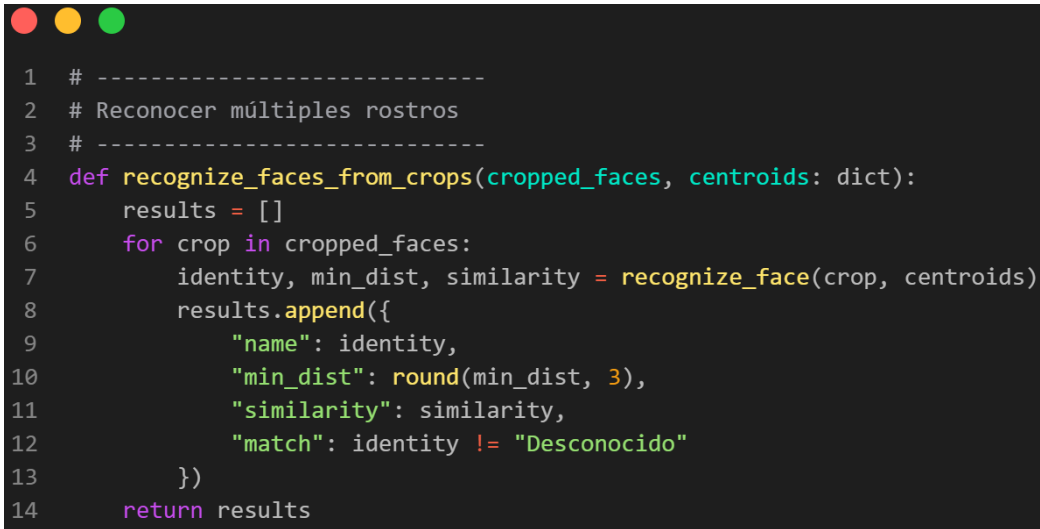
1  # Reconocer un rostro
2  # -----
3  def recognize_face(face_crop, centroids: dict, model_name="ArcFace"):
4      """
5      face_crop: imagen BGR (ej. 160x160) de un rostro ya recortado.
6      centroids: dict {persona: np.array(512,)} cargado con load_centroids()
7      Retorna: (identity, min_dist, similarity)
8      """
9      # Guardar temporalmente para que DeepFace lea la imagen
10     with tempfile.NamedTemporaryFile(suffix=".jpg", delete=False) as tmp:
11         cv2.imwrite(tmp.name, face_crop)
12         rep = DeepFace.represent(
13             img_path=tmp.name,
14             model_name=model_name,
15             enforce_detection=False,
16             detector_backend="skip"
17         )[0]
18         os.remove(tmp.name)
19
20     q = l2_normalize(np.array(rep["embedding"], dtype=np.float32))
21
22     # Buscar el centroide más cercano
23     min_dist = float("inf")
24     identity = "Desconocido"
25     for name, c in centroids.items():
26         d = cosine_distance(q, c)
27         if d < min_dist:
28             min_dist = d
29             identity = name
30
31     # Similitud: inversa de la distancia coseno (0 a 100 aprox)
32     similarity = max(0, 100 - min_dist * 100)
33
34     # Umbral → si min_dist es muy alto, marcar como desconocido
35     if min_dist > 0.45:
36         identity = "Desconocido"
37         similarity = 0
38
39     return identity, float(min_dist), round(similarity, 2)

```

Figura 6.18: Función `recognize_face` del módulo `face_recognizer`, encargada de generar el embedding del rostro y compararlo con los centroides almacenados mediante la distancia coseno Fuente: [Elaboración Propia].

Reconocimiento múltiple de rostros

La siguiente función, denominada `recognize_faces_from_crops`, permite procesar simultáneamente varios rostros detectados dentro de una misma imagen. Para cada recorte facial, el método invoca la función `recognize_face` y recopila los resultados en una lista de diccionarios que incluye el nombre identificado, la distancia mínima, el porcentaje de similitud y un valor booleano que indica si se produjo una coincidencia. Este procedimiento optimiza el rendimiento del sistema al permitir el reconocimiento de múltiples usuarios en una sola captura.

A screenshot of a code editor with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The code is written in Python and defines a function named `recognize_faces_from_crops`. The function takes two arguments: `cropped_faces` and `centroids` (a dictionary). It initializes an empty list `results` and then iterates over each `crop` in `cropped_faces`. For each crop, it calls the `recognize_face` function, passing the crop and centroids. The results of this call (identity, min_dist, similarity) are used to create a dictionary entry for the `results` list. The dictionary contains 'name' (the identity), 'min_dist' (rounded to 3), 'similarity' (the similarity value), and 'match' (a boolean expression `identity != "Desconocido"`). Finally, the function returns the `results` list.

```
1 # -----  
2 # Reconocer múltiples rostros  
3 # -----  
4 def recognize_faces_from_crops(cropped_faces, centroids: dict):  
5     results = []  
6     for crop in cropped_faces:  
7         identity, min_dist, similarity = recognize_face(crop, centroids)  
8         results.append({  
9             "name": identity,  
10            "min_dist": round(min_dist, 3),  
11            "similarity": similarity,  
12            "match": identity != "Desconocido"  
13        })  
14     return results
```

Figura 6.19: Función `recognize_faces_from_crops` del módulo `face_recognizer`, utilizada para procesar múltiples rostros detectados en una misma imagen y devolver sus resultados de identificación Fuente: [Elaboración Propia].

Referencias bibliográficas

- [1] La Nación. (2020) Cómo nació esta tecnología en una tableta de la década del 60. [Accedido: 14 de octubre de 2025]. Disponible: [en línea]. Disponible: <https://www.lanacion.com.ar>
- [2] OSAO México. (2025) Red de aeropuertos de japon instalará 66 puertas automáticas con reconocimiento facial. [En línea]. Consultado en 2025. [en línea]. Disponible: <https://osao.com.mx/aeropuertos-japon-reconocimiento-facial>
- [3] Raghav Sandhane. (n.d.) Figure 3: The relationship between ai, machine learning and deep learning. Academia.edu. [Accedido: 14 de octubre de 2025]. [en línea]. Disponible: <https://www.academia.edu/figures/13243804/figure-3-the-relationship-between-ai-machine-learning-and>
- [4] Interactive Chaos. (2025) Redes neuronales prealimentadas. [Accedido: 14 de octubre de 2025]. [en línea]. Disponible: <https://interactivechaos.com/es/manual/tutorial-de-deep-learning/redes-neuronales-prealimentadas>
- [5] El Laberinto de Falken. (2019) Visión artificial: Redes convolucionales (cnn). [Accedido: 14 de octubre de 2025]. [en línea]. Disponible: <https://www.ellaberintodefalken.com/2019/10/vision-artificial-redes-convolucionales-CNN.html>
- [6] Amazon Web Services. (2025) ¿qué es una red generativa adversarial (gan)? [Accedido: 14 de octubre de 2025].

- [en línea]. Disponible: <https://aws.amazon.com/es/what-is/gan/>. [en línea]. Disponible: <https://aws.amazon.com/es/what-is/gan/>
- [7] Mi De Águila. (2021) Formato de lista de asistencia escolar. [Accedido: 14 de octubre de 2025]. [en línea]. Disponible: <https://mideaguila.blogspot.com/2021/12/formato-de-lista-de-asistencia-escolar.html>
- [8] Sana Fatima, Najmi Ghani Haider, y Rizwan Riaz, “Yolov8 vs retinanet vs efficientdet: A comparative analysis for modern object detection,” *International Journal of Emerging Engineering and Technology (IJEET)*, vol. 3, no. 2, pp. 1–5, 2025. [en línea]. Disponible: <https://grsh.org/journal1/index.php/ijeet/article/view/35>
- [9] Yusepp, “Yolov8-face: Real-time face detection based on yolov8,” <https://github.com/Yusepp/YOLOv8-Face>, 2025, accedido: octubre 2025.
- [10] BeeDigital. (2022) Historia y evolución del reconocimiento facial. [Accedido: 14 de octubre de 2025]. [en línea]. Disponible: <https://www.beedigital.es/tendencias-digitales/historia-y-evolucion-del-reconocimiento-facial>
- [11] Domenic Molinaro. (2022) ¿qué es la seguridad biométrica? [en línea]. Disponible: <https://www.avast.com/es-es/c-what-is-biometric-data>
- [12] Conceptos.es. (2025) Biometría. [Accedido: 14 de octubre de 2025]. [en línea]. Disponible: <https://conceptos.es/biometria>
- [13] Thales Group. (2023) Inspiración en biometría. [en línea]. Disponible: [https://www.thalesgroup.com/es\[...\]](https://www.thalesgroup.com/es[...)

- [14] Bee Safe Security. (2025) Exploring different types of access control credentials. [Accedido: 14 de octubre de 2025]. [en línea]. Disponible: <https://beesafeohio.com/exploring-different-types-of-access-control-credentials/>
- [15] Estudyando.com. (2023) Qué es la seguridad biométrica: definición, sistemas y dispositivos. [en línea]. Disponible: <https://estudyando.com/que-es-la-seguridad->
- [16] Kaspersky. (2025) ¿qué es el reconocimiento facial y cómo funciona? [Accedido: 14 de octubre de 2025]. [en línea]. Disponible: <https://latam.kaspersky.com/resource-center/definitions/what-is-facial-recognition>
- [17] Tecnobits. (2023) Qué es el reconocimiento facial. [Accedido: 14 de octubre de 2025]. [en línea]. Disponible: <https://tecnobits.com/que-es-el-reconocimiento-facial/>
- [18] Microsoft Azure. (2025) What is face recognition? [Accedido: 14 de octubre de 2025]. [en línea]. Disponible: <https://azure.microsoft.com/es-mx/resources/cloud-computing-dictionary/what-is-face-recognition/>
- [19] (2025) Facial recognition. Amazon Web Services. [Accedido: 14 de octubre de 2025]. [en línea]. Disponible: <https://aws.amazon.com/es/rekognition/>. [en línea]. Disponible: <https://aws.amazon.com/es/rekognition/>
- [20] (2025) Artificial intelligence. IBM. [Accedido: 14 de octubre de 2025]. [en línea]. Disponible: <https://www.ibm.com/artificial-intelligence>
- [21] Techopedia. (2023) Inteligencia artificial (ai). [Accedido: 14 de octubre de 2025]. [en línea]. Disponible: <https://www.techopedia.com/es/definicion/inteligencia-artificial-ai>

- [22] E. Acevedo, A. Serna, y E. Serna. (2017) Principios y características de las redes neuronales artificiales. [Accedido: 15 de marzo de 2025]. [en línea]. Disponible: https://www.researchgate.net/publication/331498946_Principios_y_caracteristicas_de_las_redes_neuronales_artificiales
- [23] Google Developers. (2023) ¿qué es el aprendizaje automático? [Accedido: 12 de octubre de 2025]. [en línea]. Disponible: <https://developers.google.com/machine-learning/intro-to-ml/what-is-ml?hl=es-419>
- [24] S. Valverde Bourdié. (2019) Aplicaciones de la inteligencia artificial en la empresa. [Accedido: 16 de marzo de 2025]. [en línea]. Disponible: <https://web.unican.es/Investigacion>
- [25] I. Pérez Borrero y M. E. Gegúndez Arias. (2021) Deep learning: fundamentos, teoría y aplicación. [Accedido: 15 de marzo de 2025]. [en línea]. Disponible: <https://www.torrossa.com/de/resources/an/4947314>
- [26] Amazon Web Services. (2023) ¿qué es el deep learning? [Accedido: 10 de octubre de 2025]. [en línea]. Disponible: <https://aws.amazon.com/es/what-is/deep-learning/>
- [27] J. H. Urgel. (2021) Calibración de combinaciones de redes neuronales profundas convolucionales. [Accedido: 16 de marzo de 2025]. [en línea]. Disponible: <https://repositorio.uam.es/server/api/core/bitstreams/91300f88-b115-4e89-981e-09aee4d8bfd2/content>
- [28] Marvin Minsky y Seymour A. Papert, *Perceptrons: An Introduction to Computational Geometry*. MIT Press, 1969,

- [Accedido: 14 de octubre de 2025]. [en línea]. Disponible: <https://direct.mit.edu/books/monograph/3132/PerceptronsAn-Introduction-to-Computational-Geometry>
- [29] Universidad Nacional Autónoma de México. (2020) Concepto de trabajo. [en línea]. Disponible: https://conceptos sociales.unam.mx/conceptos_final/598trabajo.pdf
- [30] (2025) What is artificial intelligence. Google Cloud. [Accedido: 14 de octubre de 2025]. [en línea]. Disponible: <https://cloud.google.com/learn/what-is-artificial-intelligence>
- [31] S. L. Mora. (2002) Programación de aplicaciones web: historia, principios básicos y clientes web. [Accedido: 28 de marzo de 2025]. [en línea]. Disponible: <https://rua.ua.es/entities/publication/38399900-cf67-4e22-89af-3d45dfff42b8>
- [32] RecFaces. (2023) Visión por computadora: qué es, cómo funciona y soluciones. [Accedido: 15 de octubre de 2025]. [en línea]. Disponible: <https://recfaces.com/es/articles/vision-computador-soluciones>
- [33] Jonny Rodolfo Bastidas Gavilanes. (2019) Registro de asistencia de alumnos por medio de reconocimiento facial utilizando visión artificial. UTA. [en línea]. Disponible: https://repositorio.uta.edu.ec/bitstream/123456789/29179/1/Tesis_t1532masc.pdf
- [34] Tu Registro Escolar. (2023) Los registros académicos. [en línea]. Disponible: <https://www.turegistroescolar.com/software-para-el-registro-escolar/los-registros-academicos/>
- [35] Diego Armando Acosta Ledesma, Felipe Javier Monges Cañete, y José Eduardo Rojas Coppari. (2014) Detección facial mediante opencv. FPU-

- NE. [en línea]. Disponible: <http://servicios.fpune.edu.py:8080/jspui/handle/123456789/428>
- [36] Joel Jhony Sánchez Epping. (2022) Sistema de examen en línea con reconocimiento facial. FPU-NE. [en línea]. Disponible: <http://servicios.fpune.edu.py:8080/jspui/handle/123456789/1147>
- [37] Diego Fabián Centurión Talavera, Carlos Domingo Almeida Delgado, y Jorge Luis Arrúa Ginés. (2019) Sistema biométrico de reconocimiento facial utilizando redes neuronales artificiales en raspberry pi. FPU-NE. [en línea]. Disponible: <http://servicios.fpune.edu.py:8080/jspui/handle/123456789/428>
- [38] Steven Bryan Lara-Jacho, Luis Orlando Albarracín-Zambrano, y Dionisio Vitalio Ponce-Ruiz. (2020) Prototipo de reconocimiento facial para mejorar el control de asistencia de estudiantes en uniandes, quevedo. [en línea]. Disponible: <https://dspace.uniandes.edu.ec/handle/123456789/13197>
- [39] Reconocimiento facial para la automatización del registro de asistencia a clases. UTA. [en línea]. Disponible: [https://hdl.handle.net/11059\[... \]](https://hdl.handle.net/11059[...])
- [40] GoDaddy Inc. (2025) Python: qué es, para qué sirve y cómo empezar a usarlo. [En línea]. Consultado en 2025. [en línea]. Disponible: <https://www.godaddy.com/resources/latam/desarrollo/python-que-es>
- [41] Microsoft. (2025) C++ documentation — learn c++ with visual studio. [En línea]. Consultado en 2025. [en línea]. Disponible: <https://learn.microsoft.com/en-us/cpp/cpp/?view=msvc-170>

- [42] Oracle Corporation. (2025) The java™ tutorials. [En línea]. Consultado en 2025. [en línea]. Disponible: <https://docs.oracle.com/javase/tutorial/>
- [43] Microsoft. (2025) C# guide — learn to use c# and the .net platform. [En línea]. Consultado en 2025. [en línea]. Disponible: <https://learn.microsoft.com/en-us/dotnet/csharp/>
- [44] Python Software Foundation. (2025) Python 3 documentation — an informative overview. [En línea]. Consultado en 2025. [en línea]. Disponible: <https://docs.python.org/3/>
- [45] EBAC. (2025) ¿qué es javascript? [Accedido: 14 de octubre de 2025]. [en línea]. Disponible: <https://ebac.mx/blog/que-es-javascript>
- [46] PostgreSQL Global Development Group. (2025) About postgresql. [Accedido: 14 de octubre de 2025]. [en línea]. Disponible: <https://www.postgresql.org/about/>
- [47] OpenWebinars. (2025) ¿qué es visual studio code y qué ventajas ofrece? [Accedido: 14 de octubre de 2025]. [en línea]. Disponible: <https://openwebinars.net/blog/que-es-visual-studio-code-y-que-ventajas-ofrece/>
- [48] GitHub, Inc., “Acerca de github desktop,” <https://docs.github.com/es/desktop/overview/about-github-desktop>, 2025, [Accedido: 14 de octubre de 2025].
- [49] Equipo de Imagina. (2025) ¿qué es postman y para qué sirve? Imagina Formación. [en línea]. Disponible: <https://imaginaformacion.com/tutoriales/que-es-postman-y-para-que-sirve>
- [50] EBAC. (2025) ¿qué es react? [Accedido: 14 de octubre de 2025]. [en línea]. Disponible: <https://ebac.mx/blog/que->

- es-react. [en línea]. Disponible: <https://ebac.mx/blog/que-es-react>
- [51] Built In. (2025) What is flask? a guide to the python microframework. [Accedido: 14 de octubre de 2025]. [en línea]. Disponible: <https://builtin.com/software-engineering-perspectives/flask>
- [52] Joseph Redmon, Santosh Divvala, Ross Girshick, y Ali Farhadi, “You only look once: Unified, real-time object detection,” *arXiv preprint arXiv:1506.02640*, 2016, [Accedido: 14 de octubre de 2025]. [en línea]. Disponible: <https://arxiv.org/abs/1506.02640>
- [53] Mei Wang y Weihong Deng, “Deep face recognition: A survey,” *Neurocomputing*, vol. 429, pp. 215–244, 2021, [Accedido: 14 de octubre de 2025]. [en línea]. Disponible: <https://arxiv.org/abs/1804.06655>